

Evidence-First Trade-Based Money Laundering (TBML) Triage System

Comprehensive Technical Documentation and Implementation Guide

Author	Jesslyn Baolin Guo
Analytical scope	3 HS6 product families · 2017–2024 · 25,844 annual trade observations
Deployed application	https://tbml-review-triage.streamlit.app/
GitHub repository	https://github.com/jessybaolin/evidence-based-tbml-corridor-drift

HUMAN-REVIEW BOUNDARY

This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Document control

Table F.1. Document control record.

Item	Value
Author	Jesslyn Baolin Guo
Document title	Evidence-First Trade-Based Money Laundering (TBML) Triage System
Document subtitle	Comprehensive Technical Documentation and Implementation Guide
Status	Issued for technical and stakeholder review
Repository	https://github.com/jessybaolin/evidence-based-tbml-corridor-drift
Deployed application	https://tbml-review-triage.streamlit.app/
Analytical scope	3 HS6 product families, 2017–2024, 25,844 annual trade observations
Pipeline configuration version	official-data-simplified-v1 (a pipeline configuration label recorded in <code>configs/project.yml</code>)
Source data provenance	official-derived (filtered extract of official CEPII BACI files); not synthetic fixture data
Intended audience	Wholesale-banking stakeholders; anti-financial-crime and anti-money-laundering practitioners; data analysts and data scientists; model-risk and governance reviewers; technical readers reproducing the work

Human-review conclusion boundary

The sentence below is loaded verbatim from `configs/project.yml` (key `conclusion_boundary`). It is the single approved statement of what any output of this project does and does not establish. It appears on every page of the deployed dashboard, in every generated analyst brief, and it governs the reading of every number in this document.

HUMAN-REVIEW BOUNDARY

This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Loaded verbatim from `configs/project.yml` (`conclusion_boundary`) and reproduced identically in every project output.

Executive summary

What the project is

This project is a working, end-to-end analytical prototype that turns **public, official-derived international trade statistics** into a **ranked queue of unusual exporter–importer–product–year patterns for human review**, with recomputable evidence attached to every ranked record.

It is not a detection system. It does not decide that anything is suspicious. It decides only what an analyst should look at first, and it shows its work.

The implementation covers the full path from source verification to a deployed stakeholder application:

- A verified analytical panel of **25,844 annual trade observations** covering **2017–2024, 4,952 exporter–importer corridors** and **3 HS6 product families**, carrying **\$3.52 trillion** of declared trade value.
- **22 time-safe analytical features**, each computed so that an observation for a given year can only see that year and earlier.
- A controlled **scenario-evaluation layer** with **540 injected rows** across **5 scenario types**, used to compare **5 scoring methods** under a strict train / validation / test time discipline.
- A **Top 50 review queue** of real trade observations, with **200 structured evidence rows — 4 per case —** each recording an observed value, the comparison it was measured against, a threshold, a severity, a plain-English summary, and a mandatory caveat.
- A deployed **multi-page Streamlit dashboard** (10 content pages plus a welcome route) that presents the queue, the per-case evidence, the model-evaluation record, the coverage limits, and a future-state bank-integration design.

What the analysis found

The ranking method works on the controlled test, and the margin over the alternatives is large. On the held-out 2023–2024 test years, the selected blended score placed **24 of the 50 highest-ranked rows** on injected review-priority scenarios (48% precision at 50). Random review of 50 rows would be expected to surface about **0.90**. That is a concentration factor of **26.8×**. The transparent rules baseline reached 14%, logistic regression 6%, and the unsupervised Isolation Forest 6%.

The over-alerting guard held. No injected *hard negative* — an unusual-looking but deliberately benign movement — entered the top 50 on the test split for any method (0.0% hard-negative false-positive rate).

The queue is genuinely in the tail of the population, not merely at the top of a flat distribution. Reconstructing the score for all **23,656 eligible real observations** shows the queue’s median benchmark residual at the **94.0%** percentile of the population, and **18 of the 50** queue rows sit at or above the 95th percentile of their own same-product, same-year peer group.

Data quality and review priority are kept apart, and the separation is measurable. **2,188 observations** carry a valid declared value but no usable quantity, so no implied unit value — and therefore no unit-value score — can be computed for them. They are retained for audit, never scored, and never treated as suspicious. **1,877 of 12,205 gold records (15.38%)** fall in this category, but they represent only **0.0415%** of declared gold value; **99.9585%** of gold value remains assessable. Within that gap, one 2024 ARE → THA record alone accounts for **70.9%** of the excluded value, while **14 corridors** present in all 8 years lack usable quantity in every one of them — a recurring reporting-coverage pattern worth **\$1,000,119** in total, or 0.0845% of the gap. Those are two different control problems and the project reports them separately.

What it cannot conclude

The data is annual, country-to-country, product-level aggregate. It contains no invoices, no shipments, no customers, no payments, no beneficial ownership, and no confirmed money-laundering outcomes. The commodity benchmarks are broad market context, not invoice-level fair value. The scenario labels are evaluation devices, not criminal labels. The scores are ranking signals, not probabilities.

A high rank means one thing only: *this pattern is unusual on measurable public-data signals, and here is exactly why.* Everything after that requires bank-held records and human judgement.

How to use this document

This document serves several audiences at once. Not everyone needs to read it end to end.

Table F.2. Suggested reading paths by role.

If you are...	Read	Then
A wholesale-banking or AFC stakeholder	Executive summary, Chapters 1–4, Chapter 11, Chapter 12	Open the deployed application and work through the Top 50 queue and one selected case
An AFC or AML practitioner	Chapters 1–4, Chapter 9.4, Chapters 10–11	Appendix B for source provenance
A data analyst or data scientist	Chapters 5–10	Appendices A, D and E
A model-risk or governance reviewer	Chapter 4, Chapter 9 in full, Chapter 12	Appendices D and E
A technical reader reproducing the work	Chapter 5, Chapter 13	Appendices C and F

Conventions used throughout

- **Spelling.** British English is used consistently (*prioritisation, analysed, behaviour*). Field names, file names, code identifiers and quoted output are reproduced exactly as they appear in the repository, including their original spelling.
- **Language discipline.** A ranked record is described as an *observed trade record*, a *real trade observation*, or a *public-data observation* — never as *suspicious, fraudulent* or *criminal*. The word *official* describes provenance (*official-derived source data*), not a ranked row.
- **Numbers.** Every project figure in this document is extracted programmatically from the repository at build time. No value is transcribed by hand.
- **Jargon.** Each technical term is explained where it first appears. Chapter 3 is a standing glossary for the domain vocabulary.
- **Cross-references.** Chapter, section, figure and table references are clickable, as are all external URLs.

Callout types

CAUTION

Marks a statement the reader must not over-read — a boundary, a limitation, or a result that is easy to misinterpret.

KEY POINT

Marks the single most important consequence of the surrounding section.

Contents

Document control.....	ii
Executive summary.....	iii
How to use this document	v
Contents	vi

MAIN DOCUMENT

1	Project Overview and Problem Statement	1
1.1	The analytical problem	1
1.2	The grain of one observation.....	1
1.3	The intended review-priority workflow.....	2
1.4	Why a triage queue and not a determination	2
1.5	Unusual pattern versus evidence of TBML	3
1.6	Main deliverables	3
1.7	What "evidence-first" means here	4
2	Why This Matters in Wholesale Banking	5
2.1	Where trade meets the bank.....	5
2.2	Why prioritisation, not more signals	5
2.3	How public trade analytics could complement bank information.....	6
2.4	Five realistic uses inside a bank	6
2.5	What a real review would additionally require	7
2.6	What this project is not	7
3	Domain Primer: TBML, AFC, and Trade Data	8
3.1	Financial-crime vocabulary	8
3.2	Trade-data vocabulary	8
3.3	Reading a corridor observation	10
3.4	Why these three HS6 product families were selected.....	10
3.4.1	The six selection criteria.....	10
3.4.2	The specific analytical role of each family	11
3.5	The FATF–Egmont typology cards	12
4	Assumptions, Boundaries, Limitations, and Non-Goals	13
4.1	Assumptions	13
4.2	Analytical boundaries	13
4.3	Data limitations	14
4.4	Model limitations	14
4.5	Operational limitations	15
4.6	Non-goals.....	15
4.7	Safe language versus language to avoid	16
5	System Architecture and Deployment	17
5.1	The implemented architecture	17
5.2	Pipeline stages	18
5.3	Presentation layer.....	18

5.4	Environment setup	19
5.4.1	Supported Python version.....	19
5.4.2	Two dependency sets.....	19
5.4.3	Creating the environment	19
5.4.4	Validating the environment.....	19
5.4.5	Platform notes.....	20
5.4.6	Main execution commands	20
5.5	Deployment	20
5.5.1	The real deployment flow	20
5.5.2	Deployment facts that the repository does not record.....	21
5.5.3	Development container.....	21
5.6	Where the deployed application lives	21
6	Source Data and Provenance	22
6.1	Four kinds of source, four different roles	22
6.2	Source 1 — CEPII BACI	22
6.3	Source 2 — World Bank commodity benchmarks	23
6.4	Source 3 — Country and HS product-code reference data.....	24
6.5	Source 4 — FATF–Egmont typology publications.....	24
6.6	Chain of custody	25
7	Data Pipeline and Data Construction	26
7.1	Lineage from source file to output artefact.....	26
7.2	Stage 1 — Source verification	26
7.3	Stage 2 — Raw to interim staging.....	26
7.4	Stage 3 — Type and unit normalisation.....	27
7.5	Stage 4 — HS6 handling, country mapping and reference joins.....	27
7.6	Stage 5 — Trade-value conversion and quantity treatment	27
7.7	Stage 6 — Benchmark extraction, normalisation and the gold conversion	28
7.8	Stage 7 — Core derived measurements	28
7.9	Stage 8 — The canonical observation key	29
7.10	Stage 9 — Quality statuses, exclusion logic and the audit trail	29
7.11	Stage 10 — The data-quality score	30
7.12	Stage 11 — Model eligibility	31
7.13	Stage 12 — Data lineage carried on every row.....	31
7.14	Stage 13 — Output artefacts	31
7.15	Panel composition	31
8	Time-Safe Feature Engineering	33
8.1	What "time-safe" means and why it is enforced	33
8.2	Notation.....	34
8.3	Level features	34
8.3.1	Implied unit value and its logarithm	34
8.3.2	Benchmark price normalisation	35
8.3.3	Benchmark residual.....	35
8.4	History features	36
8.4.1	Shifted corridor historical median.....	36
8.4.2	Shifted corridor historical median absolute deviation	36
8.4.3	Robust historical z-score	36

8.4.4	Corridor activity history, novelty and reactivation	37
8.5	Peer features	38
8.5.1	Same-family, same-year peer percentile	38
8.6	Change features	39
8.6.1	Trade-value, quantity and unit-value change	39
8.6.2	Value–quantity divergence	40
8.6.3	Benchmark-adjusted drift	41
8.6.4	Benchmark year-over-year change	41
8.6.5	Benchmark consistency gap	42
8.7	Data-condition features	42
8.8	Feature coverage in the current run	45
8.9	The rules-based ranking components	46
8.9.1	Component normalisation	46
8.9.2	The positive subtotal	47
8.9.3	The two deductions	47
8.9.4	The rules score	48
8.10	The hybrid review-priority score	48
8.11	Evaluation metrics	48
9	Model Development, Evaluation, and Selection	50
9.1	Scenario layer for model evaluation	50
9.1.1	Why confirmed labels are unavailable	50
9.1.2	Why the scenario layer comes after the panel is frozen	50
9.1.3	How the scenarios are constructed	50
9.1.4	How hard negatives test benign but unusual movement	51
9.1.5	How labels are kept separate from features and evidence	51
9.1.6	Splits and leakage controls	51
9.1.7	Scenario limitations	52
9.2	Model comparison and selection	52
9.2.1	The five scorers	52
9.2.2	XGBoost hyper-parameter selection	52
9.2.3	Challenger selection — on validation only	53
9.2.4	Hybrid weight selection — also on validation only	53
9.2.5	Why validation selects and the test set waits	53
9.2.6	Held-out test results	53
9.2.7	The selected approach and why it was selected	54
9.2.8	Weaknesses and governance considerations	55
9.2.9	Model contribution — global SHAP	55
9.3	Application of the selected model to real trade observations	56
9.3.1	How the selected model transfers	56
9.3.2	Which records are eligible, and which are not	57
9.3.3	How rankings are produced	57
9.3.4	Why real records do not inherit synthetic labels	57
9.3.5	How the Top 50 queue is created	57
9.3.6	What the score does and does not mean	58
9.4	Evidence-grounded review of the Top 50 ranked queue records	58
9.4.1	From score to evidence	58
9.4.2	Evidence types and thresholds	59
9.4.3	A worked example — rank 1	60
9.4.4	Data-quality caveats attached to ranked records	61
9.4.5	Benign explanations that must be considered first	61

9.4.6	Information that is missing and must be obtained	62
9.4.7	Recommended human-review steps.....	62
9.4.8	How typology context stays context.....	62
9.4.9	Why SHAP is explanation, not evidence.....	62
9.4.10	The analyst-brief layer.....	63
10	Comprehensive Data Analytics and Profiling	64
10.1	Business review and data profiling	64
10.1.1	Reconstructing the full scored population	64
10.1.2	Is the queue in the tail of the population?.....	64
10.1.3	Queue composition by product family.....	65
10.1.4	Year concentration	66
10.1.5	Benchmark trends and observation frequency	67
10.1.6	Positioning one case against its full peer population	68
10.1.7	Peer positioning across the whole queue	69
10.1.8	What actually distinguishes the queue	70
10.1.9	Do the drivers behave the same way in every commodity?	71
10.1.10	Where does each queue row sit in its family's residual distribution?	72
10.1.11	How concentrated is trade across corridors?	73
10.1.12	How much support sits behind an observation?.....	74
10.1.13	Is usability evenly distributed?	74
10.1.14	Which countries appear in the current queue?	75
10.1.15	Reading the evaluation metrics as a stakeholder.....	76
10.1.16	Global model contribution	76
10.2	Gold missing-quantity analysis	76
10.2.1	The question the anomaly model cannot answer	76
10.2.2	Scale of the gap	77
10.2.3	Comparison with palm oil and copper	77
10.2.4	Impact on unit-value calculation	79
10.2.5	Impact on model eligibility — verified, not assumed.....	79
10.2.6	Is trade value still analytically useful?	79
10.2.7	Why missing quantity is a data-quality condition, not a signal	79
10.2.8	Trend over time	80
10.2.9	Concentration behind the dollar headline	80
10.2.10	Persistence versus materiality — two different control questions.....	81
10.2.11	Structure of the persistent gap	83
10.2.12	How unscored records are retained	84
10.2.13	Recommended handling in a bank implementation	84
10.2.14	Reproducibility of this analysis.....	84
10.2.15	What this extension adds.....	85
10.3	Chapter findings at a glance	85
11	Dashboard and Analyst Workflow	86
11.1	What the dashboard is, and what it is not.....	86
11.2	Architecture, routing and the boundary ribbon	86
11.3	Failure behaviour — never substitute data	88
11.4	The analyst journey	88
11.5	Product: Evidence-first Top 50 ranked queue	89
11.5.1	Purpose	89
11.5.2	Filters and ranking controls	89
11.5.3	Default view and broader views.....	90

11.5.4	Columns displayed	90
11.5.5	How the review-priority score is explained in the product	90
11.5.6	Data-quality and evidence indicators.....	91
11.5.7	Export	91
11.5.8	Row selection	91
11.6	Selected Case Review	91
11.6.1	Case identity and observed values	91
11.6.2	Comparison views	93
11.6.3	Why It Ranked High	93
11.6.4	Caveats	93
11.6.5	Benign explanations, missing information and review actions	93
11.7	Unscored gold records.....	94
11.7.1	Why these records cannot receive a score.....	94
11.7.2	Why they are retained.....	94
11.7.3	What information remains available	94
11.7.4	How the dashboard separates data quality from anomaly ranking	94
11.7.5	How a bank could investigate or enrich these records.....	95
11.8	Model Evaluation & Controls	96
11.9	Supporting pages	96
11.10	Test coverage	97
12	Bank Implementation Pathway	98
12.1	The shape of the target state.....	98
12.2	Phase 1 — Controlled analytical proof of concept	98
12.3	Phase 2 — Integration with internal trade-finance and payment data	99
12.4	Phase 3 — Analyst feedback and label development.....	99
12.5	Phase 4 — Model governance and validation	99
12.6	Phase 5 — Controlled pilot.....	100
12.7	Phase 6 — Production monitoring and continuous improvement	100
12.8	Where an AI assistant could help — and where it must not	101
12.8.1	The concept mockup, as it appears in the product	102
12.9	Summary of the pathway	103
13	Reproduction Workflow	104
13.1	Clone the repository	104
13.2	Create the environment and install dependencies.....	104
13.3	Prepare the source files	104
13.4	Validate the environment	104
13.5	Run the data pipeline	105
13.6	Run the notebook analyses (optional).....	105
13.7	Verify the outputs.....	105
13.8	Run the dashboard	106
13.9	Troubleshooting.....	106
14	Conclusion	108
14.1	What the project demonstrates	108
14.2	What the current result is useful for	108
14.3	What remains outside its scope	108
14.4	How evidence-first design improves interpretability and governance	109

14.5	What would be needed for bank implementation	109
14.6	The boundary	109

APPENDICES

Appendix A	Comprehensive Data Dictionary	110
A.1	Identity and provenance fields.....	110
A.2	Source and derived measurement fields	110
A.3	Time-safe feature fields.....	111
A.4	Data-quality and eligibility fields.....	112
A.5	Score and queue fields.....	113
A.6	Evidence fields	114
A.7	Scenario-evaluation fields.....	115
Appendix B	Data Sources and Download Links	116
Appendix C	GitHub Repository and Deployed Application.....	118
C.1	Repository structure	118
C.2	Deployment notes.....	119
Appendix D	Formula Reference	120
D.1	Symbols	120
D.2	Panel construction	120
D.3	Time-safe features.....	121
D.4	Scoring.....	123
D.5	Evaluation and evidence	124
Appendix E	Essential Functions	125
E.1	Time-safe history construction.....	125
E.2	Time-safe change computation.....	125
E.3	Scenario separation and label isolation	125
E.4	The feature-frame leakage guard.....	126
E.5	Model selection on validation only	126
E.6	Evidence construction	127
E.7	Validation boundaries — the ranking metric	128
Appendix F	Artefact and File Inventory	129
F.1	Analytical pipeline.....	129
F.2	Configuration	129
F.3	Data.....	129
F.4	Dashboard	131
F.5	Reports and notebooks	131
F.6	Tests	132
F.7	Root-level files.....	132

FIGURES

Figure 1.1	The intended review-priority workflow.....	2
Figure 3.1	Anatomy of one corridor–product–year observation, from the two published source fields throug....	10
Figure 5.1	System architecture of the implemented project.	17
Figure 5.2	The verified deployment path.....	20

Figure 7.1	Data pipeline and file lineage.....	26
Figure 7.2	Quality-status assignment.....	29
Figure 7.3	Panel composition — observation counts and declared trade value by product family and year.....	32
Figure 8.1	Time-safe feature construction for one corridor–product series.....	33
Figure 8.2	Feature coverage across the 25,844 feature rows.....	46
Figure 9.1	Model comparison on the held-out 2023–2024 test years.....	54
Figure 9.2	Global mean absolute SHAP contribution per feature for the XGBoost challenger, over a fixed 5....	56
Figure 10.1	Review-priority score across the full eligible real population by product family, with the To....	65
Figure 10.2	Queue composition adjusted for population share.....	66
Figure 10.3	Top 50 review candidates by year and product family.....	67
Figure 10.4	Annual World Bank benchmark price and observation count by product family, 2017–2024.....	68
Figure 10.5	The rank-1 observation positioned within the full 2022 Non-monetary unwrought gold peer popul....	69
Figure 10.6	Every Top 50 row positioned within its own same-family, same-year peer distribution.....	69
Figure 10.7	Distribution of four key drivers: the Top 50 against the rest of the eligible population.....	71
Figure 10.8	The four key drivers split by product family.....	72
Figure 10.9	Distribution of the benchmark residual across all panel observations in each family, with eve....	73
Figure 10.10	Cumulative share of each family's declared trade value against corridors ranked by value.....	73
Figure 10.11	Left: how many prior observed years each observation's corridor–product had, which is the dep.....	74
Figure 10.12	Quality-status composition of the panel.....	75
Figure 10.13	Exporters and importers appearing most often in the current Top 50 queue.....	75
Figure 10.14	Quantity-coverage gap by product family on two denominators.....	78
Figure 10.15	Share of rows without a usable quantity, by year and product family.....	78
Figure 10.16	Gold quantity-coverage gap by year.....	80
Figure 10.17	Pareto view of the gold coverage-gap value.....	81
Figure 10.18	Persistence against materiality for affected gold corridors.....	82
Figure 10.19	The 14 persistent gold coverage-gap corridors as a directed network.....	83
Figure 11.1	The deployed application, Top 50 Review Queue page.....	87
Figure 11.2	The end-to-end analyst journey through the deployed application.....	88
Figure 11.3	The Top 50 Review Queue grid as the deployed application renders it, at full column width.....	89
Figure 11.4	The deployed application, Selected Case Review page.....	92
Figure 11.5	The deployed application, Unsourced Gold Records page.....	95
Figure 11.6	The deployed application, Model Evaluation & Controls page — the governance surface.....	96
Figure 12.1	Present state and target state.....	98
Figure 12.2	The deployed application, Bank Implementation Pathway page — the bounded role for an AI assis....	101
Figure 12.3	The AI evidence-assistant concept mockup, captured from the deployed application's Bank Imple.....	102

CHAPTER 1

Project Overview and Problem Statement

1.1 The analytical problem

International trade generates an enormous volume of legitimate activity and a small, unknown volume of activity that abuses trade to move value. Trade-based money laundering — explained in full in Chapter 3 — exploits the price, quantity, or description of traded goods to disguise the movement of funds. It is difficult to see for three structural reasons:

1. **Volume.** The number of exporter–importer–product flows in any commodity is far larger than any team can inspect. This project’s deliberately narrow slice — three products over eight years — already contains 25,844 annual observations across 4,952 corridors.
2. **Price opacity.** “Correct” price is not a single number. Grade, purity, freight, insurance, delivery terms, contract timing and hedging all move a legitimate price. There is no public reference against which an individual invoice can be scored.
3. **Fragmented visibility.** No single institution sees the whole chain. A bank sees its own customers’ payments and documents. Customs sees declarations. A statistical agency sees aggregates. Nobody sees all three for the same shipment.

The consequence for an anti-financial-crime team is not a shortage of signals. It is the opposite: too many undifferentiated signals, and no defensible basis for choosing which to work first. An anomaly score with no explanation makes this worse, because the analyst must reconstruct why the alert exists before the review can even begin.

This project addresses the prioritisation problem, not the detection problem. It asks: *given only public trade statistics and public commodity benchmarks, which corridor–product–year patterns most deserve an analyst’s limited time, and what exactly is the evidence for that judgement?*

1.2 The grain of one observation

Everything in this project is built on one unit of analysis, and being precise about it prevents most misreadings.

exporter country × importer country × HS6 product × calendar year

one row = one whole year of trade between two countries in one product group

One observation is the annual aggregate of **all trade** in one HS6 product, from one exporter country to one importer country, in one calendar year. It is identified by the deterministic key `obs_id`, derived from `(year, exporter_code, importer_code, hs6)`.

Table 1.1. What one observation is, and what it is not.

One observation is	One observation is not
An annual country-to-country product total	An invoice, a contract, or a purchase order
Declared trade value in nominal US dollars	A shipment, a container, or a bill of lading
Declared quantity in metric tons, where reported	A customer, an account, or a counterparty
A public statistical aggregate, harmonised across reporters	A payment, a transfer, or a bank transaction
Repeatable, traceable to a hashed source row	A confirmed or alleged money-laundering case

Because the grain is an annual aggregate, a single observation can contain many underlying shipments at many different prices. The *implied unit value* the project computes — declared value divided by declared quantity — is therefore an **average across everything that flowed that year**, not a price anyone actually paid.

1.3 The intended review-priority workflow

The project produces a triage queue and hands it to a human. The full intended workflow has five steps, of which the project implements the first three.



Figure 1.1. The intended review-priority workflow. The project implements steps 1–3; steps 4 and 5 require bank-held records and human judgement and are outside its scope.

- 1. Screen systematically.** Every eligible observation in scope receives the same treatment, computed the same way, with the same time discipline. 23,656 of 25,844 observations (91.5%) are eligible for scoring.
- 2. Rank by review priority.** A blended score orders the eligible population. The top 50 become the review queue.
- 3. Attach evidence.** Each queued record carries 4 structured evidence rows naming the metric, its observed value, the comparison group, the threshold, the severity and a required caveat — plus the possible benign explanations and the information that is missing.
- 4. Human review against private records** (*outside this project*). The analyst tests the public-data pattern against customer files, trade documents, shipment records and payments.
- 5. Disposition** (*outside this project*). The analyst decides, records the outcome, and closes or escalates.

Steps 4 and 5 are deliberately absent. The project has no access to the records those steps require, and a public-data prototype that pretended otherwise would be misleading.

1.4 Why a triage queue and not a determination

A determination of suspiciousness requires facts this project structurally cannot observe. The distinction matters enough to state it as a design rule rather than a caveat.

Table 1.2. Why the output is a queue rather than a determination.

To determine suspicion you need	The project has	Available in this project?
The parties to the trade	Country codes only	No
The invoice and its terms	An annual value aggregate	No
Proof the goods moved	No shipment records at all	No
The payment and its route	No payment data	No
Beneficial ownership	None	No
A confirmed outcome to learn from	No labels exist in the public data	No

To determine suspicion you need	The project has	Available in this project?
A measurable, reproducible pattern	Time-safe features over verified public data	Yes

The last row is what the project delivers. Everything above it is what a bank would supply. The scoring output is therefore always described as a **review-priority score** — a ranking signal — and never as a probability, a likelihood, or a finding.

1.5 Unusual pattern versus evidence of TBML

The most important distinction in the entire document is the one between *unusual* and *illicit*.

An observation can be unusual for many ordinary reasons: a change in product grade or specification within the same HS6 code; a shift in delivery terms that moves freight and insurance into or out of the declared value; a re-export or transshipment flow booked through a hub; a commodity price shock; a small residual flow where a large denominator makes the arithmetic volatile; a change in a national reporting convention. None of these is wrongdoing, and all of them can produce exactly the same signature in aggregate public data as a valuation abuse.

CAUTION

An unusual public-data pattern and evidence of trade-based money laundering are different things. The first is what this project can measure. The second requires records the project does not have. A high review-priority score is a reason to look, never a conclusion about what will be found.

The project encodes this in four places, so it cannot be lost when an output is copied out of context:

- The conclusion boundary sentence is stored once in `configs/project.yml` and reproduced verbatim in every generated brief, every report, and on every dashboard page.
- Every evidence row carries a mandatory `caveat` field; there is no evidence row without one.
- `src/08_build_briefs.py` validates every generated brief against a prohibited-phrase list and fails any brief that asserts crime, fraud, or confirmed TBML.
- The dashboard maintains an explicit allowed / not-allowed interpretation-language pair on its Model Evaluation & Controls page.

1.6 Main deliverables

Table 1.3. What the project produces.

Deliverable	Artefact	Contents
Verified source record	<code>data/outputs/source_manifest.json</code> , <code>source_file_inventory.csv</code>	SHA-256 inventory of 8 declared inputs; cross-checks of years, product codes and row counts against the publisher's own provenance notes
Clean analytical panel	<code>data/processed/corridor_product_year_panel.parquet</code>	25,844 rows × 50 columns, with quality status, eligibility and a data-quality score on every row
Exclusion audit	<code>data/processed/exclusion_audit.csv</code>	13,950 rows: every observation that is caveated or excluded from modelling, with its reason
Time-safe features	<code>data/processed/corridor_features.parquet</code>	22 model/rule input features per observation
Scenario-evaluation layer	<code>scenario_panel.parquet</code> , <code>scenario_labels.parquet</code> , <code>scenario_injections.csv</code> , <code>scenario_split_manifest.json</code>	540 controlled injections with labels held in a separate file

Deliverable	Artefact	Contents
Model comparison record	<code>model_comparison.csv</code> , <code>model_selection.json</code> , <code>hybrid_validation_candidates.csv</code> , <code>xgboost_parameters.json</code>	Every metric for 5 methods by split and product family; the selection decision and its basis
Review queue	<code>data/outputs/top_ranked_corridors.csv</code>	Top 50 real observations, 21 columns each
Evidence table	<code>data/outputs/evidence_table.csv</code>	200 recomputable evidence rows over 50 cases
Caveated analyst briefs	<code>data/outputs/analyst_briefs.json</code> , <code>reports/analyst_briefs.md</code>	5 deterministic briefs, each validated against 7 checks
Deployed application	https://tbml-review-triage.streamlit.app/	10 content pages plus a welcome route, reading the artefacts above

1.7 What “evidence-first” means here

The phrase is a design constraint, not a slogan. It means that no statement about a ranked record exists in the system unless a structured, recomputable fact backs it.

Concretely: `src/07_build_evidence.py` does not summarise a score. It re-reads the feature table, selects the strongest metrics for each queued observation, and emits one row per firing metric containing the observed value, the field it was compared against, the comparison group, the numeric threshold, the direction, a severity derived from how far past the threshold it sits, a fixed-wording plain-English sentence, and a caveat. Every evidence row has a content-derived `evidence_id`, so the same observation, metric and value always produce the same identifier across runs.

The analyst brief layer then cites those identifiers. `src/08_build_briefs.py` validates that every evidence ID cited exists, that no numeric claim is unsupported, that no source is invented, that the caveats are present, that the exact boundary sentence appears, and that no prohibited phrase is used. All 5 briefs pass all checks.

The result is that a reviewer can take any sentence in any output and trace it back to a hashed source row.

CHAPTER 2

Why This Matters in Wholesale Banking

2.1 Where trade meets the bank

Wholesale banking sits inside international trade at several points at once, and each point carries a different kind of exposure to trade-based abuse.

Table 2.1. Wholesale-banking touchpoints with international trade.

Touchpoint	What the bank does	Where trade data is relevant
Documentary trade finance	Issues, advises and confirms letters of credit; handles documentary collections; examines documents against terms	The declared goods, price, quantity and terms on the documents are exactly the fields a valuation abuse would distort
Open-account and supply-chain finance	Finances receivables and payables without examining shipping documents	Far less document visibility, so external corridor and commodity context carries more weight
Cross-border payments	Executes and receives customer payments tied to trade	Payment value and counterparty can be compared with what the trade should plausibly cost
Correspondent banking	Clears payments for other banks' customers	The bank sees a payment but often not the underlying trade at all
Treasury and FX	Provides hedging and currency conversion for trade flows	Hedging and contract timing are common benign explanations for apparent price movement
Relationship and credit	Underwrites and monitors trading customers	Expected trade routes and commodities form the baseline that an unusual corridor is measured against

The common thread is that a bank's own view of a trade is partial. External corridor and commodity context is one of the few ways to ask whether a customer's declared activity is consistent with how that route and that product generally behave.

2.2 Why prioritisation, not more signals

An AFC team's binding constraint is analyst hours, not alert generation. Adding signals without ordering them makes the constraint worse: more alerts, the same capacity, and a lower average value per review.

Three properties determine whether a queue is actually useful:

- **Concentration.** Does reviewing the top of the queue find materially more than reviewing at random? On this project's controlled test, the selected method's top 50 was **26.8× more concentrated** than random selection.
- **Discipline under look-alikes.** Does the queue stay quiet when something is unusual but benign? The project tests this explicitly with *hard negatives* — injected movements that look dramatic but are consistent with the market or leave unit value unchanged. No hard negative entered the top 50 on the test split.
- **Reviewability.** Can the analyst see immediately why a row is there? Each queued record arrives with 4 named, thresholded, caveated evidence rows rather than a bare number.

KEY POINT

The value proposition is a change in the analyst's opening question. Without evidence-first design the analyst asks "why does this alert exist?" and must reconstruct the reasoning before reviewing. With it, the analyst asks "which of these stated facts should I verify first?"

2.3 How public trade analytics could complement bank information

Public trade statistics and bank records answer different questions, and their usefulness comes from the combination rather than from either one.

Table 2.2. Complementary strengths of public trade data and bank-held records.

Question	Public trade data	Bank-held records
Is this corridor–product pattern unusual against its own history and its peers?	Yes	No — the bank sees only its own customers
How does the aggregate price compare with the world market?	Yes	Partially, per deal
Who are the parties, and is this activity expected of them?	No	Yes
Do the contract, invoice and shipping documents agree?	No	Yes
Did payment follow the trade, through which banks, in what currency?	No	Yes
Who ultimately owns and controls the counterparties?	No	Yes
Is there a sanctions or adverse-information issue?	No	Yes

CAUTION

Public trade analytics can complement bank information. It cannot replace it. Nothing in this project observes a customer, an account, a document or a payment, and no ranking produced from public data is sufficient for a disposition decision.

2.4 Five realistic uses inside a bank

The project supports five concrete uses without requiring any claim that it detects financial crime.

1. Typology exploration. The five FATF–Egmont typology cards in `configs/typology_context.yml` map recognised TBML contexts — valuation, high-value/low-volume goods, phantom shipments, benign market shocks, missing private information — onto the specific features this project computes. Each card carries an explicit *allowed language* and *prohibited language* pair, giving a team a vocabulary for discussing patterns without overstating them.

2. Investigative triage. The Top 50 queue plus its evidence gives an analyst a starting position and a list of the checks that public data cannot perform. That is directly reusable as a review checklist.

3. Model governance. The project demonstrates a defensible evaluation design under a hard constraint (no real labels): a frozen clean population, controlled scenarios with hard negatives, time-based splits, selection on validation only, and a test set opened once. Chapter 9 documents the whole record. A model-risk function can review the design even though the labels are synthetic.

4. Analyst enablement. The feature explanation table, the per-case signal profile and the plain-English evidence sentences are training material for what “unusual” means in aggregate trade data, and for how quickly a plausible benign explanation appears.

5. Stakeholder communication. The deployed application gives a non-technical audience a concrete artefact — a queue, a case, an evidence panel, a coverage limitation — instead of an abstract description of an approach.

2.5 What a real review would additionally require

If a ranked record from this project were handed to a bank analyst, the review could not begin without the following. This list is the direct inverse of the project's data limitations, and it is what Chapter 12's implementation pathway is built to supply.

Table 2.3. Bank-held information required before any review conclusion.

Category	Specific records
Customer and counterparty	KYC/CDD file, expected activity profile, relationship history, jurisdictions of operation
Trade documents	Contract, commercial invoice, packing list, certificate of origin, quality/assay certificate, Incoterms
Physical movement	Bill of lading, airway bill, customs declaration and inspection record, warehouse receipts
Payment	Payment instructions and confirmations, correspondent routing, currency, value dating, third-party payers
Ownership	Beneficial-ownership structure, connected entities, control relationships
Screening	Sanctions, PEP and adverse-information results for all parties and connected entities
Case history	Prior alerts, prior reviews, prior dispositions, prior filings

HUMAN-REVIEW BOUNDARY

This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Loaded verbatim from `configs/project.yml ( conclusion_boundary )` and reproduced identically in every project output.

2.6 What this project is not

CAUTION

The implementation in this repository is an analytical prototype built on public data. It is not a production bank monitoring platform. It has no customer data, no case-management integration, no access control, no audit logging of user actions, no model-monitoring loop, no approved governance record, and no live data feed. Chapter 12 sets out what would be required to change that; Chapter 4 states the limitations in full.

CHAPTER 3

Domain Primer: TBML, AFC, and Trade Data

This chapter defines every term the rest of the document relies on. Readers already fluent in both the anti-financial-crime vocabulary and the trade-statistics vocabulary can skip to Section 3.4, which explains the specific product selection.

3.1 Financial-crime vocabulary

Trade-based money laundering (TBML). The use of trade transactions to disguise the movement of value or the origin of funds. The recognised techniques operate on what is written in trade documents rather than on the money itself: *over-invoicing* (declaring a price above true value, moving value to the exporter), *under-invoicing* (the reverse), *multiple invoicing* (billing one shipment repeatedly), *over- or under-shipment* (declaring quantities that do not match what moved), *phantom shipment* (documenting goods that never moved at all), and *misdescription* (declaring goods as something they are not). What unites them is a mismatch between a document and reality — which is why detecting them normally requires the documents.

Anti-financial crime (AFC). The umbrella function within a bank covering money laundering, terrorist financing, sanctions, bribery and corruption, fraud, and tax-evasion facilitation. It is broader than AML alone and is the organisational home of the work this project would support.

Anti-money laundering (AML). The specific legal and regulatory regime requiring institutions to know their customers, monitor activity, investigate the unusual, and report the suspicious to a financial intelligence unit. AML is a compliance obligation; AFC is the wider risk function that discharges it.

FATF–Egmont typology context. The Financial Action Task Force is the intergovernmental body that sets AML/CFT standards. The Egmont Group is the network of national financial intelligence units. Their two joint publications used here — *Trade-Based Money Laundering: Trends and Developments* (2020) and *Trade-Based Money Laundering: Risk Indicators* (2021) — describe how TBML works and which conditions are worth attention.

CAUTION

In this project FATF–Egmont material is **typology and caveat context only**. It is never a model input, never a label, never row-level evidence, and never proof of suspicious activity. `src/01_verify_sources.py` opens both PDFs solely to confirm they are readable (66 and 10 pages respectively); no text is parsed for analytical use.

3.2 Trade-data vocabulary

Trade corridor. A directed exporter-to-importer country pair, written here as `EXP→IMP` (for example `ESP→NLD`). It is a route, not a relationship or an entity. The panel contains 4,952 distinct corridors across 202 exporter and 213 importer codes.

HS and HS6. The Harmonized System is the World Customs Organization’s standard product nomenclature. Codes are hierarchical: two digits give the chapter, four the heading, six the subheading. HS6 is the finest level that is internationally comparable — below it, countries add their own digits. This project uses HS revision HS17.

KEY POINT

An HS6 code is a product **group**, not a product. Within one HS6 code, grade, purity, physical form, packaging, product mix and specification can all vary. This is the single most important source of benign variation in everything that follows.

Annual bilateral trade observation. One row: one exporter, one importer, one HS6 code, one calendar year. It aggregates every flow meeting that description over the whole year.

Trade value. The declared value of trade. The source publishes it in thousands of current US dollars; the pipeline converts it to nominal US dollars. Values are *nominal* — not adjusted for inflation — so a comparison across years mixes real change with price-level change.

Quantity. The declared physical quantity, published in metric tons. Quantity is **missing for some observations**: 2,188 of 25,844 raw rows carry no quantity. Missing quantity is a reporting condition, not a signal.

Aggregate (implied) unit value. Declared value divided by declared quantity, in US dollars per metric ton. The word *implied* is deliberate: it is arithmetic over an annual aggregate, not an observed transaction price.

Benchmark price. An annual world-market commodity price published by the World Bank in its Commodity Markets Outlook (“Pink Sheet”) historical series. It provides broad market context for the level and direction of a commodity’s price in a given year. It is not a landed cost, not a contract price, and not a legal reference price.

Benchmark residual. The natural-log difference between an observation’s implied unit value and the same-year benchmark for its product family. Zero means parity with the benchmark; positive means above; negative means below. Working in logs makes upward and downward departures symmetric and comparable across commodities whose price levels differ by four orders of magnitude.

Time-safe features. A feature is time-safe if computing it for year *Y* uses only information from year *Y* and earlier. Historical baselines are built from **strictly prior** years, peer comparisons use only the same year, and no future year ever enters. Without this discipline an evaluation measures hindsight rather than a method that could have run at the time.

Scenario labels. Because no confirmed TBML labels exist in public trade data, the project constructs its own evaluation targets by transforming a copy of the frozen clean panel in controlled ways. `synthetic_review_priority = 1` marks a row transformed into a pattern the method should rank highly. These are evaluation devices; they are not criminal labels and never touch the real review queue.

Hard negatives. Injected rows that look unusual but are deliberately benign — a movement that stays in line with the same year’s market benchmark, or value and quantity scaled together so the unit value barely changes. They test over-alerting: a method that ranks these highly would generate false work in production.

Review-priority score. A number in the range 0 to 1 used **only to order** the eligible population. It is not a probability, not a likelihood, and not a finding. `data/outputs/model_selection.json` records this in the pipeline itself: “*Scores are review-priority ranking scores, not calibrated probabilities of crime.*”

Evidence rows. Structured records — one per firing metric per queued observation — giving the metric name, the observed value, the comparison value and group, the threshold, the direction, a severity, a fixed-wording plain-English sentence, and a mandatory caveat. The project produces 200 of them over 50 cases.

Data-quality score. An additive 0-to-6 confidence measure over quantity validity, benchmark match, history depth and provenance completeness. It measures **how much can be said** about an observation, never how unusual it is. The mean across the panel is 4.51.

KEY POINT

Data quality and review priority are separate axes and are kept separate everywhere in the implementation. A poorly supported row is ranked *more cautiously* — the quality deduction can only reduce a rules score, never raise it — and a missing quantity removes a row from scoring altogether rather than making it look unusual.

3.3 Reading a corridor observation

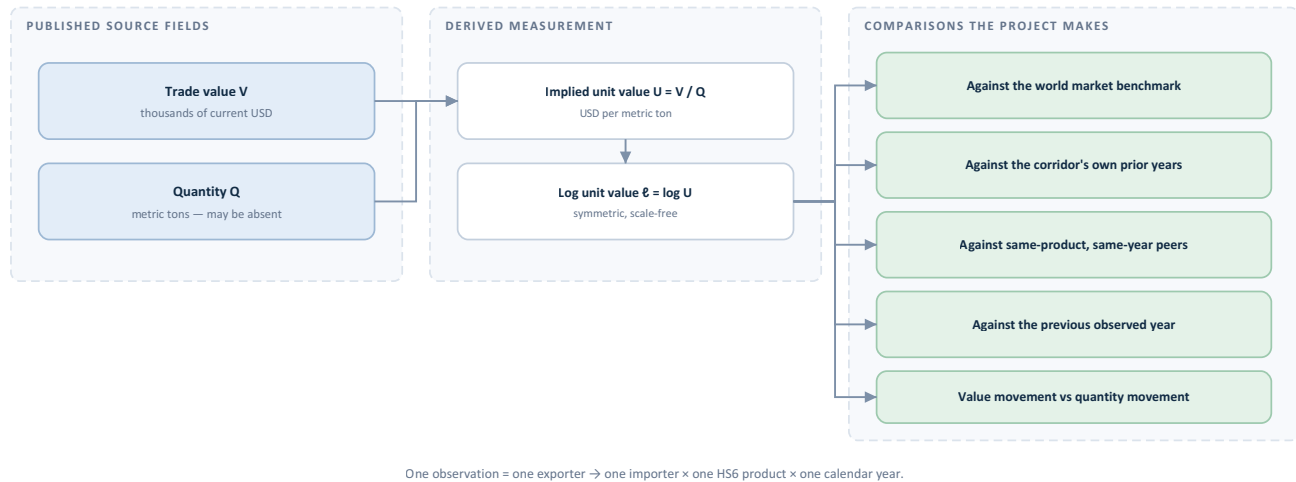


Figure 3.1. Anatomy of one corridor–product–year observation, from the two published source fields through the derived measurements to the comparisons the project makes.

3.4 Why these three HS6 product families were selected

The project deliberately restricts itself to three HS6 codes. The choice is a **contrastive analytical design**: the three commodities differ enough that a method which works across all of them is unlikely to be an artefact of one commodity's behaviour.

Table 3.1. The three selected HS6 product families.

HS6	Product	Project role	Benchmark series	Expected quantity unit
151110	Crude palm oil	Asia bulk-commodity anchor	world_bank_palm_oil	metric ton
740311	Refined copper cathodes	Industrial commodity anchor	world_bank_copper	metric ton
710812	Non-monetary unwrought gold	Typology anchor	world_bank_gold	metric ton

3.4.1 The six selection criteria

1. TBML typology relevance. FATF–Egmont material identifies high-value goods with wide pricing margins — precious metals prominently — as recurring typology context, and identifies valuation inconsistency as a recurring technique. Gold provides the typology-relevant stress case. Palm oil and copper provide the ordinary cases against which the method must stay disciplined.

2. Regional and wholesale-banking relevance. All three are financed through wholesale banking channels: documentary trade finance, commodity trade finance, cross-border payments and treasury services. Palm oil is a Southeast Asian bulk export of direct regional relevance; copper is a globally traded industrial input; gold moves through a small number of refining, vaulting and trading hubs.

3. Availability of a credible external benchmark. Every family must have a published, independent annual world price to compare against. All three appear in the World Bank Pink Sheet annual series, which is what makes the benchmark residual computable at all. A commodity without a credible public benchmark would make a large part of the method inapplicable.

4. Data availability and implementation feasibility. Restricting to three HS6 codes over eight years produces 25,844 observations — enough for corridor histories, peer distributions and family-level evaluation, small enough to verify by hand and reproduce on a laptop. The full BACI release is far larger and would obscure the methodology behind engineering.

5. Variation in commodity characteristics. The three differ in physical form (liquid bulk, solid refined metal, high-density precious metal), in specification variability within the HS6 code, in the structure of their trade (many small flows versus a few concentrated ones), and in reporting completeness.

6. Variation in the value-to-volume relationship. This is the sharpest contrast, and it matters because the method's central measurement is value divided by quantity. The observed 2024 benchmarks differ by roughly five orders of magnitude between palm oil and gold, which stress-tests every scale-sensitive step in the pipeline.

Table 3.2. Contrast across the three families — benchmark level, panel share and quantity reporting.

Family	2024 benchmark (USD/metric ton)	Panel rows	Share of panel value	Rows without usable quantity
Crude palm oil	\$963.36	6,717	2.7%	87 (1.30%)
Non-monetary unwrought gold	\$76,766,338	12,205	81.0%	1,877 (15.38%)
Refined copper cathodes	\$9,142.14	6,922	16.3%	224 (3.24%)

3.4.2 The specific analytical role of each family

Crude palm oil (151110) — the regional bulk-commodity anchor. A widely traded, high-volume agricultural commodity with a well-established public benchmark (Malaysian crude palm oil). Its role is *scale and stability*: many corridors, many observations, a benchmark that moves with a visible agricultural cycle. It is the family in which the method must **not** over-alert, because most of what happens in it is ordinary commerce. It has the lowest missing-quantity rate of the three at 1.30%.

Refined copper cathodes (740311) — the industrial anchor with an interpretable benchmark. Cathode copper is a standardised, exchange-traded product, so its world benchmark is unusually interpretable: a corridor's implied unit value can be read against it with less specification noise than most goods. Its role is *benchmark discipline*. It also carries the largest **value** share of the coverage gap (0.2389% of copper value, \$1.369B) despite a modest row-level gap of 3.24% — a useful reminder that row counts and value shares tell different stories.

Non-monetary unwrought gold (710812) — the typology-relevant stress case. Gold is high-value, compact, fungible and internationally mobile, which is precisely why it recurs in TBML typology material. Analytically it is the hardest of the three:

- **Extreme value density.** The 2024 benchmark is \$76,766,338 per metric ton. Corridor quantities are routinely small fractions of a ton, so implied unit value is computed from a very small denominator and is arithmetically volatile.
- **Different missing-quantity behaviour.** 1,877 of 12,205 gold rows (15.38%) carry a valid value but no usable quantity — proportionally far more than palm oil or copper. Chapter 10.2 analyses this in depth.
- **Unobservable specification.** HS6 710812 covers non-monetary gold in unwrought forms other than powder. Purity, recycled content, exact form and commercial terms are not visible in the data, and all of them move price.
- **Unit conversion.** The gold benchmark is published in US dollars per troy ounce and must be converted to a per-metric-ton basis before any comparison is possible (Section 7.7).

The scope is narrower than “gold”: HS6 710812 excludes non-monetary gold powder (710811), semi-manufactured gold (710813) and monetary gold (710820).

CAUTION

None of these three products, and no corridor in which they are traded, is inherently suspicious. They were selected because together they span a wide range of commodity behaviour and therefore test the method from several directions at once. A commodity's presence in this project carries no risk implication whatsoever.

3.5 The FATF–Egmont typology cards

`configs/typology_context.yml` holds 5 typology cards. Each names its source document and section, states its meaning in plain English, lists the project features it relates to, and — critically — fixes both the language that may be used and the language that may not.

Table 3.3. FATF–Egmont typology cards and the project features they contextualise.

Typology card	Plain-English meaning	Related project features
TBML_VALUATION_CONTEXT	Over- or under-valuation and inconsistent market values are recognised TBML risk contexts, but public aggregate data cannot prove invoice mispricing.	<code>benchmark_residual</code> , <code>benchmark_adjusted_drift</code> , <code>robust_historical_z</code> , <code>same_family_year_peer_percentile</code>
TBML_HIGH_VALUE_LOW_VOLUME_CONTEXT	High-value products and products with wide pricing margins, including precious metals, can be relevant typology context.	<code>unit_value_usd_per_metric_ton</code> , <code>quantity_yoy_change</code> , <code>value_quantity_divergence</code> , <code>valid_extreme_flag</code>
TBML_PHANTOM_SHIPMENT_LIMITATION	Phantom shipment is a known technique, but this lab cannot detect whether goods physically moved because it lacks shipment-level evidence.	<code>missing_quantity_flag</code> , <code>corridor_novelty_flag</code> , <code>corridor_reactivation_flag</code>
TBML_BENIGN_MARKET_SHOCK_CAVEAT	A risk indicator is not conclusive; commodity price shocks, freight, quality, and contract effects can be benign explanations.	<code>benchmark_yoy_change</code> , <code>benchmark_consistency_gap</code> , <code>benchmark_residual</code>
TBML_MISSING_PRIVATE_INFO_REVIEW_STEPS	Several useful indicators require external data held by banks, customs, or counterparties; public data is only a triage layer.	<code>missing_quantity_flag</code> , <code>missing_country_mapping_flag</code> , <code>data_quality_score</code>

The cards are used at the point of interpretation, never at the point of computation. They shape how a pattern may be described; they contribute nothing to any score, feature or evidence value.

CHAPTER 4

Assumptions, Boundaries, Limitations, and Non-Goals

This chapter separates six things that are commonly collapsed into one list. An **assumption** is something taken to be true so the analysis can proceed. A **boundary** is a deliberate scope decision. A **data limitation** is a property of the source that no modelling can repair. A **model limitation** is a weakness of the method itself. An **operational limitation** describes the gap between a prototype and production. A **non-goal** is something the project deliberately does not attempt.

4.1 Assumptions

Table 4.1. Assumptions on which the analysis rests.

#	Assumption	Why it is reasonable	What breaks if it is wrong
A1	The source extract faithfully reproduces the publisher's data for the selected scope	<code>src/01_verify_sources.py</code> re-derives row counts by year and by HS6 and matches them against the publisher's own provenance notes; all checks pass	Every downstream count and every score
A2	Declared value is in thousands of current USD and quantity in metric tons	Stated in the publisher's readme and in <code>data_source_notes.json</code> ; the workbook unit string is asserted at parse time	Unit values and benchmark residuals would be wrong by orders of magnitude
A3	A corridor's own earlier years are a usable baseline for its later years	Corridor composition is more stable year to year than the cross-section; the baseline is robust (median and MAD) rather than mean-based	Historical drift signals lose meaning for restructured corridors
A4	The World Bank annual series is a reasonable <i>market context</i> for each family	It is an independent, published, widely used world price for each of the three commodities	Benchmark residual and drift become uninterpretable
A5	Same-family, same-year corridors are comparable peers	They face the same world price and the same year's conditions	Peer percentile stops being a fair comparison
A6	Controlled scenarios are a usable proxy for evaluating <i>ranking</i> behaviour	The transformations are explicit, seeded, auditable, and paired with benign hard negatives	Model selection would rest on an untested basis
A7	A missing quantity is a reporting condition, not a signal	Quantity is missing for 2,188 raw rows across all three families and is structurally concentrated in specific long-running corridors	Data-quality gaps would be misread as risk

4.2 Analytical boundaries

- **Scope is three HS6 codes over 8 years.** `151110`, `740311`, `710812`; 2017–2024. Nothing outside that scope is observed or claimed.
- **The unit of analysis is the annual corridor–product observation.** No sub-annual, entity-level or transaction-level analysis is attempted.
- **The output is an ordering, not a classification.** No threshold separates “flagged” from “clean”. The queue is the top 50 of the eligible population.
- **Scenario labels never reach real records.** Labels live in a separate file (`scenario_labels.parquet`); `build_features()` raises an error if a forbidden label column enters the feature set; `src/06` scores real observations from `corridor_features.parquet`, which has no label column at all.
- **Typology material informs language only.** It contributes nothing to any computation.
- **No inference about parties.** The data identifies countries. It does not identify firms, banks, or people, and the project makes no statement about any of them.

4.3 Data limitations

CAUTION

Annual aggregate data is not invoice-, shipment-, customer-, or transaction-level data. Every observation is a whole year of trade between two countries in one product group, summed. It cannot be decomposed into the deals that produced it.

L1 — HS6 groups hide real economics. Within a single HS6 code, grade, purity, physical form, packaging and product mix vary. So do freight, insurance, Incoterms and contract timing, all of which move declared value without any change in the underlying goods. Two corridors with genuinely different implied unit values may simply be trading different specifications on different terms.

L2 — Commodity benchmarks are macro context, not invoice-level fair value. The configuration records this per series: the palm-oil benchmark is a *“Malaysia crude palm oil benchmark; not corridor-specific landed or invoice price”*; copper is an *“LME-related macro benchmark; not corridor-specific contract price”*; gold is a *“Spot gold benchmark; purity, form, and commercial terms are not observed”*. A departure from a benchmark is a reason to ask a question, never proof of mispricing.

L3 — Quantity may be missing or unsuitable for unit-value modelling. 2,188 panel observations (8.5% of the panel) have a valid declared value but no usable quantity, so no implied unit value exists for them. They are retained in full for audit and are never scored. See Chapter 10.2.

L4 — Values are nominal. No inflation adjustment is applied, so multi-year comparisons of value mix real and price-level change.

L5 — Confirmed TBML labels are unavailable in the public dataset. There is no ground truth. This is the constraint that forces the scenario-evaluation design of Chapter 9 and it cannot be engineered away.

L6 — Country codes include grouped and residual reporters. Some codes represent aggregates rather than single states (for example `S19`, “Other Asia, not elsewhere specified”). These are retained with their source code and must not be read as a country.

L7 — Short corridor histories are common. 7,991 feature rows (30.9%) have no prior-year baseline, because the corridor–product appears for the first time. History-based signals are undefined for those rows, which is recorded explicitly rather than imputed.

4.4 Model limitations

M1 — Scenario labels are evaluation devices, not real criminal labels. Every metric in Chapter 9 describes how well a method ranks *injected* patterns in a *fixed* synthetic design. Nothing in this project estimates real-world detection performance, and no such claim is made anywhere.

M2 — Model scores are ranking signals, not probabilities of crime. The blended score has no calibration and no probabilistic interpretation. Two scores can be compared for order; neither can be read as a likelihood.

M3 — The evaluation is descriptive, with no uncertainty quantification. The repository produces no confidence intervals, no bootstrap resampling, and no external validation. Differences between methods are point estimates on one fixed scenario set.

M4 — ROC-AUC is not reported. `src/06_train_evaluate_models.py` computes precision, recall and lift at k , average precision, and two false-positive rates. ROC-AUC is not produced, because with a positive prevalence of 1.790% it can appear strong while the top of the queue — the only part an analyst reviews — is weak.

M5 — Family-level performance differs materially. On the test split the selected method separated injected patterns best for `gold_unwrought` and least well for `crude_palm_oil`. Family sizes and base rates differ, so this is a model-performance warning, not a statement that one commodity carries more risk.

M6 — Explanation is global, not per-row. Only a global mean-absolute-SHAP summary over a fixed 500-row sample is produced. Row-level SHAP is not computed and is therefore never displayed per case.

M7 — Model contribution is not evidence. A SHAP value describes how an input moved a model's output. It is not a fact about the trade. The project keeps these strictly apart: evidence rows come from recomputable feature values and comparisons; SHAP appears only in the model-governance section, labelled as global.

M8 — The rules baseline is a weighted sum, not a weighted average. The nine positive weights total 1.18, and the result is clipped into 0–1 after two deductions. The number is an engineering ranking score with no external meaning.

4.5 Operational limitations

O1 — The dashboard is an analytical prototype, not a production AFC platform. It is a read-only presentation layer over generated files. It never retrains a model, rebuilds the panel, or writes analytical output.

O2 — There is no live data feed. Every artefact is produced by a manual pipeline run over a fixed, hash-pinned extract.

O3 — There is no case management, access control, or user audit logging. No user action is recorded, and no disposition can be captured.

O4 — There is no model monitoring or drift detection. Nothing observes whether the scoring behaviour changes as new data arrives.

O5 — No feedback loop exists. No review outcome flows back, because there are no reviews.

O6 — Deployment configuration is only partly verifiable from the repository. The application entry point (`dashboard/streamlit_app.py`), the pinned runtime dependency set, and the Streamlit theme are in the repository. The Streamlit Community Cloud branch binding and any application settings are held in the hosting account and are not recorded here.

O7 — No continuous-integration workflow exists. There is no `.github/workflows/` directory in the repository, so no automated build, test or deployment behaviour is claimed.

4.6 Non-goals

The project does **not** attempt to:

1. Detect trade-based money laundering.
2. Estimate the probability that any observation is illicit.
3. Identify firms, banks, individuals or beneficial owners.
4. Value an individual invoice, contract or shipment.
5. Replace bank monitoring, screening or case management.
6. Produce a regulatory filing, or any input to one, without human review.
7. Rank countries or corridors by risk.
8. Draw conclusions about the reliability of any national statistical reporter.
9. Generate narrative text with a large language model. `src/08_build_briefs.py` renders briefs deterministically from structured inputs; the prompt assets it writes document how a future model *could* be constrained, and no model is called anywhere in the repository.

4.7 Safe language versus language to avoid

Table 4.2. Approved and prohibited phrasing for describing project outputs.

Safe language	Language to avoid
"This observation ranks highly for human review because several aggregate trade signals are unusual."	"The model detected money laundering."
"The implied unit value sits 6.36× the annual benchmark for this product and year."	"The World Bank benchmark proves that an invoice was mispriced."
"Review-priority score 0.9891 — used to order the queue."	"A 0.9891 probability of financial crime."
"The corridor—product pattern merits a caveated human review."	"This corridor is a laundering route."
"Quantity is not usable for this record, so no unit-value score can be computed."	"This record is hiding its quantity."
"High-value, low-volume commodity context raises the need for caveated review."	"Gold trade is inherently suspicious."
"The public-data pattern may motivate checking shipment evidence."	"No goods moved."
"This ranking is a starting point; private records and human judgement are required."	"The public-data model is sufficient for disposition."
"Model contribution shows which inputs moved the model output on average."	"SHAP shows why this trade is suspicious."
"Injected scenario patterns; not real cases."	"The model found 24 laundering cases in the test years."

The two right-hand statements at the foot of the table are drawn from the prohibited-language fields of the typology cards and the dashboard's own not-allowed list; the prohibited-phrase check in `src/08_build_briefs.py` enforces the same discipline mechanically over generated text.

CHAPTER 5

System Architecture and Deployment

5.1 The implemented architecture

The project has two halves that meet at a set of files. The **analytical pipeline** (`src/`) reads verified sources and writes artefacts. The **presentation layer** (`dashboard/`) reads those artefacts and renders them. The dashboard never computes a score, never retrains a model, and never writes into the analytical outputs; the pipeline never knows the dashboard exists. Everything they share is a file on disk with a documented contract.

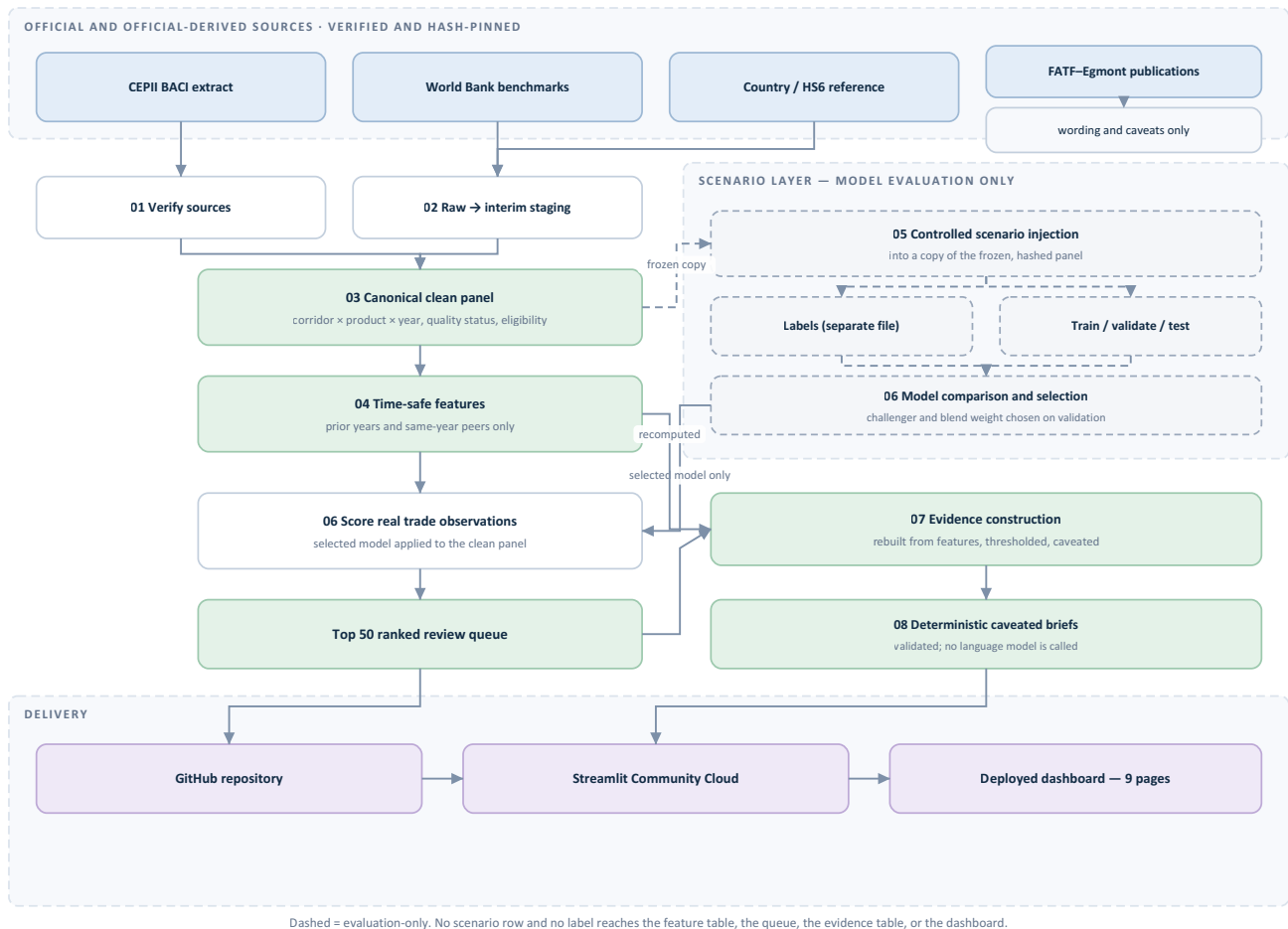


Figure 5.1. System architecture of the implemented project. The scenario layer (dashed) is confined to model evaluation: its labels never reach the feature table, the real-observation scoring path, the evidence table, the review queue, or the dashboard.

Three properties of the diagram are load-bearing.

The scenario layer is a closed loop. Scenarios are injected into a *copy* of the frozen clean panel. They produce evaluation metrics and a selected model. The selected model — not the scenario data — is then applied to real observations. No scenario row, and no scenario label, ever appears in `corridor_features.parquet`, `top_ranked_corridors.csv`, `evidence_table.csv`, or anywhere in the dashboard.

Evidence is rebuilt from features, not from scores. `src/07_build_evidence.py` re-reads `corridor_features.parquet` and recomputes each metric against its threshold. It does not decompose the score. That is what makes an evidence row independently recomputable.

Typology context enters only at the interpretation layer. The FATF–Egmont material feeds the typology cards, which feed brief wording and caveats. It has no path into any feature, score, or evidence value.

5.2 Pipeline stages

Table 5.1. The analytical pipeline — 11 Python modules in `src/`, 2,681 lines in total.

Script	Reads	Writes	Purpose
<code>00_check_environment.py</code>	installed environment	<code>environment_check.json</code>	Imports every required package, records versions, exits non-zero if any is absent
<code>01_verify_sources.py</code>	<code>data/raw/*</code>	<code>source_file_inventory.csv</code> , <code>source_manifest.json</code>	Chain of custody: hashes, cross-checks against publisher notes, benchmark unit checks, PDF readability
<code>02_ingest_official_sources.py</code>	BACI parquet, World Bank workbook	<code>data/interim/*.parquet</code>	Faithful raw→interim staging; no derived fields
<code>03_build_panel.py</code>	interim staging	<code>corridor_product_year_panel.parquet</code> , <code>panel.csv</code> , <code>exclusion_audit.csv</code>	Canonical panel construction, reference joins, quality status, eligibility
<code>04_build_features.py</code>	panel	<code>corridor_features.parquet</code> , <code>reports/feature_explanation_table.md</code>	Time-safe feature engineering and its data dictionary
<code>05_build_scenarios.py</code>	frozen panel	<code>scenario_panel.parquet</code> , <code>scenario_labels.parquet</code> , <code>scenario_injections.csv</code> , <code>scenario_split_manifest.json</code>	Seeded scenario injection with labels held separately
<code>06_train_evaluate_models.py</code>	scenario panel + labels, real features	<code>model_*.csv/json</code> , <code>model_bundle.joblib</code> , <code>top_ranked_corridors.csv</code> , <code>shap_summary_values.csv</code> , figures	Fit, evaluate, select, then score real observations and cut the Top 50
<code>07_build_evidence.py</code>	features + queue	<code>evidence_table.csv</code> (+ repo-root copy), updated queue	Recomputable evidence rows and evidence counts
<code>08_build_briefs.py</code>	queue, evidence, typology cards	<code>analyst_briefs.json/.md</code> , <code>brief_validation_report.csv</code> , <code>prompts/*</code>	Deterministic caveated briefs and their validation
<code>09_generate_reports.py</code>	all of the above	<code>reports/*.md/.html/.pdf</code> , <code>reports/figures/*</code> , <code>report_manifest.json</code>	Diagram rendering and written deliverables

`src/tbml_common.py` (867 lines) is the shared library holding the panel builder, the feature builder, the scenario injector, the benchmark parser, and the boundary constant. It is imported by every numbered script and writes nothing itself.

KEY POINT

The numbered scripts are ordered and each depends on its predecessor's outputs. `src/03` fails with a clear error if `src/02` has not run; the dashboard fails with a named file and the exact command to run if any required artefact is missing. Nothing substitutes demo data at any point.

5.3 Presentation layer

The dashboard is a multi-page Streamlit application: 11 page modules (10 content pages plus a full-screen welcome route), 11 service modules, and 18 shared UI components, 13,183 lines in total.

Two architectural decisions are worth recording.

Page files live in `app_pages/`, not `pages/`. A directory literally named `pages` beside the entry point triggers Streamlit's older multi-page compatibility path, under which a cold-session deep link executes the page file alone — without the global styles, the sidebar, or the boundary ribbon. Naming the directory `app_pages/` and registering routes explicitly through `st.navigation` guarantees that the human-review boundary rides on every page.

Every file path is declared once. `dashboard/services/path_resolver.py` walks upward from its own location until it finds both `configs/project.yml` and `src/tbml_common.py`, so the application runs from any working directory. All artefacts are registered in a single `DATA_FILES` map with a purpose and a required/optional flag. A missing **required** file raises a dashboard-facing error naming the file and the pipeline command that regenerates it; a missing **optional** file degrades to a labelled empty state.

5.4 Environment setup

5.4.1 Supported Python version

The pipeline and dashboard were developed and verified on **Python 3.12**. The repository's development-container definition pins `mcr.microsoft.com/devcontainers/python:1-3.11-bookworm` (Python 3.11), and no code in the repository uses syntax newer than 3.11. Either interpreter is expected to work; 3.12 is what the recorded run used.

5.4.2 Two dependency sets

The repository deliberately splits its dependencies, because the deployed application needs far less than the pipeline.

Table 5.2. The two dependency files.

File	Purpose	Contents
<code>requirements.txt</code>	Deployed dashboard runtime , fully pinned. This is what Streamlit Community Cloud installs.	<code>streamlit==1.59.2, plotly==6.9.0, pandas==3.0.4, numpy==2.4.6, pyarrow==24.0.0, PyYAML==6.0.3</code>
<code>requirements-pipeline.txt</code>	Full pipeline, tests and report generation , unpinned	24 packages including <code>pandas, pyarrow, openpyxl, numpy, scikit-learn, xgboost, shap, matplotlib, seaborn, PyYAML, weasyprint, reportlab, pypdf, pillow, joblib, markdown, graphviz, ipykernel, streamlit, plotly, pytest</code>

KEY POINT

The deployed application imports only the six packages in `requirements.txt`. It has no machine-learning dependency at runtime because it never scores anything — it reads scores the pipeline already wrote.

5.4.3 Creating the environment

```
# from the repository root
py -m venv .venv
.venv\Scripts\activate

# full analytical pipeline, tests and report generation
python -m pip install -r requirements-pipeline.txt

# or, for the dashboard alone (the pinned deployed runtime)
python -m pip install -r requirements.txt
```

5.4.4 Validating the environment

```
python src/00_check_environment.py
```

`src/00_check_environment.py` imports every required package, records each version into `data/outputs/environment_check.json`, and exits non-zero if any import fails — so a missing dependency stops the run immediately rather than deep inside a later stage.

5.4.5 Platform notes

- **Windows.** Commands above are PowerShell. The recorded run used Windows 11 with Python 3.12. `weasyprint` (used by `src/09` for PDF output) requires its native GTK/Pango libraries on Windows; if they are absent, `src/09` fails at PDF conversion only — every analytical output and the dashboard are unaffected.
- **Graphviz.** `src/09_generate_reports.py` renders its diagrams by calling the system `dot` binary. If `dot` is not on the PATH the script falls back to writing a placeholder image and still completes; the `.dot` sources are always written.
- **Linux / macOS / Codespaces.** Replace `.venv\Scripts\activate` with `source .venv/bin/activate`. The development container image installs `requirements.txt` and launches the dashboard automatically on port 8501.

5.4.6 Main execution commands

```
python src/00_check_environment.py
python src/01_verify_sources.py
python src/02_ingest_official_sources.py
python src/03_build_panel.py
python src/04_build_features.py
python src/05_build_scenarios.py
python src/06_train_evaluate_models.py
python src/07_build_evidence.py
python src/08_build_briefs.py
python src/09_generate_reports.py

python -m streamlit run dashboard/streamlit_app.py
```

5.5 Deployment

5.5.1 The real deployment flow

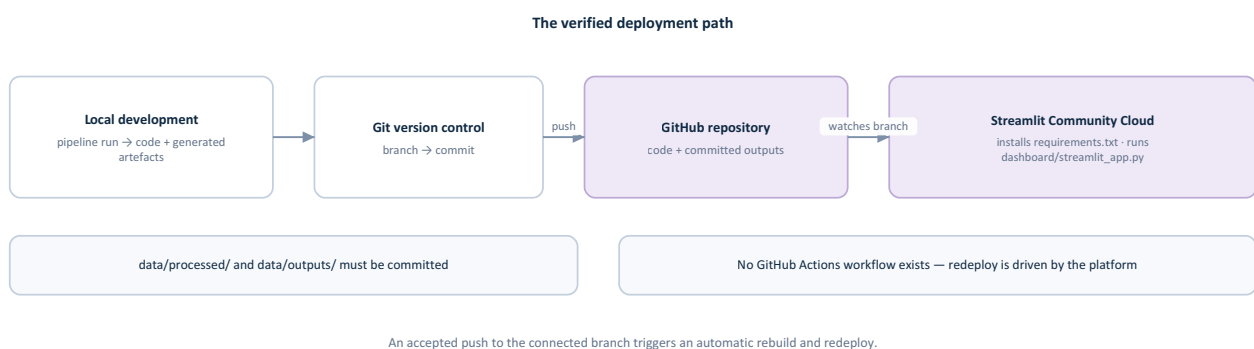


Figure 5.2. The verified deployment path. Local development produces both code and analytical artefacts; both are pushed to GitHub; Streamlit Community Cloud builds the application from the pinned runtime requirements and serves it.

1 — Local development. Code and pipeline outputs are produced locally. Because the dashboard is a read-only reader of generated files, the artefacts under `data/processed/` and `data/outputs/` must exist in the repository for the deployed application to have anything to show. They are not regenerated in the cloud.

2 — Version control. Git, with `main` as the default branch. Development at the time of writing is on `develop` at commit `50c4096`.

- 3 — GitHub repository.** <https://github.com/jessybaolin/evidence-based-tbml-corridor-drift> (remote <https://github.com/jessybaolin/evidence-based-tbml-corridor-drift>).
- 4 — Application entry point.** `dashboard/streamlit_app.py`. This is the path Streamlit Community Cloud is pointed at.
- 5 — Dependency file.** `requirements.txt` — the pinned six-package runtime set. Streamlit Community Cloud installs exactly this.
- 6 — Streamlit configuration.** `.streamlit/config.toml` sets the base theme to `light`, the primary colour to `#0F8F8F`, the text colour to `#22314A`, a matching dataframe header colour, a monospace code font, `headless = true`, and disables usage-statistics gathering. Dashboard-specific design tokens live separately in `dashboard/config/dashboard_theme.yml`.
- 7 — Secrets.** None. The application reads no credential, calls no external service, and contains no reference to `st.secrets`. There is no secrets configuration to manage.
- 8 — Connection and redeployment.** The GitHub repository is connected to Streamlit Community Cloud, which serves the application at <https://tbml-review-triage.streamlit.app/>. An accepted push to the configured branch triggers an automatic rebuild and redeploy.

CAUTION

GitHub Actions is not required for Streamlit auto-deployment, and this repository does not use it. There is no `.github/workflows/` directory. Redeployment is driven by Streamlit Community Cloud watching the connected repository, not by any workflow in this project.

5.5.2 Deployment facts that the repository does not record

The repository does not contain the Streamlit Community Cloud application settings. The **deployment branch binding**, the app URL slug configuration, and any resource settings are held in the hosting account. They are therefore stated here as unverified rather than asserted.

5.5.3 Development container

`.devcontainer/devcontainer.json` defines a reproducible Codespaces/devcontainer environment: image `mcr.microsoft.com/devcontainers/python:1-3.11-bookworm`, Python extensions, installation of `requirements.txt`, and a post-attach command that launches the dashboard on forwarded port 8501 with CORS and XSRF protection disabled (appropriate for a containerised preview, not for a public deployment).

5.6 Where the deployed application lives

Deployed application	https://tbml-review-triage.streamlit.app/
GitHub repository	https://github.com/jessybaolin/evidence-based-tbml-corridor-drift

CHAPTER 6

Source Data and Provenance

6.1 Four kinds of source, four different roles

The project uses four categories of input and never lets them cross roles. This separation is the reason a reader can trust that a typology document did not become a model input.

Table 6.1. Source categories and their strictly separated roles.

Category	Source	Role	Never used as
Row-level analytical data	CEPII BACI (filtered HS17 extract)	The observations that are cleaned, featurised, scored and ranked	—
Commodity benchmark context	World Bank Commodity Markets Outlook, annual nominal prices	Broad market comparison for implied unit values	An invoice price, a landed cost, or a fair-value estimate
Code / reference data	BACI country codes; HS17 product codes	Mapping numeric codes to ISO3 country codes, names and product descriptions	Analytical signal
Typology and policy context	FATF–Egmont 2020 and 2021 publications	Interpretation vocabulary, caveats, recommended review steps	Training data, features, labels, or row-level evidence

6.2 Source 1 — CEPII BACI

CEPII BACI	Row-level analytical data
The observations that are cleaned, featurised, scored and ranked. This is the only source that contributes rows to the analysis.	

Publisher. CEPII (Centre d’Etudes Prospectives et d’Informations Internationales).

Dataset. BACI — International Trade Database at the Product Level. Trade flows at the year–exporter–importer–product level, in HS 6-digit nomenclature. The academic reference recorded in the source readme is Gaulier, G. and Zignago, S. (2010), *BACI: International Trade Database at the Product-Level*, CEPII Working Paper N°2010-23.

Version. Release **202601**, HS revision **HS17**, release date 22 January 2026.

Years used. 2017–2024 (8 consecutive annual snapshots).

Fields used. Exactly six, retained unaltered: `t` (year), `k` (HS6 product code), `i` (exporter country code), `j` (importer country code), `v` (trade value), `q` (quantity).

Units. `v` in thousands of current US dollars; `q` in metric tons; `k` a six-character string.

Access and retrieval. Downloaded from the CEPII BACI page, then filtered locally to the three HS6 codes and eight years by `extract_baci_products.py` before entering this repository. The filtering step is documented in `data/raw/data_source_notes.json`, which records the 8 original annual CSV files used and states that no rows were aggregated and no derived fields were added.

Local filename. `data/raw/baci_hs17_v202601_selected_2017_2024_151110_740311_710812.parquet` — SHA-256 `0e493e057c8fd579...`, 2026-06-28.

Role. The analytical population. Every ranked observation is one of these rows.

Transformations applied by this project. `v × 1000` to nominal USD; `q` retained as metric tons; `k` normalised to a zero-padded six-character string; numeric country codes mapped to ISO3 codes and names; a deterministic `obs_id` and a `source_row_id` hash added for lineage. Nothing else is altered before feature engineering.

Verification performed. `src/01_verify_sources.py` re-derives the actual row counts from the parquet and compares them against the publisher’s own provenance notes:

Table 6.2. BACI verification checks — all pass.

Check	Expected	Verified
Required columns present	<code>t, k, i, j, v, q</code>	Yes
Years exactly 2017–2024	8 years	Yes
HS6 codes exactly <code>151110</code> , <code>740311</code> , <code>710812</code>	3 codes	Yes
Total row count	25,844	25,844 — matches
Row counts by HS6	matches notes	Yes
Row counts by year	matches notes	Yes
Provenance is official-derived, not synthetic	asserted in notes	Yes
No derived fields added before upload	asserted in notes	Yes

Key limitations. Annual aggregates only. No firms, invoices, shipments, payments or ownership. Grouped reporter codes exist. Values are nominal. The repository file is a filtered three-code extract, not the full BACI release.

Official link. https://www.cepii.fr/CEPII/en/bdd_modele/bdd_modele_item.asp?id=37

6.3 Source 2 — World Bank commodity benchmarks

World Bank Commodity Markets Outlook	Commodity benchmark context
Broad market comparison for implied unit values. Never an invoice price, a landed cost, or a fair-value estimate.	

Publisher. World Bank.

Dataset. Commodity Markets Outlook — the “Pink Sheet” historical data, annual series.

Version. `CMO-Historical-Data-Annual.xlsx`, sheet **Annual Prices (Nominal)**. SHA-256 `9fbc348f40ecdb02eb1bcf8...`.

Years used. 2017–2024.

Fields used. Three annual nominal price columns: *Palm oil*, *Copper*, *Gold*.

Units. Palm oil and copper in **US dollars per metric ton**; gold in **US dollars per troy ounce**.

Access and retrieval. Downloaded from the World Bank commodity-markets page and stored under `data/raw/`.

Role. Broad market context. It supplies the denominator of the benchmark residual and the market-movement term in the benchmark-consistency check.

Transformations applied. The parser in `src/tbml_common.py` locates the commodity header row and the year column by content rather than by fixed cell coordinates, so a layout change does not silently mis-read the sheet. It then **asserts the workbook’s own unit string** against the unit declared in `configs/benchmark_series.yml` and raises an error on any mismatch. Palm oil and copper are used as published. Gold is converted to a per-metric-ton basis:

$$B_y^{\text{gold}} [\text{USD/mt}] = B_y^{\text{gold}} [\text{USD/troy oz}] \times \frac{1,000,000}{31.1034768} \quad (6.1)$$

where: 1 metric ton = 1,000,000 grams and 1 troy ounce = 31.1034768 grams, giving a conversion factor of 32,150.7466.
Implemented in `src/tbml_common.py` — `GOLD_TROY_OUNCE_TO_METRIC_TON_FACTOR`, `extract_worldbank_benchmarks()`

giving a factor of **32,150.7466**. The 2024 converted gold benchmark is \$76,766,338 per metric ton. The parser finally asserts that exactly 3 commodities × 8 years = **24 rows** were produced.

Table 6.3. Extracted annual benchmark prices, in US dollars per metric ton (gold also shown as published, per troy ounce).

Year	Crude palm oil (USD/mt)	Refined copper (USD/mt)	Unwrought gold (USD/mt)	Gold as published (USD/troy oz)
2017	\$750.81	\$6,169.94	\$40,431,493	\$1,257.56
2018	\$638.66	\$6,529.80	\$40,806,692	\$1,269.23
2019	\$601.37	\$6,010.15	\$44,769,915	\$1,392.50
2020	\$751.77	\$6,173.77	\$56,914,859	\$1,770.25
2021	\$1,130.58	\$9,317.05	\$57,859,448	\$1,799.63
2022	\$1,275.99	\$8,822.37	\$57,890,634	\$1,800.60
2023	\$886.45	\$8,490.29	\$62,458,291	\$1,942.67
2024	\$963.36	\$9,142.14	\$76,766,338	\$2,387.70

Key limitations. Recorded per series in `configs/benchmark_series.yml`: the palm-oil series is a *Malaysia crude palm oil benchmark*; not corridor-specific landed or invoice price; copper is an *LME-related macro benchmark*; not corridor-specific contract price; gold is a *spot gold benchmark* for which *purity, form, and commercial terms are not observed in BACI*.

Official link. <https://www.worldbank.org/en/research/commodity-markets> — this URL is maintained in `dashboard/services/source_registry.py` (verified 2026-07-14) because project metadata does not record it.

6.4 Source 3 — Country and HS product-code reference data

Table 6.4. Reference datasets.

File	Contents	Role	SHA-256 (first 24)
<code>data/raw/country_codes_V202601.csv</code>	BACI numeric country code → ISO3 code and country name	Maps <code>i</code> and <code>j</code> to readable, joinable identifiers	<code>c6892f28bea23d689b29b074...</code>
<code>data/raw/product_codes_HS17_V202601.csv</code>	HS17 code → official product description	Supplies the product description carried on every panel row	<code>4abb74d330c3a1969ccfc465...</code>

Both are published by CEPII alongside the BACI release. The country file is validated for uniqueness of the numeric code before it is used as a join key — a duplicate would silently multiply rows, so `load_country_codes()` raises rather than proceed. Product codes are zero-padded to six characters on load so a numeric reader cannot corrupt a leading zero.

Rows whose country code cannot be mapped would be excluded entirely; in the current run **0 rows** fall into that category — every code in the extract maps successfully.

6.5 Source 4 — FATF–Egmont typology publications

FATF–Egmont TBML publications	Typology and policy context
Interpretation vocabulary, caveats and recommended review steps. Never training data, never a feature, never a label, never row-level evidence.	

Publisher. FATF and the Egmont Group.

Documents.

Table 6.5. FATF–Egmont reference documents.

Document	Published	Pages	Local filename	SHA-256 (first 24)
<i>Trade-Based Money Laundering: Trends and Developments</i>	December 2020	66	Trade-Based-Money-Laundering-Trends-and-Developments_2020.pdf	0969e49877a1937a7bc49ac0...
<i>Trade-Based Money Laundering: Risk Indicators</i>	March 2021	10	Trade-Based-Money-Laundering-Risk-Indicators_2021.pdf	670b80270b33354b17263d44...

Role. Typology and caveat context only. They inform the 5 typology cards in `configs/typology_context.yml`, which fix the allowed and prohibited language for describing each pattern and list the recommended human-review steps.

Verification performed. `src/01_verify_sources.py` opens each PDF and extracts text from up to the first three pages solely to confirm the document is real and readable. Both pass. No text is parsed for analytical use.

CAUTION

These documents are **not** training data, **not** a model input, and **not** row-level evidence. Describing them otherwise would misrepresent the architecture: there is no code path from either PDF to any feature, score, or evidence value.

Official links.

- 2020 Trends and Developments: <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Trade-based-money-laundering-trends-and-developments.html>
- 2021 Risk Indicators: <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Trade-based-money-laundering-indicators.html>

Both URLs are maintained in `dashboard/services/source_registry.py` (verified 2026-07-14), because project metadata does not record them.

6.6 Chain of custody

`src/01_verify_sources.py` produces two artefacts that together constitute the project’s chain of custody, and the pipeline will not proceed past a failed check.

Table 6.6. SHA-256 inventory of all 8 declared inputs (`data/outputs/source_file_inventory.csv`).

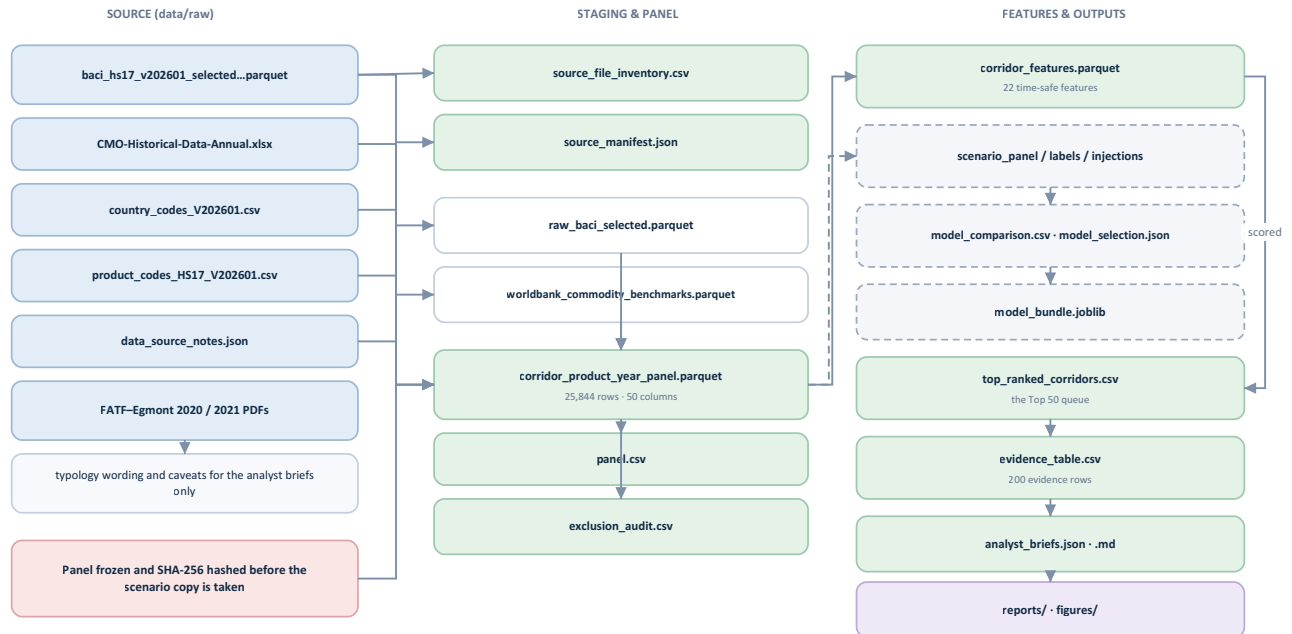
File	Size (bytes)	SHA-256 (first 24)
<code>baci_hs17_v202601_selected_2017_2024_151110_740311_710812.parquet</code>	340,203	0e493e057c8fd57902328551...
<code>data_source_notes.json</code>	4,342	a8a973d0797cbfcd4064b1df...
<code>CMO-Historical-Data-Annual.xlsx</code>	3,177,955	9fbc348f40ecdb02eb1bcf8...
<code>Trade-Based-Money-Laundering-Trends-and-Developments_2020.pdf</code>	3,108,080	0969e49877a1937a7bc49ac0...
<code>Trade-Based-Money-Laundering-Risk-Indicators_2021.pdf</code>	2,079,148	670b80270b33354b17263d44...
<code>country_codes_V202601.csv</code>	5,346	c6892f28bea23d689b29b074...
<code>product_codes_HS17_V202601.csv</code>	598,831	4abb74d330c3a1969ccfc465...
<code>baci-readme.txt</code>	571	156c8fe9d8582f24fdec5c43...

Every panel row then carries three provenance stamps: `source_filename`, `source_version` (`CEPII_BACI_HS17_V202601_filtered_official_derived`), and `source_file_hash` — the SHA-256 of the BACI extract — so any row in any downstream artefact can be traced back to the exact input file that produced it.

CHAPTER 7

Data Pipeline and Data Construction

7.1 Lineage from source file to output artefact



Every box is an actual repository file. Dashed boxes and arrows are evaluation-only and never reach the review queue.

Figure 7.1. Data pipeline and file lineage. Each box is an actual file in the repository; each arrow is a stage in ``src/``. The clean panel is frozen and hashed before the scenario copy is taken, which is what guarantees that evaluation data can never contaminate the real-observation path.

7.2 Stage 1 — Source verification

Before anything is read for analysis, `src/01_verify_sources.py` establishes that the inputs are what they claim to be (Chapter 6). The result is `source_manifest.json`, which every later stage and every governance reader can treat as the authoritative description of the inputs. The manifest also embeds the conclusion-boundary sentence, so the constraint travels with the provenance record.

7.3 Stage 2 — Raw to interim staging

`src/02_ingest_official_sources.py` performs a deliberately dull hand-off. It reads the two official inputs and writes tidy Parquet staging copies with **no derived fields added**:

- `data/interim/raw_baci_selected.parquet` — 25,844 rows, the six official columns, types normalised.
- `data/interim/worldbank_commodity_benchmarks.parquet` — 24 rows, units normalised.

The separation matters for auditability: a reviewer can compare the staged copy against the original file without re-running any analysis.

7.4 Stage 3 — Type and unit normalisation

`read_baci_raw()` enforces the schema on the way in and fails loudly rather than coercing silently:

- The six required columns must be present, or the load raises.
- `k` is cast to a string and **left zero-filled to six characters**, so a reader that inferred a numeric type cannot destroy a leading zero.
- `t`, `i`, `j` are parsed as 64-bit integers with `errors="raise"` — a malformed code stops the run.
- `v` and `q` are parsed numerically with `errors="coerce"`, which turns the source's NA quantity encoding into `NaN`. This is the point at which 2,188 missing quantities become explicit `NaN` rather than an ambiguous blank.

7.5 Stage 4 — HS6 handling, country mapping and reference joins

`build_panel()` in `src/tbml_common.py` assembles the canonical table. Four reference joins are made, each with an explicit cardinality assertion so an unexpected duplicate cannot silently multiply rows:

Table 7.1. Reference joins performed during panel construction.

Join	Key	Cardinality asserted	Adds
HS6 family metadata	<code>hs6</code>	many-to-one	<code>family_id</code> , <code>product_name</code> , <code>benchmark_series_id</code> , <code>project_role</code>
HS17 product description	<code>hs6</code>	many-to-one	<code>hs6_product_code_description</code>
Exporter country	<code>exporter_code</code>	many-to-one	<code>exporter_iso3</code> , <code>exporter_name</code>
Importer country	<code>importer_code</code>	many-to-one	<code>importer_iso3</code> , <code>importer_name</code>
Annual benchmark	<code>family_id</code> , <code>hs6</code> then narrowed to matching year	many-to-many, then filtered	benchmark price (original and normalised), unit, year, conversion method, caveat, benchmark year-over-year change

The benchmark join deserves comment: it is performed as a many-to-many join on (`family_id`, `hs6`) and then narrowed with `panel["year"] == panel["benchmark_year"]`. This is equivalent to a year-aware join but keeps the benchmark's own metadata attached without a second pass. Every panel row ends up with a matched benchmark: **25,844 of 25,844 (100.0%)**.

7.6 Stage 5 — Trade-value conversion and quantity treatment

Trade value is converted from the published unit to nominal US dollars:

$$V_{c,p,y} = v_{c,p,y} \times 1000 \quad (7.1)$$

where: $V_{c,p,y}$ is the published trade value in thousands of current US dollars; $V_{c,p,y}$ is the nominal US-dollar value used throughout the project. · Implemented in `src/tbml_common.py` — `build_panel()`

Quantity is retained as published, in metric tons, and is recorded with an explicit `quantity_unit_raw` field so the unit is carried in data rather than assumed in code.

Three validity flags are then set, and they are the gate for everything downstream:

$$\begin{aligned}
\text{valid}^V &= \mathbb{1}[V \text{ observed} \wedge V > 0] \\
\text{valid}^Q &= \mathbb{1}[Q \text{ observed} \wedge Q > 0] \\
\text{valid}^U &= \text{valid}^V \cdot \text{valid}^Q
\end{aligned} \tag{7.2}$$

where: A declared value or quantity counts as valid only when it is present and strictly positive. An implied unit value exists only where both are valid. · Implemented in `src/tbml_common.py` — `build_panel()`

7.7 Stage 6 — Benchmark extraction, normalisation and the gold conversion

The benchmark parser is described in Chapter 6. The one non-identity conversion is gold, which is published per troy ounce and must be expressed per metric ton before it can be compared with a BACI quantity:

$$B_y^{\text{gold}} = P_y^{\text{troy}} \cdot \frac{g_{\text{mt}}}{g_{\text{oz}}} = P_y^{\text{troy}} \cdot \frac{1,000,000}{31.1034768} \tag{7.3}$$

where: P_y^{troy} is the published spot gold price in US dollars per troy ounce; g_{mt} and g_{oz} are grams per metric ton and per troy ounce. · Implemented in `src/tbml_common.py` — `extract_worldbank_benchmarks()`

Using 1 metric ton = 1,000,000 grams and 1 troy ounce = 31.1034768 grams, the factor is **32,150.7466**. The conversion method string is stored on every gold panel row (`gold_usd_per_troy_ounce * (1000000 / 31.1034768)`), so the arithmetic is visible in the data rather than buried in code.

CAUTION

This conversion changes the *unit*, not the *meaning*. A spot gold benchmark expressed per metric ton is still a broad market reference. It does not become an invoice-level fair value by being rescaled.

7.8 Stage 7 — Core derived measurements

Three measurements are computed at panel level, each defined **only** where its inputs are valid:

$$U_{c,p,y} = \frac{v_{c,p,y}}{Q_{c,p,y}} \quad \text{defined only where } \text{valid}_{c,p,y}^U = 1 \tag{7.4}$$

where: $U_{c,p,y}$ is the implied aggregate unit value in US dollars per metric ton. It is an annual average across every flow in the observation, not a transaction price. · Implemented in `src/tbml_common.py` — `build_panel()`

$$\ell_{c,p,y} = \log U_{c,p,y} \tag{7.5}$$

where: The natural logarithm of the implied unit value. Log space makes upward and downward departures symmetric and comparable across commodities of very different price levels. · Implemented in `src/tbml_common.py` — `build_panel()`

$$r_{c,p,y} = \ell_{c,p,y} - \log B_{f(p),y} \tag{7.6}$$

where: The log gap between the implied unit value and the same-year market benchmark. Zero means parity with the benchmark; positive is above it; negative is below it. · Implemented in `src/tbml_common.py` — `build_panel()`

A *valid extreme* flag marks usable rows whose implied unit value is more than five times or less than one-fifth of the benchmark:

$$\text{extreme}_{c,p,y} = \text{valid}_{c,p,y}^U \cdot \mathbb{1} \left| \frac{U_{c,p,y}}{B_{f(n),v}} > 5 \vee \frac{U_{c,p,y}}{B_{f(n),v}} < 0.2 \right| \quad (7.7)$$

where: Marks a usable observation whose implied unit value is more than five times, or less than one-fifth of, the benchmark. Such rows are retained with a caveat, never discarded. · Implemented in `src/tbml_common.py` — `build_panel()`

2,710 panel rows (10.5%) carry this flag. They are **retained**, not discarded: an extreme ratio computed from valid inputs is a real observation and removing it would be the analytical equivalent of hiding the tail. The flag attaches a caveat and feeds one rules component.

7.9 Stage 8 — The canonical observation key

Two deterministic identifiers are attached to every row:

$$\text{obs_id} = \text{obs_} \parallel \text{SHA256}_{20}(y \parallel i \parallel j \parallel p) \quad (7.8)$$

where: $\parallel$ denotes concatenation and SHA256_{20} the first 20 hexadecimal characters of the digest; i and j are the numeric exporter and importer codes. `source_row_id` uses the same construction over (y, p, i, j, v, q) with 24 characters. · Implemented in `src/tbml_common.py` — `stable_id()`, `baci_source_row_id()`

`obs_id` is the analytical key; `source_row_id` additionally includes the value and quantity, so it identifies the exact raw record. Because both are content-derived SHA-256 prefixes, the same inputs always produce the same identifiers across runs and machines — which is what makes evidence rows and queue entries stable between pipeline executions.

The panel then **asserts uniqueness** of the canonical key (`year`, `exporter_iso3`, `importer_iso3`, `hs6`) over all retained rows and raises with example duplicates if the assertion fails. The source is already one row per key; this is defensive insurance against a bad join.

7.10 Stage 9 — Quality statuses, exclusion logic and the audit trail

Each row is assigned exactly one quality status by an explicit decision sequence:

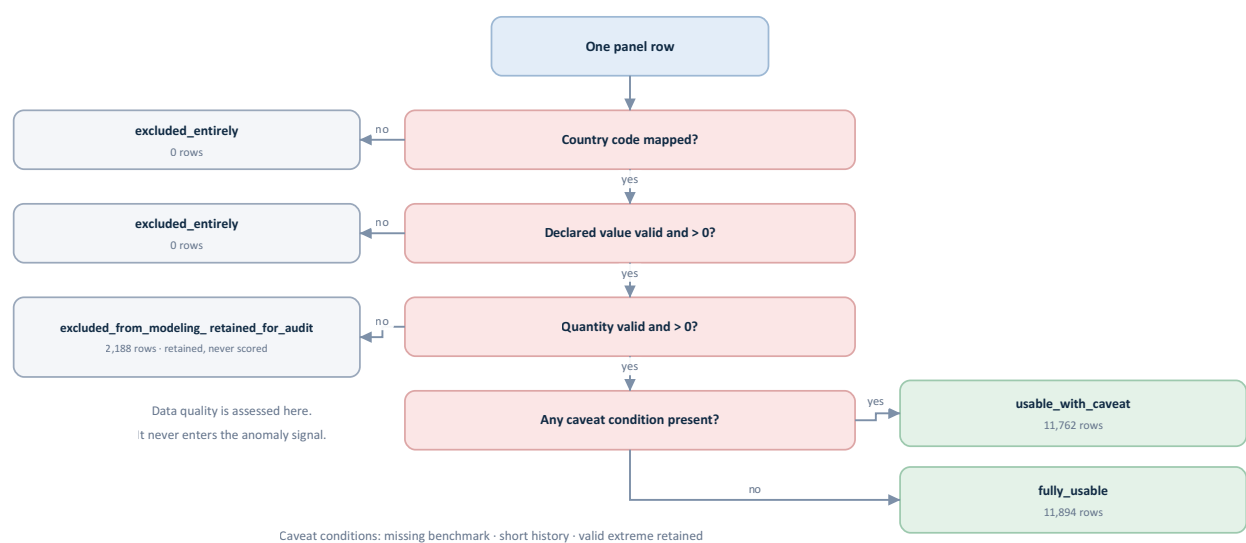


Figure 7.2. Quality-status assignment. The two upper branches remove a row from scoring; the lower branches keep it, with or without caveats. Data quality is assessed here and never enters the anomaly signal.

Table 7.2. Quality-status distribution across the panel.

Quality status	Rows	Share of panel
fully_usable	11,894	46.0%
usable_with_caveat	11,762	45.5%
excluded_from_modeling_retained_for_audit	2,188	8.5%
excluded_entirely	0	0.0%

Table 7.3. Exclusion and caveat reasons.

Reason	Rows	Share of panel
(none — fully usable)	11,894	46.0%
short_history	9,052	35.0%
missing_or_non_positive_quantity	2,188	8.5%
short_history;valid_extreme_retained	1,550	6.0%
valid_extreme_retained	1,160	4.5%

Three points follow directly from these numbers.

No row is lost. 0 rows are excluded entirely — every country code in the extract maps successfully and every declared value is positive. The panel therefore contains all 25,844 source rows.

Most caveats are short history, not anomaly. The dominant caveat reason is `short_history` — a corridor–product with fewer than two prior observed years. This is a statement about how much can be compared, not about the trade.

Every non-perfect row is audited. `data/processed/exclusion_audit.csv` contains 13,950 rows: every observation that is not `fully_usable`, with its status and reason. Nothing is silently dropped or silently caveated.

7.11 Stage 10 — The data-quality score

An additive 0-to-6 confidence score is computed from four components:

$$DQ_{c,p,y} = \text{valid}_{c,p,y}^Q + 2 \cdot \mathbb{1}[\text{benchmark matched}] + h_{c,p,y} + \mathbb{1}[\text{keys complete}] \quad (7.9)$$

where: The four terms contribute 0–1, 0 or 2, 0–2 and 0–1 respectively. $h_{c,p,y} = 2$ when at least five prior years are available, 1 when at least three, and 0 otherwise. The total ranges from 0 to 6 and measures confidence, never anomaly. · Implemented in `src/tbml_common.py` — `build_panel()`

Table 7.4. Data-quality score distribution (mean 4.51).

Data-quality score	Rows	Share of panel
3	1,592	6.2%
4	14,227	55.0%
5	5,337	20.7%
6	4,688	18.1%

KEY POINT

The data-quality score answers *"how much can be said about this row?"* — never *"how unusual is this row?"* In the rules score it appears only as a **deduction**: a low-quality row is ranked more cautiously. It can never increase review priority.

7.12 Stage 11 — Model eligibility

$$\text{eligible}_{c,p,y} = \text{valid}_{c,p,y}^U \cdot \mathbb{1}[\text{status} \in \{\text{fully_usable}, \text{usable_with_caveat}\}] \quad (7.10)$$

where: An observation may be scored only where an implied unit value exists and its quality status is usable, with or without caveats. · Implemented in `src/tbml_common.py` — `build_panel()`

23,656 of 25,844 observations (91.5%) are eligible. The **2,188 ineligible rows** are exactly those with a valid declared value but no usable quantity. They cannot receive a unit-value-based score because the unit value does not exist for them, and they are retained in full for audit and coverage reporting.

7.13 Stage 12 — Data lineage carried on every row

Every panel row carries `source_filename`, `source_version`, `source_file_hash`, `hs_revision`, `source_row_id` and `obs_id`. The evidence table adds `source_table` and `source_fields`. The result is that a claim in a brief can be traced to an evidence row, to a feature value, to a panel row, to a raw source row, to a hashed file.

7.14 Stage 13 — Output artefacts

Table 7.5. Files written by the data-construction stages.

File	Rows	Written by	Contents
<code>data/interim/raw_baci_selected.parquet</code>	25,844	<code>src/02</code>	Staged raw BACI fields
<code>data/interim/worldbank_commodity_benchmarks.parquet</code>	24	<code>src/02</code>	Annual benchmarks, normalised units, benchmark year-over-year change
<code>data/processed/corridor_product_year_panel.parquet</code>	25,844	<code>src/03</code>	The canonical panel, 50 columns
<code>data/processed/panel.csv</code>	25,844	<code>src/03</code>	CSV mirror of the panel for manual inspection
<code>data/processed/exclusion_audit.csv</code>	13,950	<code>src/03</code>	Every caveated or modelling-excluded row with its reason
<code>data/processed/corridor_features.parquet</code>	25,844	<code>src/04</code>	Time-safe features, 44 columns
<code>reports/feature_explanation_table.md</code>	16 entries	<code>src/04</code>	Approved derivation, purpose, time-safety rule and interpretation per feature

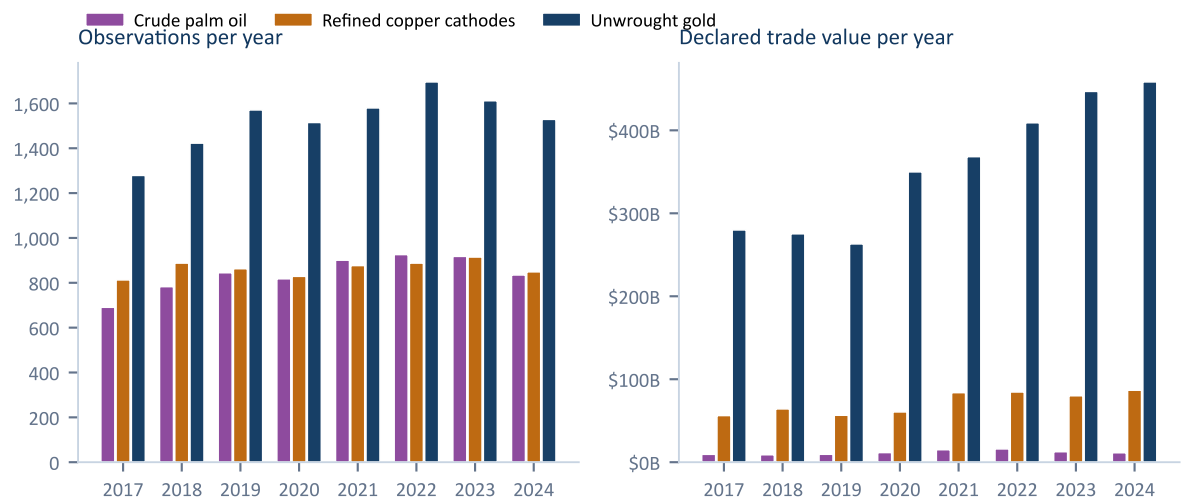
7.15 Panel composition

Table 7.6. Panel composition by product family.

Product family	HS6	Rows	Corridors	Model-eligible	No usable quantity	Declared value	Share of value
Crude palm oil	151110	6,717	2,162	6,630 (98.7%)	87 (1.30%)	\$95.1B	2.7%
Non-monetary unwrought gold	710812	12,205	3,068	10,328 (84.6%)	1,877 (15.38%)	\$2,851.2B	81.0%
Refined copper cathodes	740311	6,922	2,005	6,698 (96.8%)	224 (3.24%)	\$573.3B	16.3%

Table 7.7. Panel composition by year.

Year	Observations	Corridors	Model-eligible	Declared value
2017	2,783	2,086	2,601	\$345.8B
2018	3,094	2,305	2,891	\$348.6B
2019	3,279	2,481	3,001	\$329.4B
2020	3,162	2,408	2,922	\$422.2B
2021	3,358	2,522	3,037	\$467.3B
2022	3,510	2,655	3,171	\$509.9B
2023	3,445	2,594	3,145	\$539.8B
2024	3,213	2,385	2,888	\$556.6B



Source: data/processed/corridor_product_year_panel.parquet. Values are nominal US dollars, not inflation-adjusted.

Figure 7.3. Panel composition — observation counts and declared trade value by product family and year. Gold dominates by value and by row count; palm oil and copper are comparable in both. Values are nominal US dollars and are not inflation-adjusted.

CHAPTER 8

Time-Safe Feature Engineering

This is the analytical core of the project. Every feature in this chapter is extracted from the current implementation in `src/tbml_common.py` (`build_features`, `_prior_median_mad`, `_log_change`) and `src/06_train_evaluate_models.py` (`score_rules`, `bounded_abs`). No formula appears here that is not implemented.

8.1 What “time-safe” means and why it is enforced

A feature computed for year Y may use information from year Y and earlier, and nothing else. Three concrete rules follow:

1. **Historical baselines use strictly prior years.** The median and dispersion of a corridor’s past behaviour are computed from values that came *before* the current one — the current value is appended to the history only after its own baseline has been recorded.
2. **Peer comparisons use the same year only.** Comparing an observation with other corridors trading the same product in the same year is permitted; comparing it with a distribution that includes later years is not.
3. **Year-over-year changes use the previous observed year only.** Never a later one, and never an interpolated one.

Without this discipline, an evaluation measures hindsight. A feature that has seen the future will separate injected patterns beautifully and will be worthless in operation.

KEY POINT

Time safety is enforced structurally, not by convention. `_prior_median_mad()` walks each corridor’s series in order and records the baseline *before* appending the current value, so a row cannot see itself. `build_features()` then asserts that no infinite value was produced and that none of `synthetic_review_priority`, `scenario_id`, `money_laundering`, `fraud`, `criminal` or `tbml_confirmed` has entered the feature frame — and raises if any has.

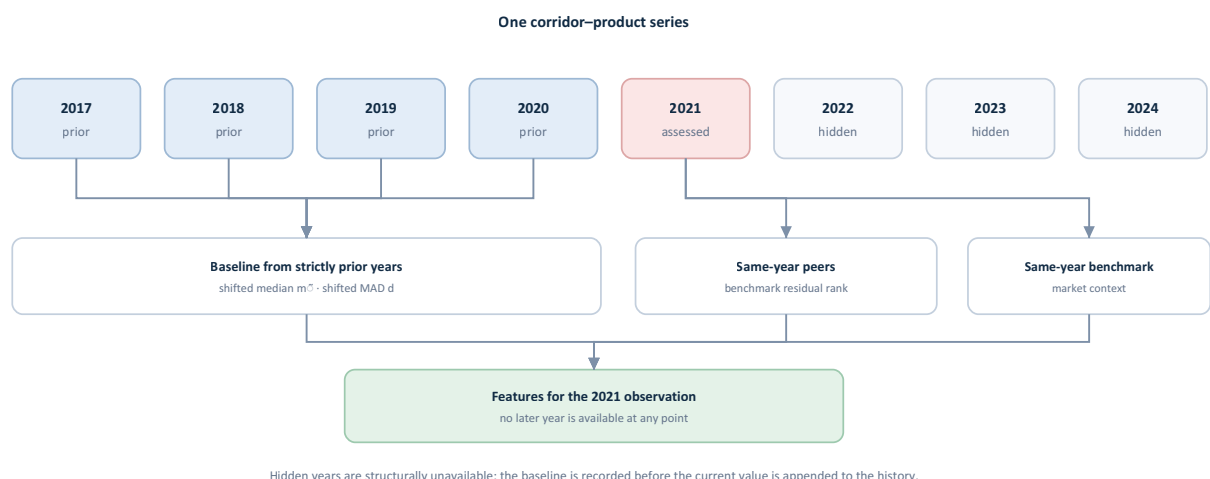


Figure 8.1. Time-safe feature construction for one corridor–product series. Only the shaded prior years contribute to the baseline for the assessed year; later years are structurally unavailable.

8.2 Notation

Table 8.1. Symbols used throughout this chapter.

Symbol	Meaning
c	Corridor — a directed exporter→importer pair
p	HS6 product code
f	Product family (one-to-one with p in this project)
y	Calendar year
$V_{c,p,y}$	Declared trade value in nominal US dollars
$Q_{c,p,y}$	Declared quantity in metric tons
$U_{c,p,y}$	Implied unit value, US dollars per metric ton
$\ell_{c,p,y}$	$\log U_{c,p,y}$ — log implied unit value
$B_{f,y}$	Annual World Bank benchmark price, US dollars per metric ton
$r_{c,p,y}$	Benchmark residual
$\mathcal{H}_{c,p,y}$	The set of strictly prior observed years for corridor–product (c, p)
$\Pi(c, p, y)$	The most recent observed year before y for (c, p)
$\mathbb{1}[\cdot]$	Indicator function: 1 if the condition holds, 0 otherwise

All logarithms are natural. Working in log space makes upward and downward departures symmetric and makes ratios comparable across commodities whose price levels differ by orders of magnitude.

Every feature below is grouped and sorted by (`corridor_id`, `hs6`, `year`) before computation, so “prior rows” means exactly “earlier years for the same corridor and product”.

8.3 Level features

8.3.1 Implied unit value and its logarithm

$$U_{c,p,y} = \frac{V_{c,p,y}}{Q_{c,p,y}} \quad (8.1)$$

where: Defined only where declared value and quantity are both valid and strictly positive; undefined otherwise. · Implemented in `src/tbml_common.py` — `build_panel()`

$$\ell_{c,p,y} = \log U_{c,p,y} \quad \text{undefined where } \text{valid}_{c,p,y}^U = 0 \quad (8.2)$$

where: $\ell_{c,p,y}$ is the log implied unit value — the quantity on which every history and change feature is built. · Implemented in `src/tbml_common.py` — `build_panel()`, `build_features()`

log_unit_value	
Source fields	trade_value_usd , quantity_metric_ton
Grouping level	Row-level — no grouping
Time-safety rule	Uses only the current row's declared value and quantity
Why it exists	Stabilises a heavily skewed quantity so corridors of very different scale become comparable
Contribution to prioritisation	The base quantity for every history and change signal; not itself a strong discriminator
Interpretation	The implied price on a log scale
Missing / zero treatment	Undefined where the unit value is undefined; never imputed
Limitations	An annual average across every flow in the observation

8.3.2 Benchmark price normalisation

Palm oil and copper are published per metric ton and used as they stand. Gold is published per troy ounce and converted:

$$\begin{aligned}
 B_{f,y} &= P_{f,y} \quad \text{for } f \in \{ \text{palm oil, copper} \} \\
 B_{f,y} &= P_{f,y} \cdot \frac{1,000,000}{31.1034768} \quad \text{for } f = \text{gold}
 \end{aligned}
 \tag{8.3}$$

where: $P_{f,y}$ is the published World Bank annual nominal price. Palm oil and copper are published per metric ton and used as they stand; gold is published per troy ounce and converted. · Implemented in `src/tbml_common.py` — `extract_worldbank_benchmarks()`

The workbook's own unit string is asserted against the configured unit at parse time, so a change in the published sheet cannot silently pass through an unconverted price.

8.3.3 Benchmark residual

$$r_{c,p,y} = \log U_{c,p,y} - \log B_{f(p),y} = \log \left(\frac{U_{c,p,y}}{B_{f(p),y}} \right) \tag{8.4}$$

where: $B_{f(p),y}$ is the annual World Bank benchmark for the product family of p , normalised to US dollars per metric ton. Exponentiating gives the benchmark multiple. · Implemented in `src/tbml_common.py` — `build_panel()`

benchmark_residual	
Source fields	unit_value_usd_per_metric_ton , benchmark_price_usd_per_metric_ton
Grouping level	Row-level, against the family-year benchmark
Time-safety rule	Uses the same year's benchmark only
Why it exists	Separates a corridor-specific gap from the commodity's own price level
Contribution to prioritisation	The strongest single driver in the current run (Chapter 10.1.8) and the largest average model contributor
Interpretation	Zero is parity with the market; exponentiating gives the multiple
Missing / zero treatment	Undefined without a unit value or a matched benchmark
Limitations	The benchmark is macro context, not invoice-level fair value

8.4 History features

8.4.1 Shifted corridor historical median

$$\tilde{m}_{c,p,y} = \text{median}\{ \ell_{c,p,y'} : y' \in \mathcal{H}_{c,p,y} \} \quad (8.5)$$

where: $\mathcal{H}_{c,p,y} = \{ y' < y : (c, p, y') \text{ observed with a valid unit value} \}$ — strictly prior years only. Undefined when the corridor–product has no prior observation. · Implemented in `src/tbml_common.py` — `_prior_median_mad()`

where $\mathcal{H}_{c,p,y} = \{ y' < y : (c, p, y') \text{ observed with a valid unit value} \}$.

shifted_corridor_history_median	
Source fields	<code>log_unit_value</code> for the same corridor and product
Grouping level	<code>(corridor_id, hs6)</code> , ordered by year
Time-safety rule	Strictly prior years only — the baseline is recorded before the current value is appended to the history
Why it exists	Gives every route its own baseline instead of a global average
Contribution to prioritisation	The reference point for the robust z-score
Interpretation	What this route-product normally looked like before this year
Missing / zero treatment	Undefined for a corridor's first observed year (30.9% of feature rows)
Limitations	A short history makes the baseline unstable; recorded as a caveat

8.4.2 Shifted corridor historical median absolute deviation

$$d_{c,p,y} = \text{median}\{ | \ell_{c,p,y'} - \tilde{m}_{c,p,y} | : y' \in \mathcal{H}_{c,p,y} \} \quad (8.6)$$

where: The median absolute deviation of the same strictly-prior values: a robust measure of how much this corridor–product used to vary, insensitive to a single past outlier. · Implemented in `src/tbml_common.py` — `_prior_median_mad()`

shifted_corridor_history_mad	
Source fields	<code>log_unit_value</code> for the same corridor and product
Grouping level	<code>(corridor_id, hs6)</code> , ordered by year
Time-safety rule	Strictly prior years only
Why it exists	A robust measure of the route's own variability, insensitive to a single past outlier
Contribution to prioritisation	The denominator of the robust z-score — it makes a movement matter more when the route was historically stable
Interpretation	How much this route-product used to vary, in log units
Missing / zero treatment	Undefined without prior years; a near-zero value is floored at 0.05 in the z-score denominator
Limitations	With two or three prior points the MAD is very noisy

8.4.3 Robust historical z-score

This is the project's central drift signal. It asks: *how far is this year from what this specific route normally did, measured in units of how much this route normally varied?*

$$z_{c,p,y} = \frac{\ell_{c,p,y} - m_{c,p,y}}{\max(d_{c,p,y}, 0.05)} \quad \text{undefined where } \tilde{m}_{c,p,y} \text{ is undefined} \quad (8.7)$$

where: The floor of 0.05 replaces a historical MAD of essentially zero, preventing a degenerate denominator for a corridor whose past unit values were near-identical. The floor is applied in log space, so it corresponds to roughly 5% dispersion. · Implemented in `src/tbml_common.py — build_features()`

The denominator uses a floor of 0.05 whenever the historical MAD is essentially zero. Without it, a corridor that traded at a near-identical unit value for several years would produce a division by ~ 0 and an arbitrarily large z-score for any movement at all. The floor is applied in log space, so it corresponds to roughly a 5% dispersion — small enough not to mask real drift, large enough to prevent a degenerate denominator.

robust_historical_z	
Source fields	<code>log_unit_value</code> , <code>shifted_corridor_history_median</code> , <code>shifted_corridor_history_mad</code>
Grouping level	<code>(corridor_id, hs6)</code>
Time-safety rule	Both baseline terms are strictly prior
Why it exists	The project's core drift signal — measures departure from a route's own past
Contribution to prioritisation	The highest-weighted rules component (0.2) and an evidence metric at threshold 4.0
Interpretation	How many robust deviations this year sits from the route's prior median
Missing / zero treatment	Undefined without a prior baseline; the MAD floor prevents a degenerate denominator
Limitations	Sensitive where history is short or unusually tight — the mandatory evidence caveat says so

8.4.4 Corridor activity history, novelty and reactivation

$$a_{c,p,y} = |\{y' < y : (c, p, y') \text{ observed}\}| \quad (8.8)$$

where: A count of prior appearances of this corridor–product in the panel, computed as the within-group cumulative count over rows sorted by year. · Implemented in `src/tbml_common.py — build_features()`

$$\begin{aligned} \text{novel}_{c,p,y} &= \mathbb{1}[a_{c,p,y} = 0] \\ \text{react}_{c,p,y} &= \mathbb{1}[a_{c,p,y} > 0 \wedge y - \Pi(c, p, y) > 1] \end{aligned} \quad (8.9)$$

where: $\Pi(c, p, y)$ is the most recent observed year before y for this corridor–product. Novelty marks a first appearance; reactivation marks a return after a gap. · Implemented in `src/tbml_common.py — build_features()`

corridor_activity_history	
Source fields	Observed years for the same corridor and product
Grouping level	(corridor_id, hs6)
Time-safety rule	Prior years only
Why it exists	Records how much history supports a comparison
Contribution to prioritisation	Feeds the novelty flag and the data-quality history score
Interpretation	How many earlier years this route-product appeared
Missing / zero treatment	Zero on the first appearance, never missing
Limitations	Counts appearances in this panel only, not real-world trade history

corridor_novelty_flag	
Source fields	corridor_activity_history
Grouping level	(corridor_id, hs6)
Time-safety rule	Prior activity only
Why it exists	A first-ever appearance changes how the other signals should be read
Contribution to prioritisation	Rules weight 0.06 — deliberately the smallest of the nine
Interpretation	This route-product is new in the observed panel
Missing / zero treatment	Always 0 or 1
Limitations	New corridors are overwhelmingly ordinary commerce (7,235 rows carry the flag)

corridor_reactivation_flag	
Source fields	corridor_activity_history, the previous observed year
Grouping level	(corridor_id, hs6)
Time-safety rule	Prior activity pattern only
Why it exists	A route that stopped and returned has a discontinuous baseline
Contribution to prioritisation	Rules weight 0.08
Interpretation	The route-product disappeared and came back
Missing / zero treatment	Always 0 or 1
Limitations	Commodity cycles, policy changes and reporting changes all cause reactivation (2,189 rows)

8.5 Peer features

8.5.1 Same-family, same-year peer percentile

$$\rho_{c,p,y} = \frac{\text{rank}(\Gamma_{c,p,y}; \mathcal{P}_{f(p),y})}{|\mathcal{P}_{f(p),y}|} \quad (8.10)$$

where: $\mathcal{P}_{f,y}$ is the set of all observations in family f and year y with a defined benchmark residual; ranking uses the average method for ties. The result is a share in $[0, 1]$, where 0.5 is the middle of the same-year peer group. · Implemented in `src/tbml_common.py` — `build_features()`

The percentile is computed with average ranking over the benchmark residual, within the same product family and the same year. Using the *residual* rather than the raw unit value is what makes the comparison fair: it removes the commodity's own price level and the year's market conditions before comparing corridors.

same_family_year_peer_percentile	
Source fields	benchmark_residual across the family-year
Grouping level	(family_id, year)
Time-safety rule	Same-year peers only; no future-year distribution
Why it exists	Answers whether a corridor is unusual <i>relative to everyone else facing the same market that year</i>
Contribution to prioritisation	Rules weight 0.1, two-sided; an evidence metric at threshold 0.95
Interpretation	0.5 is the middle of the peer group; the tails are the notable positions
Missing / zero treatment	Undefined without a benchmark residual
Limitations	Peers are compared at HS6 aggregate level and can differ in specification

8.6 Change features

All year-over-year changes use the same log-ratio helper, defined only where both endpoints are strictly positive:

$$\Delta \log X_{c,p,y} = \log \left(\frac{X_{c,p,y}}{X_{c,n,\pi}} \right) \quad \text{defined only where } X_{c,p,y} > 0 \wedge X_{c,p,\pi} > 0 \quad (8.11)$$

where: $\pi = \pi(c, p, y)$ is the previous **observed** year for this corridor-product — not necessarily $y-1$. A zero or missing endpoint yields an undefined value, never an infinity. · Implemented in `src/tbml_common.py` — `_log_change()`

8.6.1 Trade-value, quantity and unit-value change

$$\Delta V_{c,p,y} = \log \frac{V_{c,p,y}}{V_{c,n,\pi}} \quad \Delta Q_{c,p,y} = \log \frac{Q_{c,p,y}}{Q_{c,n,\pi}} \quad \Delta U_{c,p,y} = \log \frac{U_{c,p,y}}{U_{c,n,\pi}} \quad (8.12)$$

where: $\pi = \pi(c, p, y)$ throughout. All three are natural-log ratios against the previous observed year for the same corridor and product. · Implemented in `src/tbml_common.py` — `build_features()`

trade_value_yoy_change	
Source fields	trade_value_usd, previous observed year
Grouping level	(corridor_id, hs6)
Time-safety rule	Previous observation only
Why it exists	Captures sudden movement in declared value
Contribution to prioritisation	Enters the score through value-quantity divergence
Interpretation	Exponentiating gives the multiple against the previous active year
Missing / zero treatment	Undefined where either endpoint is missing or non-positive
Limitations	The previous <i>observed</i> year may not be the previous calendar year

quantity_yoy_change	
Source fields	quantity_metric_ton , previous observed year
Grouping level	(corridor_id, hs6)
Time-safety rule	Previous observation only
Why it exists	Separates value growth from physical growth
Contribution to prioritisation	Enters through value–quantity divergence
Interpretation	Exponentiating gives the volume multiple
Missing / zero treatment	Undefined where either endpoint is missing or non-positive
Limitations	Quantity revisions and rounding move this materially at small volumes

unit_value_yoy_change	
Source fields	unit_value_usd_per_metric_ton , previous observed year
Grouping level	(corridor_id, hs6)
Time-safety rule	Previous observation only
Why it exists	The most directly interpretable movement signal for a valuation review
Contribution to prioritisation	Rules weight 0.12; an evidence metric at threshold 0.80; the input to the benchmark-consistency gap
Interpretation	Exponentiating gives the implied-price multiple
Missing / zero treatment	Undefined where either endpoint is missing or non-positive
Limitations	Annual implied unit values are aggregates, not invoice prices

8.6.2 Value–quantity divergence

$$\delta_{c,p,y} = \Delta V_{c,p,y} - \Delta Q_{c,p,y} \quad (8.13)$$

where: Positive when declared value grew faster than physical quantity; negative when quantity grew faster than value. · Implemented in `src/tbml_common.py — build_features()`

Because both terms are logs, the difference is exactly the log change in unit value when both endpoints exist — which is why the two features can agree on rows where quantity and value are both observed in consecutive years, and diverge where one is missing.

value_quantity_divergence	
Source fields	trade_value_yoy_change , quantity_yoy_change
Grouping level	(corridor_id, hs6)
Time-safety rule	Previous-observation changes only
Why it exists	Isolates the case where declared value moved and volume did not
Contribution to prioritisation	Rules weight 0.12; an evidence metric at threshold 0.80
Interpretation	Positive means value outgrew quantity
Missing / zero treatment	Undefined where either change is undefined
Limitations	Product-mix and terms changes produce the same signature

8.6.3 Benchmark-adjusted drift

$$\Delta r_{c,p,y} = r_{c,p,y} - r_{c,p,\pi(c,p,y)} \quad (8.14)$$

where: The change in the gap to the market, not the change in price. Near zero when a corridor's unit value moved because the whole market moved. · Implemented in `src/tbml_common.py — build_features()`

This is the *change in the gap to the market*, not the change in price. If a corridor's unit value rose because the whole world price rose, the residual is unchanged and this feature is near zero. That is the point: it removes broad market movement before asking whether the corridor moved.

benchmark_adjusted_drift	
Source fields	<code>benchmark_residual</code> , previous observed residual
Grouping level	<code>(corridor_id, hs6)</code>
Time-safety rule	Previous residual only
Why it exists	Removes broad market movement before asking whether the corridor moved
Contribution to prioritisation	Rules weight 0.18; an evidence metric at threshold 0.70
Interpretation	The change in the gap to the market, not the change in price
Missing / zero treatment	Undefined without a previous residual
Limitations	Contract timing, freight, insurance and reporting changes all move it

8.6.4 Benchmark year-over-year change

$$\Delta B_{f,y} = \log \left(\frac{B_{f,y}}{B_{f,y-1}} \right) \quad (8.15)$$

where: The benchmark's own year-over-year log change, computed once per family and year during benchmark extraction and joined onto every observation in that family-year. · Implemented in `src/tbml_common.py — extract_worldbank_benchmarks()`

benchmark_yoy_change	
Source fields	<code>benchmark_price_usd_per_metric_ton</code> for the family
Grouping level	<code>family_id</code> , ordered by year
Time-safety rule	Computed once at benchmark extraction; the same year's value is joined
Why it exists	Supplies the market-movement term the consistency gap needs
Contribution to prioritisation	Indirect — through the benchmark-consistency gap
Interpretation	How much the world price for this commodity moved that year
Missing / zero treatment	Undefined for the first year of the window
Limitations	One world price per family; no regional or grade differentiation

8.6.5 Benchmark consistency gap

$$g_{c,p,y} = |\Delta U_{c,p,y} - \Delta B_{f(p),y}| \quad (8.16)$$

where: How differently the corridor moved from its wider commodity market. Near zero means the movement is market-explained; large means it is not. · Implemented in `src/tbml_common.py — build_features()`

benchmark_consistency_gap	
Source fields	<code>unit_value_yoy_change</code> , <code>benchmark_yoy_change</code>
Grouping level	Row-level against the family-year benchmark change
Time-safety rule	Current-year benchmark change only
Why it exists	The project's principal de-escalation signal
Contribution to prioritisation	Enters as a <i>credit</i> worth up to 0.15, decaying to zero at a gap of 0.35
Interpretation	Near zero means the movement is explained by the market
Missing / zero treatment	Undefined where either change is undefined; contributes no credit
Limitations	A corridor can legitimately diverge from a world benchmark for many reasons

KEY POINT

The benchmark consistency gap is the project's principal *de-escalation* signal. In the rules score it enters as a **credit** — a movement that tracks the market reduces review priority. It is the mechanism by which a large but market-explained move is allowed to stay quiet.

8.7 Data-condition features

These record what is missing. They are deliberately not treated as signals of anything except missingness.

$$\begin{aligned} \text{miss}^Q &= \mathbb{1}[Q \text{ absent}] & \text{miss}^B &= \mathbb{1}[\text{benchmark unmatched}] \\ \text{miss}^H &= \mathbb{1}[\tilde{m} \text{ undefined}] & \text{miss}^C &= \mathbb{1}[\text{ISO3 unmapped}] \end{aligned} \quad (8.17)$$

where: Four explicit binary indicators of what is absent. They record data conditions and are never interpreted as signals of behaviour. · Implemented in `src/tbml_common.py — build_features()`

missing_quantity_flag	
Source fields	quantity_metric_ton
Grouping level	Row-level
Time-safety rule	Reflects the data available for that row
Why it exists	Makes a coverage limitation explicit rather than silent
Contribution to prioritisation	None within the scored population — eligibility already requires a valid quantity, so the flag is constant zero there
Interpretation	Quantity is absent for this observation
Missing / zero treatment	Always 0 or 1
Limitations	Records a data condition; it says nothing about behaviour

missing_benchmark_flag	
Source fields	benchmark_join_status
Grouping level	Row-level
Time-safety rule	Reflects the data available for that row
Why it exists	Marks rows where no market comparison is possible
Contribution to prioritisation	Feeds the data-quality score; 0 rows in the current run
Interpretation	No benchmark matched this family-year
Missing / zero treatment	Always 0 or 1
Limitations	A data condition, not a signal

missing_history_flag	
Source fields	shifted_corridor_history_median
Grouping level	(corridor_id, hs6)
Time-safety rule	Prior years only
Why it exists	Marks rows where no own-history comparison exists
Contribution to prioritisation	Feeds the data-quality score; 7,991 rows (30.9%)
Interpretation	This route-product has no prior observed year
Missing / zero treatment	Always 0 or 1
Limitations	A data condition, not a signal

missing_country_mapping_flag	
Source fields	exporter_iso3 , importer_iso3
Grouping level	Row-level
Time-safety rule	Reflects the data available for that row
Why it exists	Marks rows whose country codes did not map
Contribution to prioritisation	None in the current run — 0 rows carry it
Interpretation	A country code could not be resolved to ISO3
Missing / zero treatment	Always 0 or 1
Limitations	An unmapped code would exclude the row entirely

valid_extreme_flag	
Source fields	unit_value_usd_per_metric_ton , benchmark_price_usd_per_metric_ton
Grouping level	Row-level against the family-year benchmark
Time-safety rule	Same-year benchmark only
Why it exists	Records a real extreme computed from valid inputs, instead of discarding it
Contribution to prioritisation	Rules weight 0.14; also attaches a quality caveat
Interpretation	The implied unit value is beyond 5× or below 0.2× the benchmark
Missing / zero treatment	Always 0 or 1; requires a valid unit value
Limitations	2,710 panel rows (10.5%) carry it — extremes are common in aggregate trade data

data_quality_score	
Source fields	quantity validity, benchmark match, history depth, key completeness
Grouping level	Row-level, with the history term grouped by (corridor_id, hs6)
Time-safety rule	The history component uses prior years only
Why it exists	Separates <i>how much can be said</i> from <i>how unusual it is</i>
Contribution to prioritisation	Deduction only — up to 0.08; it can never raise a score
Interpretation	0–6; the panel mean is 4.51
Missing / zero treatment	Always defined; an undefined score is treated as 0 in the deduction
Limitations	A confidence measure, never a risk measure

A human-readable companion string, `data_quality_flags`, is also written to every feature row in the form `quantity_missing=0;benchmark_missing=0;history_missing=1;country_missing=0`, so a reviewer inspecting a single row does not need to consult four separate columns.

CAUTION

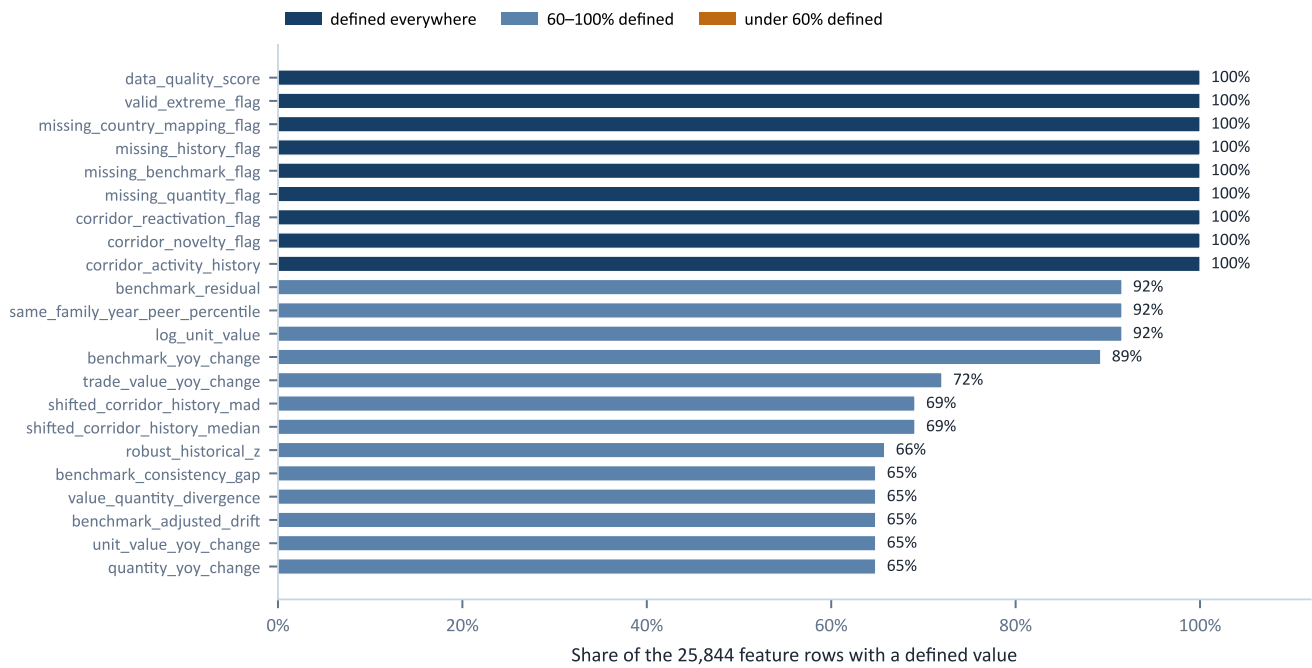
`missing_quantity_flag` is present in the feature schema but is **structurally constant at zero within the scored population**, because model eligibility already requires a valid quantity. It documents a limitation; it does not teach any model how to rank missing-quantity rows. This is verified directly in Chapter 10.2.

8.8 Feature coverage in the current run

Coverage is not uniform, and that is expected: history-based features are undefined for a corridor's first observed year, and change features are undefined without a previous observation.

Table 8.2. Coverage and distribution of every model input feature across the 25,844 feature rows.

Feature	Defined rows	Coverage	5th pct	Median	95th pct
log_unit_value	23,656	91.5%	6.487	9.540	18.013
shifted_corridor_history_median	17,853	69.1%	6.483	10.089	17.855
shifted_corridor_history_mad	17,853	69.1%	0.000	0.061	0.615
robust_historical_z	17,004	65.8%	-13.014	1.474	27.653
same_family_year_peer_percentile	23,656	91.5%	0.051	0.501	0.950
trade_value_yoy_change	18,609	72.0%	-3.811	0.045	3.776
quantity_yoy_change	16,753	64.8%	-3.548	0.000	3.379
unit_value_yoy_change	16,753	64.8%	-1.069	0.054	1.305
benchmark_residual	23,656	91.5%	-1.417	0.003	2.089
benchmark_adjusted_drift	16,753	64.8%	-1.169	-0.003	1.200
corridor_activity_history	25,844	100.0%	0.000	2.000	6.000
corridor_novelty_flag	25,844	100.0%	0.000	0.000	1.000
corridor_reactivation_flag	25,844	100.0%	0.000	0.000	1.000
value_quantity_divergence	16,753	64.8%	-1.069	0.054	1.305
missing_quantity_flag	25,844	100.0%	0.000	0.000	1.000
missing_benchmark_flag	25,844	100.0%	0.000	0.000	0.000
missing_history_flag	25,844	100.0%	0.000	0.000	1.000
missing_country_mapping_flag	25,844	100.0%	0.000	0.000	0.000
valid_extreme_flag	25,844	100.0%	0.000	0.000	1.000
benchmark_consistency_gap	16,753	64.8%	0.007	0.149	2.048
data_quality_score	25,844	100.0%	3.000	4.000	6.000
benchmark_yoy_change	23,061	89.2%	-0.162	0.057	0.408



Source: data/processed/corridor_features.parquet. Undefined values are recorded as missing, never imputed.

Figure 8.2. Feature coverage across the 25,844 feature rows. Level and data-condition features are defined everywhere; history and change features are undefined for first-observed years and after gaps. Missingness is recorded explicitly and never imputed.

8.9 The rules-based ranking components

The transparent baseline scorer, `score_rules()` in `src/06_train_evaluate_models.py`, converts features into nine bounded components, applies fixed weights from `configs/thresholds.yml`, and subtracts two deductions. Every weight and reference value is configuration, not code.

8.9.1 Component normalisation

Seven components use the same bounded-absolute transform, which maps “how far past normal” onto 0–1:

$$\text{bnd}(x; \varrho) = \min\left(\frac{|x|}{\varrho}, 1\right) \quad \text{bnd}(\text{undefined}; \varrho) = 0 \quad (8.18)$$

where: ϱ is the component's configured reference value — the magnitude at which the component saturates. An undefined feature contributes nothing rather than penalising the row. · Implemented in `src/06_train_evaluate_models.py` — `bounded_abs()`

The peer component is two-sided, because both tails matter:

$$s^{\text{peer}} = \min(2|\rho_{c,p,y} - 0.5|, 1) \quad (8.19)$$

where: Distance from the middle of the peer distribution, scaled to $[0, 1]$, so that both the high and the low tail contribute. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`

The novelty, reactivation and valid-extreme components are the flags themselves.

Table 8.3. Rules components, their reference values and their weights (`configs/thresholds.yml`).

Component	Feature	Reference value	Weight	What it captures
Own-history deviation	<code>robust_historical_z</code>	4	0.2	How far the route sits from its earlier pattern

Component	Feature	Reference value	Weight	What it captures
Market-benchmark gap	<code>benchmark_residual</code>	1	0.18	How far its implied unit value is from the benchmark
Change after the benchmark	<code>benchmark_adjusted_drift</code>	0.7	0.18	Whether its benchmark gap changed since the previous observation
Annual implied-value movement	<code>unit_value_yoy_change</code>	0.8	0.12	How sharply its implied unit value changed
Value versus quantity	<code>value_quantity_divergence</code>	0.8	0.12	Whether declared value moved differently from physical volume
Same-product peer position	<code>same_family_year_peer_percentile</code>	0.9	0.1	How extreme the route is among comparable routes that year
New route	<code>corridor_novelty_flag</code>	—	0.06	Whether the route-product has no earlier active year
Returning route	<code>corridor_reactivation_flag</code>	—	0.08	Whether the route-product returned after a gap
Valid extreme value	<code>valid_extreme_flag</code>	—	0.14	Whether a usable row is far above or below its benchmark
Market-explanation credit	<code>benchmark_consistency_gap</code>	0.35	-0.15	Deduction when the corridor moved with its market
Data-reliability penalty	<code>data_quality_score</code>	0–6 scale	-0.08	Deduction when the row is poorly supported

8.9.2 The positive subtotal

$$\begin{aligned}
 S^+ = & 0.20 \text{ bnd}(z; 4.0) + 0.18 \text{ bnd}(r; 1.0) + 0.18 \text{ bnd}(\Delta r; 0.70) \\
 & + 0.12 \text{ bnd}(\Delta U; 0.80) + 0.12 \text{ bnd}(\delta; 0.80) + 0.10 s^{\text{peer}} \\
 & + 0.06 \text{ novel} + 0.08 \text{ react} + 0.14 \text{ extreme}
 \end{aligned}
 \tag{8.20}$$

where: Nine bounded components with fixed weights from `configs/thresholds.yml`. The weights sum to 1.18, so S^+ is a weighted sum rather than a weighted average and can exceed 1 before the deductions are applied. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

The nine weights sum to **1.18**, so this is a weighted *sum*, not a weighted average, and the subtotal can exceed 1 before deductions.

8.9.3 The two deductions

$$C = 0.15 \cdot (1 - \text{bnd}(g_{c,p,y}; 0.35)) \tag{8.21}$$

where: A market-explanation deduction. A corridor that moved exactly with its benchmark loses the full 0.15; the deduction decays to zero at a movement gap of 0.35. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

A corridor that moved exactly with its commodity benchmark receives the full **0.15** deduction. The deduction decays to zero at a movement gap of **0.35**.

$$P = 0.08 \cdot \frac{6 - \min(DQ, 6)}{6} \tag{8.22}$$

where: A data-reliability deduction. A row scoring 6 of 6 loses nothing; each point below 6 subtracts approximately 0.0133, to a maximum of 0.08. An undefined score is treated as 0. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

A row scoring 6 of 6 on data quality loses nothing. Each point below 6 subtracts about **0.0133**, to a maximum of **0.08**.

KEY POINT

Both adjustments are subtractions. Neither can raise a review-priority score. A market-explained movement is de-escalated, and a poorly supported row is ranked more cautiously — never the reverse.

8.9.4 The rules score

$$s_{c,p,y}^{\text{rule}} = \min(\max(S^+ - C - P, 0), 1) \quad (8.23)$$

where: The transparent baseline score, clipped into $[0, 1]$. It is an engineering ranking score with no external or probabilistic meaning. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`

8.10 The hybrid review-priority score

The final ranking score blends the transparent rules score with the selected supervised challenger, using the weight chosen on validation only (Chapter 9.2):

$$s_{c,p,y}^{\text{hyb}} = (1 - w) \cdot s_{c,p,y}^{\text{rule}} + w \cdot s_{c,p,y}^{\text{chal}}, \quad w = 0.75 \quad (8.24)$$

where: s^{chal} is the selected challenger's output and w is the blend weight chosen on the validation split only. Stored as `selected_review_priority_score` for real observations. · Implemented in `src/06_train_evaluate_models.py` — `main()`; `data/outputs/model_selection.json`

with $w = 0.75$ and s^{chal} the XGBoost output. Applied to real observations the same expression is stored as `selected_review_priority_score`.

CAUTION

s^{chal} is a classifier output in the range 0–1 and it is **not** a calibrated probability of anything. Blending it with a rules score produces an ordering, not a likelihood. `data/outputs/model_selection.json` records the constraint in the pipeline itself: "Scores are review-priority ranking scores, not calibrated probabilities of crime."

8.11 Evaluation metrics

The metrics used to compare scorers, all computed in `metric_at_k()` in `src/06_train_evaluate_models.py` over a ranking of the eligible rows in a split.

$$\text{Precision@k} = \frac{|\{\text{positives in top } k\}|}{k} \quad \text{Recall@k} = \frac{|\{\text{positives in top } k\}|}{|\{\text{positives in split}\}|} \quad (8.25)$$

where: Rows are ranked by score descending with `obs_id` ascending as a deterministic tie-breaker; k is capped at the number of rows available. Recall@k is the true-positive rate at that cut-off. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

$$\text{Lift@k} = \frac{\text{Precision@k}}{\omega_{\text{split}}}, \quad \omega_{\text{split}} = \frac{|\{\text{positives in split}\}|}{n_{\text{split}}} \quad (8.26)$$

where: ω_{split} is the injected-positive prevalence in the split. A lift of 1 would match random review. This is a ranking-concentration measure, not an improvement in real-world detection. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

$$\begin{aligned}\text{HN-FPR@k} &= \frac{|\{\text{hard negatives in top } k\}|}{|\{\text{hard negatives in split}\}|} \\ \text{Ord-FPR@k} &= \frac{|\{\text{ordinary negatives in top } k\}|}{|\{\text{ordinary negatives in split}\}|}\end{aligned}\tag{8.27}$$

where: A hard negative is an injected but deliberately benign row; an ordinary negative is any other non-positive row. Both use the score at rank k as the threshold. Lower is better. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

Average precision is computed with `sklearn.metrics.average_precision_score` over the full ranking, which summarises precision across all thresholds rather than only at k . It is used for challenger selection because injected positives are rare (1.790% prevalence on the test split) and a top- k metric alone is noisy at that rate.

KEY POINT

All ranking metrics use `obs_id` as a deterministic tie-breaker, so two runs over identical inputs produce an identical ranking — including where scores tie exactly.

CHAPTER 9

Model Development, Evaluation, and Selection

9.1 Scenario layer for model evaluation

9.1.1 Why confirmed labels are unavailable

Public trade statistics contain no outcome variable. There is no column recording that a flow was later found to be illicit, and no public dataset links a corridor–product–year aggregate to a confirmed money-laundering case. Supervised learning on real labels is therefore not merely difficult here — it is impossible.

Three responses were available. Train on nothing and ship an unvalidated rules score. Use an unsupervised outlier detector and accept that “outlier” and “review-worthy” are different things. Or construct an explicit, auditable evaluation target and be precise about what it does and does not measure. The project takes the third route and states the limitation in every artefact that reports a metric.

9.1.2 Why the scenario layer comes after the panel is frozen

Order is what makes the design defensible. The clean panel is completed and **content-hashed** before any scenario exists:

Table 9.1. Hashes recorded in `scenario_split_manifest.json`.

Artefact	SHA-256 (first 16)
Clean panel at the moment of injection	1552233d8f262081...
Scenario panel after injection	66a21af344a21df2...
Label table	7e9b871919b7cc4d...

`inject_scenarios()` computes the clean-panel hash first, then works on a **copy**. It also refuses to run if the incoming panel already contains `scenario_id` or `synthetic_review_priority`. The consequence is that the clean panel cannot have been tuned to the scenarios, because it was fixed and fingerprinted before they existed.

9.1.3 How the scenarios are constructed

Injection is seeded (`primary_seed = 20,260,117`) and runs within each *split × product family* cell, so positives and hard negatives appear in train, validation and test alike, and in all three commodities. Candidates within a cell must be in-split, in-family, model-eligible and carry a positive quantity. **12 rows per scenario per cell** are selected without replacement.

Table 9.2. The five scenario types.

Scenario	Target	Transformation	Rows injected
<code>synthetic_overvaluation</code>	1	Trade value multiplied by a factor drawn from $U(2.2, 3.5)$; quantity held constant, so implied unit value rises sharply	108
<code>synthetic_undervaluation</code>	1	Trade value multiplied by a factor drawn from $U(0.25, 0.45)$; quantity held constant, so implied unit value collapses	108
<code>synthetic_low_volume_high_value</code>	1	Value multiplied by $U(1.8, 2.8)$ and quantity multiplied by $U(0.35, 0.65)$ — a small shipment reported as worth a lot	108
<code>hard_negative_benchmark_consistent_movement</code>	0	Unit value moved to the family’s median benchmark residual for that year — a large move that is fully explained by the market	108
<code>hard_negative_proportional_value_quantity_growth</code>	0	Value and quantity both scaled by the same factor from $U(2.0, 4.0)$, leaving unit value essentially unchanged	108

In the recorded run the realised mean value multiple was **2.91×** for over-valuation and **0.35×** for under-valuation.

After each transformation, `recompute_panel_price_fields()` recomputes unit value, log unit value, benchmark residual and the valid-extreme flag for that row, so a transformed row remains internally consistent rather than carrying stale derived fields. The complete before/after record — seed, clean-panel hash, split, family, year, scenario, and the value and quantity on both sides — is written to `scenario_injections.csv` (540 rows).

9.1.4 How hard negatives test benign but unusual movement

The two hard negatives are the most important part of the design, because the failure mode that matters operationally is not missing a pattern — it is generating work that leads nowhere.

- **Benchmark-consistent movement** produces a large change in unit value that is *entirely* accounted for by the commodity market. A method that ranks it highly has learned “big move” rather than “unexplained move”.
- **Proportional value and quantity growth** multiplies both sides of the unit-value fraction, so the ratio barely changes while the headline value figure triples or quadruples. A method that ranks it highly has learned “big number” rather than “unusual price”.

Both are labelled 0. Both are counted separately from ordinary negatives, and the **hard-negative false-positive rate** — the share of all hard negatives that reach the top k — is reported for every method, split and family.

9.1.5 How labels are kept separate from features and evidence

Four independent mechanisms, any one of which would catch a leak:

1. **Separate file.** Labels are written to `scenario_labels.parquet`. `scenario_panel.parquet` contains no label column. The manifest states this in the data: “*scenario_id and synthetic_review_priority are stored in scenario_labels.parquet only; they are not in scenario_panel or model feature files.*”
2. **Injection guard.** `inject_scenarios()` raises if the incoming clean panel already contains label metadata.
3. **Feature-build guard.** `build_features()` raises if any of `synthetic_review_priority`, `scenario_id`, `money_laundering`, `fraud`, `criminal` or `tbml_confirmed` appears among the feature columns.
4. **Separate scoring path.** Real observations are scored from `corridor_features.parquet`, built from the clean panel. The labels table is never read on that path.

Labels join to features only inside `fit_models()`, immediately before fitting, on the composite key (`obs_id`, `year`, `family_id`) with a `one_to_one` cardinality assertion.

9.1.6 Splits and leakage controls

The split is **temporal**, not random. Random splitting across years would let a model learn from a corridor’s future to score its past.

Table 9.3. Train / validation / test design.

Split	Years	Rows	Injected positives	Hard negatives	Role
Train	2017, 2018, 2019, 2020	12,318	108	72	Fit the supervised models
Validation	2021, 2022	6,868	108	72	Select the challenger and the blend weight
Test	2023, 2024	6,658	108	72	Report once, after the design is frozen

Positives and hard negatives are balanced by construction: 108 positives and 72 hard negatives in every split. The split sets are asserted disjoint by `obs_id` before the manifest is written.

Leakage controls in force:

- **Temporal.** Time-based split; time-safe features (Chapter 8); baselines from strictly prior years only.
- **Label.** The four mechanisms in 9.1.5.

- **Selection.** The test years are opened once, after the design is frozen. Model choice and blend weight are made on validation alone.
- **Eligibility.** Only model-eligible rows are fitted or scored, on both the scenario and the real path, so the two populations are defined identically.

9.1.7 Scenario limitations

CAUTION

Every metric in this chapter describes how well a method ranks **injected** patterns in a **fixed** synthetic design. The transformations are simple and multiplicative; real abuse need not look like them. Prevalence is set by construction (1.790% on the test split), not observed. No confidence interval, probability calibration, external validation, or live-bank outcome is claimed anywhere. These are model-comparison results, not detection rates.

9.2 Model comparison and selection

9.2.1 The five scorers

Table 9.4. The five scoring methods evaluated.

Method	Type	How it is built	Role
Rules baseline	Deterministic weighted sum	Nine bounded components, fixed weights and reference values from <code>configs/thresholds.yml</code> , minus a market-consistency credit and a data-quality penalty	Transparent, fully auditable anchor. Requires no fitting.
Logistic regression	Linear supervised	Median imputation with missing indicators → standardisation → balanced <code>liblinear</code> logistic regression, <code>max_iter=2000</code>	Linear supervised baseline
XGBoost	Non-linear supervised	160 trees, <code>binary:logistic</code> , <code>aucpr</code> eval metric, histogram method, <code>subsample 0.9</code> , <code>colsample 0.9</code> , <code>reg_alpha 0.15</code> , <code>scale_pos_weight = neg/pos</code> ; 3 small regularised configurations compared	Non-linear challenger
Isolation Forest	Unsupervised	Median imputation → 160-tree Isolation Forest; decision function negated so higher = more anomalous, then min-max scaled to 0–1	Side signal — tests whether “outlier” is the same as “review-worthy”
Hybrid	Blend	$(1 - w) \times \text{rules} + w \times \text{selected challenger}$	The retained scoring approach

All supervised models are fitted on the **train** split only. All are seeded with `random_state = 20,260,117` and `n_jobs=1`, so the run is reproducible.

9.2.2 XGBoost hyper-parameter selection

3 deliberately small, regularised configurations were compared on **validation average precision**. Small trees are a governance choice as much as a statistical one: a depth-2 or depth-3 model over 22 features is far easier to reason about than a deep one, and the scenario set is small.

Table 9.5. XGBoost configurations tested, ranked by validation average precision.

max_depth	learning_rate	min_child_weight	reg_lambda	Validation average precision
2	0.06	2	4	0.3860 ← selected
2	0.09	6	8	0.3797
3	0.05	4	6	0.3706

The selected configuration is `max_depth=2`, `learning_rate=0.06`, `min_child_weight=2.0`, `reg_lambda=4.0`, with validation average precision **0.3860**. Ties are broken deterministically on the sorted parameter tuple.

9.2.3 Challenger selection — on validation only

The supervised challenger is chosen between logistic regression and XGBoost on the configured metric `average_precision` across the **full validation ranking**, with hard-negative false-positive rate as a secondary, ascending tie-breaker.

Table 9.6. Validation results, all product families combined (6,208 eligible rows, 108 injected positives, k = 50).

Method	k	Precision@k	Recall@k	Lift	Average precision	Hard-negative FPR@k	Ordinary FPR@k
Rules baseline	50	6%	2.8%	3.4	0.086	1.39%	0.76%
Logistic regression	50	4%	1.9%	2.3	0.080	1.39%	0.78%
XGBoost	50	62%	28.7%	35.6	0.386	0.00%	0.32%
Isolation Forest	50	8%	3.7%	4.6	0.057	0.00%	0.76%
Hybrid (selected)	50	46%	21.3%	26.4	0.309	1.39%	0.43%

XGBoost wins the challenger contest decisively: average precision **0.386** against **0.080** for logistic regression, **0.086** for the rules baseline and **0.057** for the Isolation Forest. `selected_challenger` is recorded as `xgboost`.

The Isolation Forest result is informative in itself. As a pure outlier detector it reaches only 6% precision at 50 on the test split — barely better than the rules baseline and far below the supervised methods. *Statistically unusual* and *worth reviewing* are demonstrably not the same thing in this data.

9.2.4 Hybrid weight selection — also on validation only

Three blend weights are pre-specified in configuration (0.25, 0.50, 0.75). Each is evaluated on validation using `precision_at_k`, because the operational output is a fixed-capacity 50-row queue. Average precision is a deterministic secondary tie-break, then the more rules-heavy blend.

Table 9.7. Validation results for each candidate blend weight (`hybrid_validation_candidates.csv`).

Candidate blend	Injected patterns in Top 50	Precision@50	Recall@50	Lift	Average precision	Hard-negative FPR@50
75% XGBoost / 25% rules ← selected	23 of 50	46%	21.3%	26.4	0.309	1.39%
50% XGBoost / 50% rules	16 of 50	32%	14.8%	18.4	0.264	1.39%
25% XGBoost / 75% rules	13 of 50	26%	12.0%	14.9	0.199	1.39%

The **75/25** blend — 75% XGBoost, 25% rules — is selected. It placed **23 of 50** validation rows on injected positives, against 16 and 13 for the more rules-heavy blends.

9.2.5 Why validation selects and the test set waits

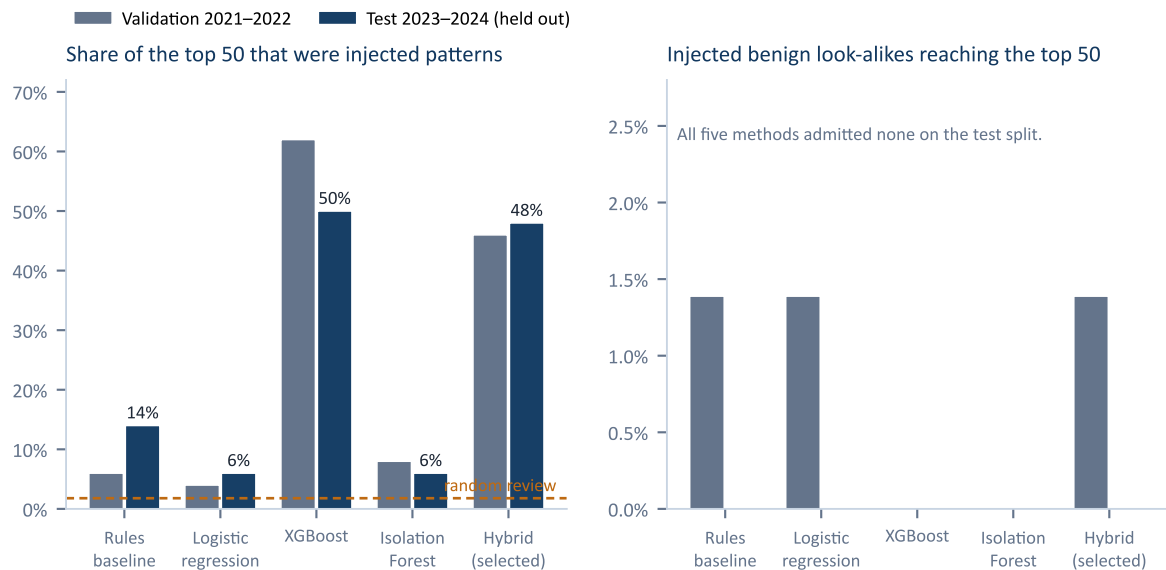
Selecting a model on the same data used to report its performance produces an optimistic number, because the selection has already fitted the noise. The project therefore assigns three separate jobs to three separate time periods: **2017–2020 fits, 2021–2022 selects, 2023–2024 reports once**.

The test years are opened only after the challenger, the blend weight, the feature set, the eligibility rule and the queue size are all fixed. Everything in the next section is a single, unrepeated measurement.

9.2.6 Held-out test results

Table 9.8. Test results, all product families combined (6,033 eligible rows, 108 injected positives, k = 50).

Method	k	Precision@k	Recall@k	Lift	Average precision	Hard-negative FPR@k	Ordinary FPR@k
Rules baseline	50	14%	6.5%	7.8	0.100	0.00%	0.73%
Logistic regression	50	6%	2.8%	3.4	0.077	0.00%	0.80%
XGBoost	50	50%	23.1%	27.9	0.306	0.00%	0.43%
Isolation Forest	50	6%	2.8%	3.4	0.050	0.00%	0.80%
Hybrid (selected)	50	48%	22.2%	26.8	0.298	0.00%	0.44%



Source: data/outputs/model_comparison.csv. Synthetic-scenario results; not real-world detection rates.

Figure 9.1. Model comparison on the held-out 2023–2024 test years. Left: share of each method's top 50 that were injected review-priority patterns. Right: hard-negative false-positive rate — the share of injected benign look-alikes that reached the top 50. Synthetic-scenario results; not real-world detection rates.

Reading the table:

- **The selected hybrid placed 24 of 50** ranked rows on injected patterns — 48% precision at 50 — against an expected **0.90 of 50** under random review. A concentration factor of **26.8×**.
- **No hard negative entered the top 50 for any method** on the test split (0.0%). The over-alerting guard held completely. On validation the hybrid did admit hard negatives at a 1.39% rate, comfortably inside the configured tolerance of 0.2.
- **The ordinary false-positive rate is 0.44%** for the hybrid — the lowest of the five methods alongside XGBoost.
- **Standalone XGBoost scored marginally higher** (50% against 48% precision at 50; average precision 0.306 against 0.298).

That last point is deliberate and is discussed next.

9.2.7 The selected approach and why it was selected

The retained scoring approach is the hybrid: 75% XGBoost plus 25% rules baseline, recorded as `selected_score_column: hybrid_score` in `data/outputs/model_selection.json`.

CAUTION

Standalone XGBoost scored slightly higher on the held-out test. The hybrid is retained **by design, not because it won**. The requirement that the final score carry an auditable, fixed-weight component was fixed before the test set was opened. Choosing the standalone model after seeing the test result would have converted the test set into a second selection set and invalidated it.

Four reasons support the choice:

1. **An auditable anchor.** 25% of the score comes from nine named components with published weights and reference values. A reviewer can recompute that portion by hand.
2. **Governance discipline.** The eligible blends were pre-specified in configuration. Post-hoc substitution of a better-performing option would defeat the split design.

3. **Robustness to a single model's failure mode.** A gradient-boosted model fitted on 108 injected positives can learn scenario-specific structure. The rules component is unaffected by that risk.
4. **Negligible operational cost.** The difference at the queue cut-off is one row in fifty.

9.2.8 Weaknesses and governance considerations

Table 9.9. Known weaknesses of the retained approach.

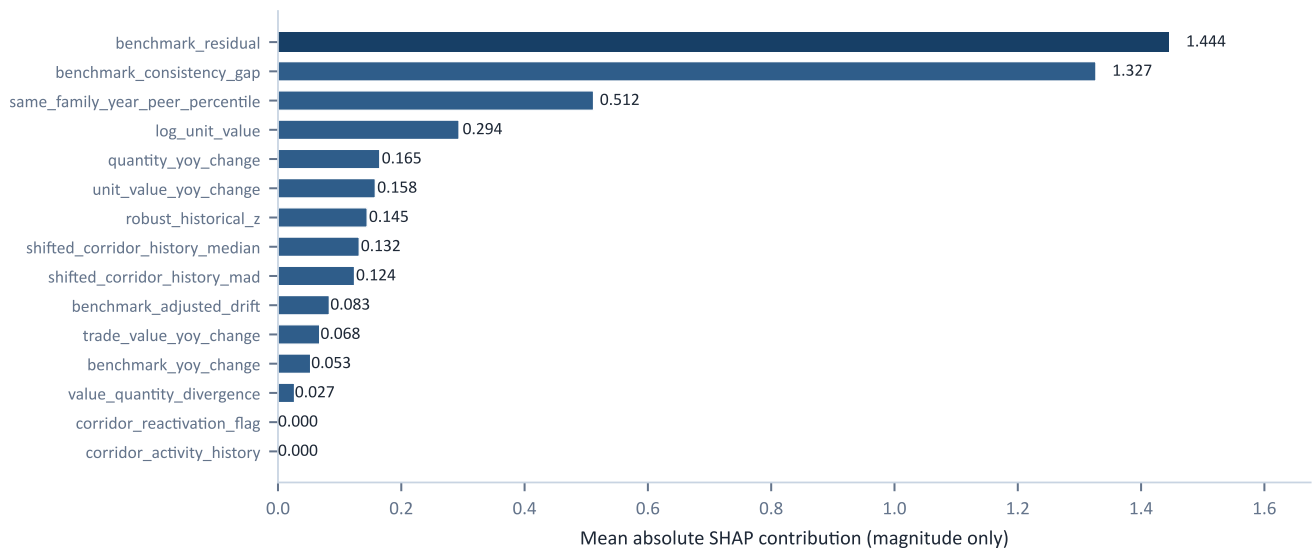
Weakness	Consequence	Mitigation in place
Trained on synthetic labels	Metrics do not estimate real detection	Stated on every metric surface; scenario limitation section in the dashboard and here
No calibration	Score is not a probability	Never presented as one; <code>score_language</code> recorded in the pipeline output
No uncertainty quantification	Differences between methods are point estimates	Reported as descriptive; ranking-concentration language used throughout
Family-level variation	Test average precision 0.590 for gold against 0.183 for palm oil	Family-level metrics published in full; framed as a model-performance warning, not a commodity-risk statement
Only global explanation	No per-row attribution	SHAP labelled as global and confined to the governance section; case evidence comes from recomputable features
25% rules is a formula weight	It is not a guarantee of interpretability	Stated explicitly; interpretability also rests on the evidence table
Blend weights limited to three	A finer grid might do better	Pre-specified for governance; the full validation grid is published

Table 9.10. Held-out test results by product family, selected hybrid score.

Product family	Rows scored	Injected positives	Precision@50	Recall@50	Lift	Average precision	Hard-negative FPR@50
Non-monetary unwrought gold	2,608	36	48%	66.7%	34.8	0.590	0.0%
Refined copper cathodes	1,698	36	34%	47.2%	16.0	0.235	0.0%
Crude palm oil	1,727	36	16%	22.2%	7.7	0.183	0.0%

9.2.9 Model contribution — global SHAP

A global mean-absolute-SHAP summary is computed for the XGBoost challenger over a fixed, seeded sample of 500 eligible real observations. The strongest average contributor is `benchmark_residual`.



Source: data/outputs/shap_summary_values.csv — a fixed 500-row sample of eligible real observations. Model explanation, not case evidence.

Figure 9.2. Global mean absolute SHAP contribution per feature for the XGBoost challenger, over a fixed 500-row sample of eligible real observations. Magnitude only — this shows how strongly an input moved the model output on average, not direction, not causation, and not case evidence.

CAUTION

SHAP is **model explanation**, not factual evidence. It describes how an input moved a model's output. It says nothing about the trade itself. Row-level SHAP is not produced by the pipeline and is therefore never displayed per case.

9.3 Application of the selected model to real trade observations

9.3.1 How the selected model transfers

The scenario layer produces three things: fitted models, a selected challenger, and a selected blend weight. Only those three cross into the real-observation path. The scenario **data** does not.

The transfer is explicit in `src/06_train_evaluate_models.py`:

```
# ---- Apply the selected hybrid to the REAL official observations ----
# Score the real official observations. This ranking excludes synthetic scenario rows entirely.
official_features = pd.read_parquet(DATA_PROCESSED / "corridor_features.parquet")
official_eligible = official_features[official_features["model_eligible"]].copy()
official_rule = score_rules(official_eligible)
official_ml = add_model_scores(official_eligible, cols, logistic, xgb, iso)
official_scores = official_eligible.merge(official_rule, on="obs_id", validate="one_to_one") \
    .merge(official_ml, on="obs_id", validate="one_to_one")
official_scores["selected_challenger_score"] = official_scores[selected_col]

# The final review-priority score uses the same hybrid weight chosen on validation.
official_scores["selected_review_priority_score"] = (
    (1 - selected_weight) * official_scores["rule_score"]
    + selected_weight * official_scores[selected_col])

# Take the top-k highest-scoring official rows as the review queue.
top = official_scores.sort_values(["selected_review_priority_score", "obs_id"],
    ascending=[False, True], kind="mergesort").head(top_k).copy()
top.insert(0, "rank", range(1, len(top) + 1))
```

Source: `src/06_train_evaluate_models.py` — `main()`

`corridor_features.parquet` is the feature table built from the **clean** panel. It contains no scenario row and no label column. The same `score_rules()` function and the same fitted models are applied, and the same blend weight $w = 0.75$ — chosen on validation — is used.

9.3.2 Which records are eligible, and which are not

Table 9.11. Scoring eligibility across the real panel.

Population	Rows	Basis
All panel observations	25,844	Every source row is retained
Scored (model-eligible)	23,656 (91.5%)	Valid positive value and valid positive quantity and quality status usable
Not scored	2,188	Valid declared value but no usable quantity

The unscored rows are not rejected by any model. They are excluded by an arithmetic fact: without a usable quantity there is no implied unit value, and every unit-value signal in the feature set is undefined. 0 observations in the review queue have a missing quantity — the eligibility rule is verified to hold at the output, not merely intended.

KEY POINT

An unscored record is a **coverage limitation**, never a signal. The project reports these records separately as a data-quality follow-up (Chapter 10.2 and dashboard page *Unscored Gold Records*) and never allows missingness to enter the anomaly ranking.

9.3.3 How rankings are produced

All 23,656 eligible observations receive `selected_review_priority_score`. They are sorted descending, with `obs_id` ascending as a deterministic tie-breaker, and `rank` is assigned 1...N.

9.3.4 Why real records do not inherit synthetic labels

There is no code path by which they could. `top_ranked_corridors.csv` is built from `official_scores`, which derives from `corridor_features.parquet` — a table with no label column, built from a panel that predates the scenarios and is hashed to prove it. The dashboard's Model Evaluation & Controls page carries a live integrity check confirming that no queue row is a scenario row.

9.3.5 How the Top 50 queue is created

The top 50 rows by score become the queue, `top_ranked_corridors.csv`, with 21 columns per row: rank and identity, the declared value and quantity, the implied unit value and the benchmark, the blended score with its rules and challenger components, quality status and data-quality score, the valid-extreme flag, the evidence count, and the provenance stamps.

Table 9.12. Composition of the Top 50 review queue.

Dimension	Distribution
Score range	0.9542 – 0.9891 (median 0.9647)
By family	18 gold (36%) · 18 palm oil (36%) · 14 copper (28%)
Peak year	2022 (16 of 50)
Quality status	45 <code>usable_with_caveat</code> · 5 <code>fully_usable</code>
Carrying the valid-extreme flag	39 of 50
Distinct exporters / importers	34 / 33
Evidence rows per case	4 (uniform)

Table 9.13. The twelve highest-ranked real trade observations. Multiple is implied unit value ÷ same-year benchmark. A high rank means the row shows unusual analytical signals; it is not a finding of wrongdoing.

Rank	Year	Corridor	Product	Trade value (USD)	Quantity (mt)	Unit value (USD/mt)	Benchmark (USD/mt)	Multiple	Score
1	2022	ESP → NLD	Gold	\$11,787,415	0.032	\$368,356,719	\$57,890,634	6.36×	0.9891
2	2022	KWT → NPL	Gold	\$2,912,170	0.009	\$323,574,444	\$57,890,634	5.59×	0.9858
3	2022	QAT → NPL	Gold	\$90,853,064	0.280	\$324,475,229	\$57,890,634	5.60×	0.9855
4	2022	HKG → NPL	Gold	\$3,899,093	0.013	\$299,930,231	\$57,890,634	5.18×	0.9855
5	2022	SAU → NPL	Gold	\$23,978,692	0.066	\$363,313,515	\$57,890,634	6.28×	0.9847
6	2022	ITA → MKD	Palm oil	\$39	0.006	\$6,500	\$1,276	5.09×	0.9802
7	2024	GBR → ITA	Copper	\$2,086	0.044	\$47,409	\$9,142	5.19×	0.9802
8	2022	CAF → CHE	Gold	\$3,179,015	0.018	\$176,611,944	\$57,890,634	3.05×	0.9788
9	2018	USA → IRL	Copper	\$264,595	6.879	\$38,464	\$6,530	5.89×	0.9787
10	2020	ITA → TUR	Copper	\$24,289,083	628.199	\$38,665	\$6,174	6.26×	0.9753
11	2022	SGP → JPN	Copper	\$112,739	2.090	\$53,942	\$8,822	6.11×	0.9747
12	2024	CHN → GHA	Copper	\$527	0.011	\$47,909	\$9,142	5.24×	0.9732

9.3.6 What the score does and does not mean

Table 9.14. Reading a review-priority score.

The score does	The score does not
Order the eligible population for review	Estimate any probability
Concentrate injected patterns near the top under controlled test	Measure real-world detection performance
Combine an auditable component with a learned one	Provide a complete explanation on its own
Stay comparable across corridors and families	Carry any external or absolute meaning
Reflect only public data available at that year	Account for anything the bank knows

9.4 Evidence-grounded review of the Top 50 ranked queue records

9.4.1 From score to evidence

`src/07_build_evidence.py` does not decompose the score. It re-reads the feature table for each queued observation, evaluates every metric in a fixed specification, ranks them by how far each exceeds its own threshold, keeps the strongest four, and emits one fully described row per kept metric. That independence is what makes an evidence row recomputable: a reviewer can check it against the feature table without reference to any model.

$$\begin{aligned}
 M(m, x) &= 2|x - 0.5| && \text{for } m = \text{same_family_year_peer_percentile} \\
 M(m, x) &= |x| && \text{otherwise}
 \end{aligned}
 \tag{9.1}$$

where: The non-negative strength of metric m at observed value x . The peer percentile is two-sided, so both a very high and a very low position count as notable. · Implemented in `src/07_build_evidence.py` — `metric_magnitude()`

$$\begin{aligned}
 \text{sev}(M, \tau) &= \text{high} && \text{when } M/\tau \geq 2 \\
 \text{sev}(M, \tau) &= \text{medium} && \text{when } 1 \leq M/\tau < 2 \\
 \text{sev}(M, \tau) &= \text{low} && \text{when } M/\tau < 1
 \end{aligned} \tag{9.2}$$

where: τ is the metric's configured threshold. The four metrics with the largest M/τ are kept for each queued observation. · Implemented in `src/07_build_evidence.py` — `severity()`, `build_evidence()`

9.4.2 Evidence types and thresholds

Table 9.15. The 8 evidence metric specifications (`METRIC_SPECS` in `src/07_build_evidence.py`).

Metric	Evidence type	Threshold	Direction	Comparison group
<code>robust_historical_z</code>	<code>history_deviation</code>	4	absolute	same exporter-importer-HS6 corridor, prior years only
<code>benchmark_residual</code>	<code>benchmark_gap</code>	0.7	absolute	same-family World Bank annual commodity benchmark
<code>benchmark_adjusted_drift</code>	<code>benchmark_adjusted_drift</code>	0.7	absolute	previous observed same-corridor benchmark residual
<code>unit_value_yoy_change</code>	<code>annual_unit_value_change</code>	0.8	absolute	previous observed same-corridor-HS6 year
<code>value_quantity_divergence</code>	<code>value_quantity_divergence</code>	0.8	absolute	same corridor-HS6 value growth versus quantity growth
<code>same_family_year_peer_percentile</code>	<code>peer_position</code>	0.95	high_or_low	same HS6/family and year peer corridors
<code>corridor_novelty_flag</code>	<code>corridor_activity</code>	1	high	prior active years for same corridor-HS6
<code>corridor_reactivation_flag</code>	<code>corridor_activity</code>	1	high	prior active years and gap pattern for same corridor-HS6

Each evidence row records: a content-derived `evidence_id`, the `obs_id`, the source table and source row ID, the evidence type, the metric name, the observed value, the comparison value where one exists, the comparison group, the threshold, the direction, the severity, the contributing source fields, the source version, a fixed-wording plain-English summary, and a mandatory caveat.

Table 9.16. Evidence produced for the current queue.

Measure	Value
Evidence rows	200
Cases covered	50 of 50
Rows per case	4 (uniform — the per-case cap)
High severity	152
Medium severity	48
Low severity	0

Table 9.17. Evidence rows by type.

Evidence type	Rows	Share
<code>history_deviation</code>	47	24%
<code>annual_unit_value_change</code>	46	23%
<code>benchmark_adjusted_drift</code>	45	22%
<code>benchmark_gap</code>	33	16%
<code>value_quantity_divergence</code>	29	14%

CAUTION

Every case in the current queue carries exactly 4 evidence rows, because the per-observation cap is four and every queued case has at least four metrics available. Evidence **count** therefore carries no information in this run and must not be read as evidence *strength*. Severity and the observed values carry that information. The dashboard documents the same limitation.

9.4.3 A worked example — rank 1

Rank 1 — 2022 ESP → NLD · Non-monetary unwrought gold	
Observation ID	obs_c1178b54e91326a9b0f5
Review rank / score	1 · 0.9891 (rules 1.0000 · challenger 0.9855)
Year	2022
Corridor	Spain → Netherlands (ESP → NLD)
Product	Non-monetary unwrought gold · HS6 710812
Declared trade value	\$11,787,415
Declared quantity	0.032 metric tons
Implied unit value	\$368,356,718.75 per metric ton
World Bank benchmark	\$57,890,634.27 per metric ton
Unit value vs benchmark	6.36× benchmark
Peer position	99.36% of 1,405 same-product, same-year corridors
Data quality	usable_with_caveat · 6 of 6
Provenance	baci_0805b6c6be2600ba0835c697

The four evidence rows attached to this observation:

Table 9.18. Evidence rows for the rank-1 observation, obs_c1178b54e91326a9b0f5 .

Evidence ID	Metric	Observed	Threshold	Severity	Required caveat
ev_64fe9330ad36973b98	robust_historical_z	18.827	4.00	high	Short or low-dispersion history can make this metric less stable.
ev_89c989e7cf036ef947	benchmark_adjusted_drift	1.858	0.70	high	Annual residual movement can reflect contract timing, quality, freight, insurance, or reporting differences.
ev_03b1acd36c34032643	benchmark_residual	1.850	0.70	high	The benchmark is broad market context, not invoice-level fair value.
ev_d25e9b9a9f2e0660b3	unit_value_yoy_change	1.859	0.80	high	Aggregate annual unit values are not invoice prices.

The full time-safe signal profile for the same observation:

Table 9.19. Complete signal profile for the rank-1 observation.

Field	Display label	Case value	Time-safety rule
benchmark_adjusted_drift	Change in benchmark gap	1.8583	Previous observation only
benchmark_consistency_gap	Difference from wider market movement	1.8583	Current-year benchmark change only
benchmark_residual	Distance from annual benchmark	1.8505	Current-year benchmark only
corridor_activity_history	Prior active-year history	5.0000	Prior activity only

Field	Display label	Case value	Time-safety rule
log_unit_value	Log implied unit value	19.7246	Current row only
quantity_yoy_change	Change in declared quantity	-1.2792	Previous observation only
robust_historical_z	Deviation from prior history	18.8267	Prior years only
same_family_year_peer_percentile	Position among same-product peers	0.9936	Same-year peers only
shifted_corridor_history_mad	Prior corridor dispersion (log MAD)	0.1131	Prior years only
shifted_corridor_history_median	Prior corridor median (log unit value)	17.5955	Prior years only
trade_value_yoy_change	Change in declared trade value	0.5796	Previous observation only
unit_value_yoy_change	Change in implied unit value	1.8588	Previous observation only
value_quantity_divergence	Value versus quantity movement	1.8588	Previous observation only

Reading it: the corridor's implied unit value sits **6.36× the 2022 benchmark**, is **99.36%** of 1,405 same-product corridors that year, and is **18.8267 robust deviations** above the median of its own 5.0000 prior observed years — a corridor history that was itself tight (MAD 0.1131 in log space). The benchmark-adjusted drift of 1.8583 says the gap to the market widened rather than the market moving. Quantity fell (-1.2792 in logs) while value rose (0.5796), giving the value–quantity divergence.

CAUTION

That is a description of measurements, not a conclusion. The declared quantity is 0.032 metric tons — a very small denominator, where a rounding difference in the reported quantity moves the implied unit value substantially. Gold's HS6 code does not reveal purity, form or commercial terms. This record is a **priority for human review**, and nothing more.

9.4.4 Data-quality caveats attached to ranked records

39 of the 50 queued records carry the valid-extreme flag, and 45 `usable_with_caveat` · 5 `fully_usable` describes their quality statuses. Each is a usable observation with an attached caveat, not an error. Data-quality caveats are displayed alongside the ranking and never folded into it.

9.4.5 Benign explanations that must be considered first

For any ranked record, all of the following can produce the observed signature and none of them is wrongdoing:

- **Product heterogeneity within the HS6 code** — different grade, purity, form or specification traded under the same code.
- **Commercial terms** — a change in Incoterms moving freight and insurance into or out of the declared value.
- **Contract timing and hedging** — a price agreed in an earlier period, settled in this one.
- **Re-export, transshipment or hub effects** — value booked through an intermediary jurisdiction.
- **Small-denominator arithmetic** — a residual flow where a tiny quantity makes the implied unit value volatile.
- **Market shocks** — a commodity price movement affecting the whole family, which the benchmark-consistency gap is designed to detect.
- **Reporting and classification changes** — a national convention change, a reclassification, or a revision.

The typology card `TBML_BENIGN_MARKET_SHOCK_CAVEAT` makes this explicit: *“A risk indicator is not conclusive; commodity price shocks, freight, quality, and contract effects can be benign explanations.”*

9.4.6 Information that is missing and must be obtained

The typology card `TBML_MISSING_PRIVATE_INFO_REVIEW_STEPS` records that several useful indicators require data held by banks, customs or counterparties. For any queued record the project cannot supply: the parties; the invoices, contracts and customs declarations; the shipping and inspection records; the payment routing; beneficial ownership; sanctions and adverse-information context; or the customer's expected activity profile.

9.4.7 Recommended human-review steps

Drawn from the typology cards' `recommended_human_review_steps` :

1. Compare invoices, contracts, customs declarations and settlement records against the declared aggregate.
2. Check product grade, purity, form, freight, insurance, Incoterms and contract timing.
3. Obtain shipping documents, bills of lading and customs inspection records to establish whether goods moved as declared.
4. Compare the movement with market benchmarks and with peer corridors before treating it as unusual.
5. Gather customer due diligence, ownership, payment, invoice and logistics records.
6. Assess whether the trade volumes are commercially plausible for the entities involved.

9.4.8 How typology context stays context

Typology cards attach to a case by **feature relationship**, not by row. `TBML_VALUATION_CONTEXT` relates to `benchmark_residual`, `benchmark_adjusted_drift`, `robust_historical_z` and `same_family_year_peer_percentile`; a case whose evidence involves those features may be discussed in that vocabulary. The card then supplies the allowed phrasing (*"The observation has valuation-style features that merit human review"*), the prohibited phrasing (*"This proves over-invoicing, money laundering, fraud, or criminal intent"*), the required caveats, and the review steps.

KEY POINT

A typology card never becomes a fact about a row. It supplies vocabulary and constraints for describing a pattern the evidence already establishes. `src/08_build_briefs.py` enforces the constraint mechanically: any brief containing a prohibited phrase fails validation.

9.4.9 Why SHAP is explanation, not evidence

Table 9.20. Model contribution compared with factual evidence.

	Global SHAP	Evidence row
What it describes	How an input moved the model's output, averaged over a sample	An observed value against a named comparison and threshold
Scope	Global (500-row sample)	One specific observation
Recomputable without the model?	No	Yes — from the feature table alone
Depends on the model?	Entirely	Not at all
Survives a model change?	No	Yes
Where it appears	Model-governance section only, labelled global	Case evidence panel and analyst briefs

9.4.10 The analyst-brief layer

`src/08_build_briefs.py` renders 5 deterministic briefs for the top-ranked cases from the queue, the evidence rows and the typology cards. **No language model is called.** The script also writes prompt assets — a system prompt, a user template and an output schema — that document how a future model *could* be constrained, but nothing in the repository invokes one.

Every brief is validated:

Table 9.21. Brief validation checks — all 5 briefs pass.

Check	What it enforces	Result across all briefs
<code>evidence_id_coverage</code>	Every evidence ID cited by the brief exists in the evidence table	all pass
<code>unsupported_numeric_claim_count</code>	Numbers in the brief that no evidence row supports	0
<code>hallucinated_source_count</code>	Cited sources that are not declared project sources	0
<code>missing_caveat_flag</code>	Brief omits the required caveats	0 briefs flagged
<code>prohibited_language_flag</code>	Brief asserts crime, fraud, or confirmed TBML	0 briefs flagged
<code>conclusion_boundary_present</code>	Exact human-review boundary sentence present	all pass
<code>typology_card_ids_present</code>	Brief cites real FATF–Egmont typology card IDs	all pass

CHAPTER 10

Comprehensive Data Analytics and Profiling

Two analytical notebooks in the repository interrogate the outputs rather than produce them. This chapter converts both into report narrative. Every figure and table below was **regenerated from the current repository data** rather than copied from a stored cell output, so the numbers here are live. The notebooks were **inspected, not executed** — their analytical logic was reimplemented in `extract_facts.py` and `make_charts.py`, and where a notebook records an expected value the regenerated value was checked against it.

Table 10.1. The two source notebooks.

Notebook	Cells	Markdown / code	Size	Question it answers
<code>data-profiling/business_Review_Data_profiling_before_revamp.ipynb</code>	34	16 / 18	6.11 MB	Is the queue actually selective, and what drives it?
<code>data-profiling/gold_nan_quantity_analysis.ipynb</code>	24	13 / 11	0.42 MB	Where does the unit-value method have no coverage, and does that matter?

10.1 Business review and data profiling

10.1.1 Reconstructing the full scored population

The pipeline persists scores for the Top 50 only. Every question in this section needs the **whole** eligible population as a denominator, so the notebook reconstructs it: load `model_bundle.joblib`, apply the same `score_rules()` and `add_model_scores()` functions to all model-eligible real observations, and re-blend with the recorded weight. That yields **23,656 scored real observations** — the same count as `model_eligible` in the panel, which is the first consistency check.

KEY POINT

Without this reconstruction, a score-distribution chart would describe only the queue — fifty rows already selected for being extreme — and would look impressive while saying nothing. Comparing the queue against the full eligible population is what makes the selectivity claim testable.

10.1.2 Is the queue in the tail of the population?

Question. Does the Top 50 occupy the right tail of the full eligible population, or does it merely reflect small differences among already-similar rows?

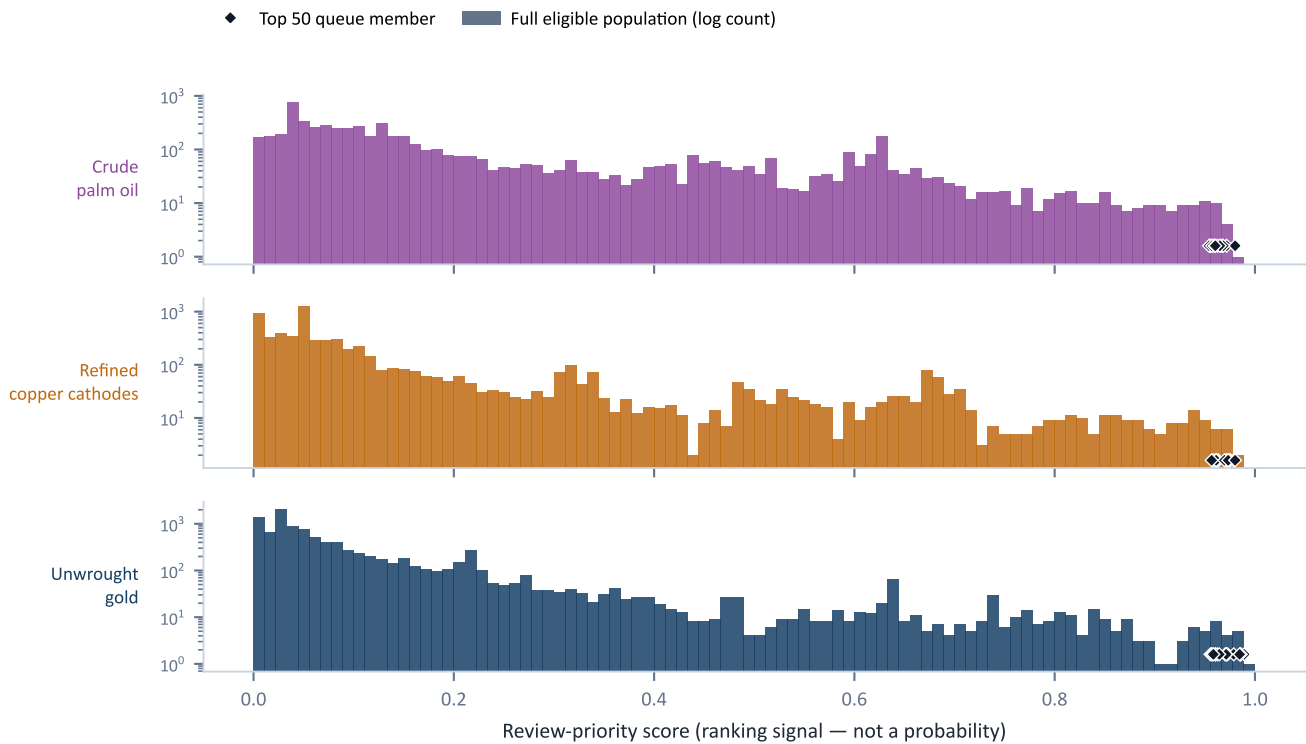
Why it matters. This is the primary test of whether the ranking is doing any work at all. A queue drawn from the middle of a flat distribution would be arbitrary.

Method. Score every eligible observation, plot the distribution per product family, and mark where the 50 queue rows fall.

Result.

Table 10.2. Review-priority score distribution across the full eligible population, by product family.

Product family	Eligible rows	In Top 50	Median score	95th percentile	Maximum
Crude palm oil	6,630	18	0.129	0.683	0.980
Non-monetary unwrought gold	10,328	18	0.048	0.428	0.989
Refined copper cathodes	6,698	14	0.059	0.674	0.980



Source: reconstructed from data/outputs/model_bundle.joblib over the 23,656 eligible real observations. Counts are on a log scale.

Figure 10.1. Review-priority score across the full eligible real population by product family, with the Top 50 queue positions marked. The queue occupies the extreme right tail of every family. Scores order the queue; they are not probabilities.

The population medians are 0.129 (palm oil), 0.048 (gold) and 0.059 (copper); the 95th percentiles are 0.683, 0.428 and 0.674. The queue's lowest score is **0.9542** — above the 95th percentile of every family.

Business interpretation. The queue is genuinely selective. An analyst working it is not reviewing marginal rows.

Data-quality caveat. Gold's much lower median score (0.048) reflects the shape of its distribution, not lower risk. Scores are comparable **within** the ranking; they carry no absolute meaning.

How it informs the process. It justifies a fixed-capacity queue. The score falls away sharply below the cut-off, so extending the queue adds progressively weaker candidates — which the review-capacity analysis in the dashboard quantifies directly.

10.1.3 Queue composition by product family

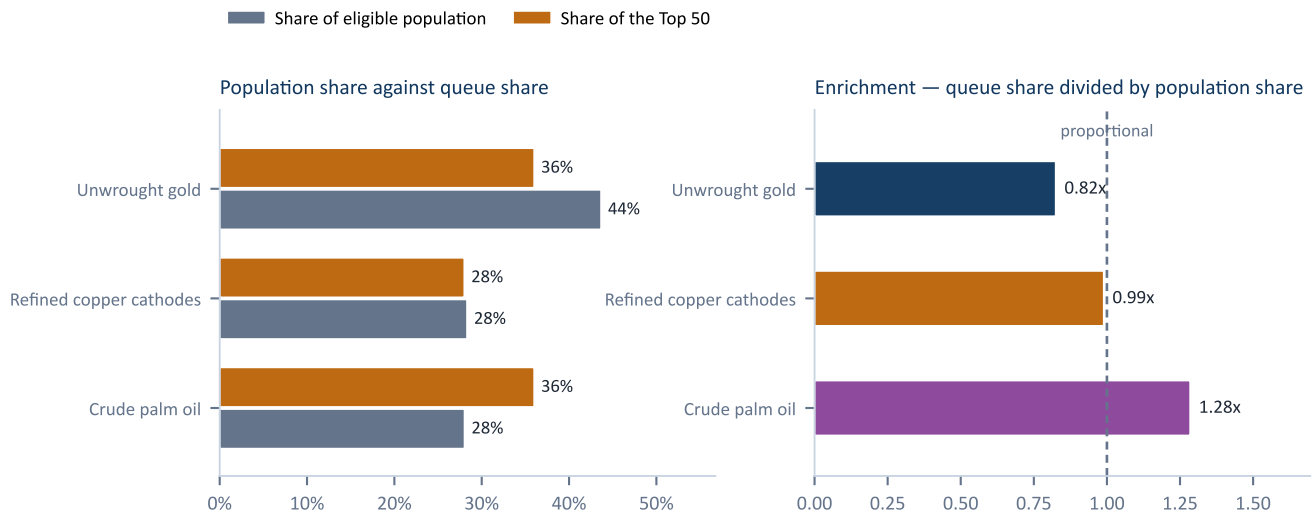
Question. Is the queue dominated by one commodity?

Method. Count queue rows by family; compare with each family's share of the eligible population.

Result.

Table 10.3. Top 50 composition against population share.

Product family	Share of eligible population	Share of Top 50	Enrichment
Crude palm oil	28.0%	36.0%	1.28×
Refined copper cathodes	28.3%	28.0%	0.99×
Non-monetary unwrought gold	43.7%	36.0%	0.82×



Source: reconstructed scores over the 23,656 eligible real observations and data/outputs/top_ranked_corridors.csv.

Figure 10.2. Queue composition adjusted for population share. Left: each family's share of the eligible population against its share of the Top 50. Right: the ratio of the two. A value of 1.00x means a family appears in the queue exactly as often as its size would predict.

Business interpretation. The queue is spread across all three families and no family dominates. Palm oil is over-represented at **1.28x** its population share, copper is almost exactly proportional at **0.99x**, and gold is *under*-represented at **0.82x** despite being the largest family in the panel.

KEY POINT

The enrichment view is the honest one. A raw count chart would show gold and palm oil tied at 18 rows each and suggest gold is heavily represented. Adjusted for the fact that gold is 43.7% of the scored population, gold is actually the *least* enriched family in the queue.

Data-quality caveat. Family composition is a property of this run at this queue size. It is not a statement about commodity risk.

How it informs the process. A cross-commodity queue means an analyst cannot specialise away from it, and it is evidence that the method is not simply learning one commodity's price scale.

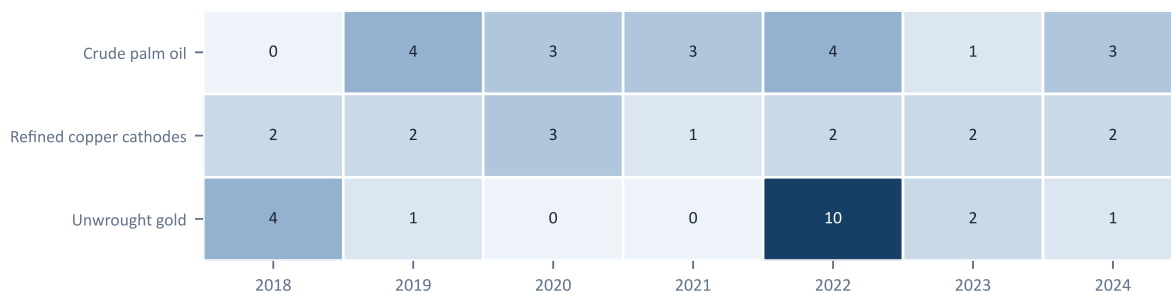
10.1.4 Year concentration

Question. Does the queue cluster in particular years, and if so which?

Result.

Table 10.4. Top 50 records by year and product family.

Product family	2018	2019	2020	2021	2022	2023	2024	Total
Crude palm oil	—	4	3	3	4	1	3	18
Refined copper cathodes	2	2	3	1	2	2	2	14
Non-monetary unwrought gold	4	1	—	—	10	2	1	18
All families	6	7	6	4	16	5	6	50



Source: data/outputs/top_ranked_corridors.csv. Cell shade and the printed number both give the count. Counts, not rates — a year cluster can reflect market conditions or reporting coverage.

Figure 10.3. Top 50 review candidates by year and product family. The 2022 gold cluster is the dominant concentration in the current run.

2022 carries 16 of 50 candidates, driven almost entirely by gold.

Business interpretation. A year cluster is a prompt to check market context before reading anything into it. 2022 was a year of elevated gold prices and heightened commodity volatility generally.

Data-quality caveat. Reporting coverage also changes year to year; a cluster can reflect data as easily as trade.

How it informs the process. In a bank deployment, a year-concentrated queue would be a signal to re-check whether a market event, not a behaviour change, is driving the ranking.

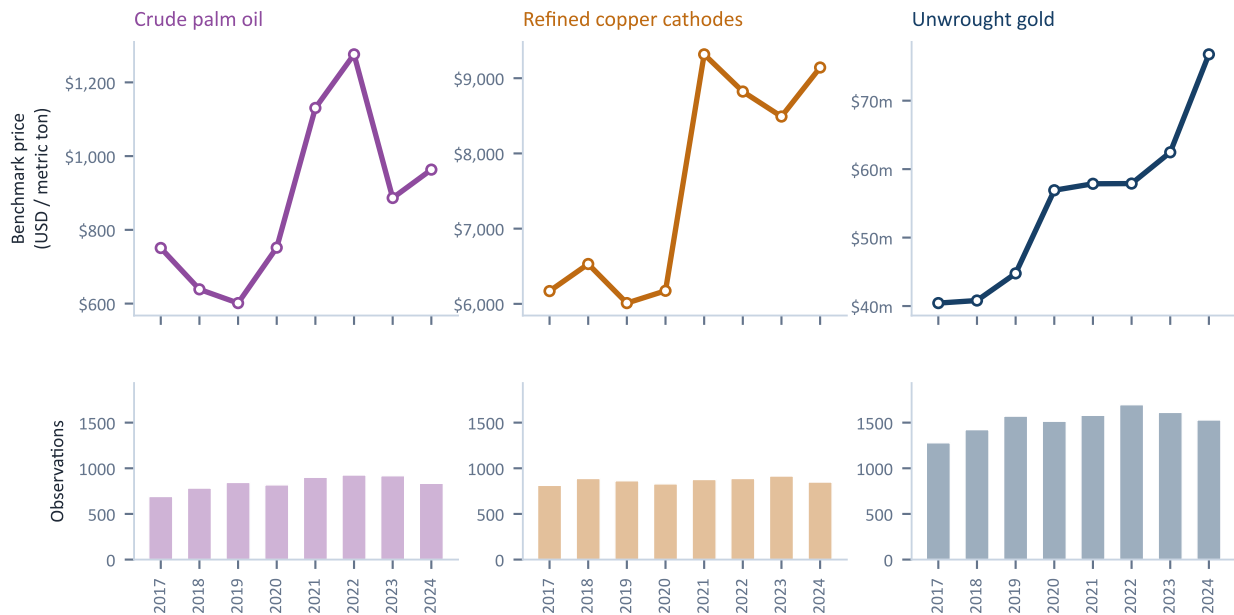
10.1.5 Benchmark trends and observation frequency

Question. How did the world benchmark for each commodity move over the window, and how many observations sit behind each family-year?

Why it matters. The benchmark is the denominator of the residual. If it moves sharply, every corridor's residual moves with it — which is exactly the situation the benchmark-consistency gap exists to handle.

Method. Median benchmark per family-year from the panel, with the count of distinct observations, on natural per-family scales. An index would make the three comparable but would hide the five-orders-of-magnitude difference in economic scale, which is analytically relevant here.

Result.



Source: data/processed/corridor_product_year_panel.parquet. Each family keeps its own scale — annual nominal prices, not inflation-adjusted.

Figure 10.4. Annual World Bank benchmark price and observation count by product family, 2017–2024. Each family keeps its own natural US-dollars-per-metric-ton scale. Bars are observation counts; the line is the benchmark price. Nominal values, not inflation-adjusted.

Table 10.5. Benchmark price and observation count by family and year.

Year	Palm oil benchmark	Copper benchmark	Gold benchmark	Palm oil obs.	Copper obs.	Gold obs.
2017	\$750.81	\$6,169.94	\$40,431,492.85	691	813	1,279
2018	\$638.66	\$6,529.80	\$40,806,692.07	783	888	1,423
2019	\$601.37	\$6,010.15	\$44,769,914.60	845	863	1,571
2020	\$751.77	\$6,173.77	\$56,914,859.11	818	829	1,515
2021	\$1,130.58	\$9,317.05	\$57,859,448.05	901	877	1,580
2022	\$1,275.99	\$8,822.37	\$57,890,634.27	926	888	1,696
2023	\$886.45	\$8,490.29	\$62,458,290.84	918	915	1,612
2024	\$963.36	\$9,142.14	\$76,766,337.58	835	849	1,529

Business interpretation. All three commodities show a marked upswing from 2020–2021: palm oil roughly doubled between 2019 and 2022, copper rose about 50%, gold rose steadily throughout and accelerated in 2024. Any method that treated a price rise as unusual would have flagged the entire market in those years.

How it informs the process. This is the direct justification for the benchmark-adjusted drift feature and the benchmark-consistency credit. The project measures a corridor against its market, not against a fixed number.

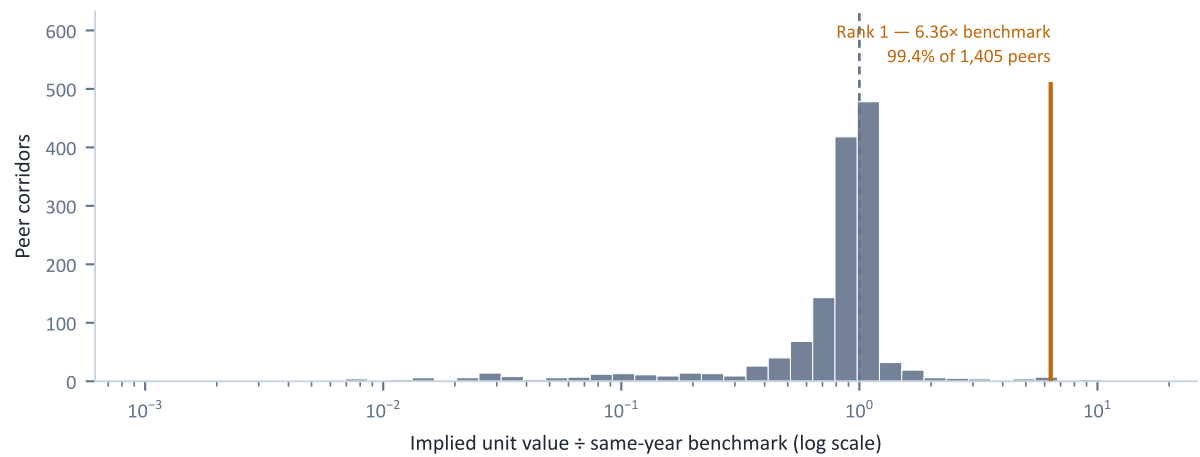
10.1.6 Positioning one case against its full peer population

Question. For a single queued observation, how unusual is it against **all** same-family, same-year corridors — not just against the other queued rows?

Why it matters. Comparing a queue member with other queue members compares one unusual row with other unusual rows and tells you nothing.

Method. Take the ratio of implied unit value to same-year benchmark for every corridor in the same family and year; locate the case within that distribution.

Result. For the rank-1 observation (2022 ESP→NLD, Non-monetary unwrought gold): ratio **6.36×** benchmark, sitting at the **99.36%** percentile of **1,405** same-family, same-year corridors.



Source: data/processed/corridor_product_year_panel.parquet — Unwrought gold, 2022.

Figure 10.5. The rank-1 observation positioned within the full 2022 Non-monetary unwrought gold peer population. The histogram is every same-family, same-year corridor's implied-unit-value-to-benchmark ratio on a log scale; the dashed line at 1.0 is benchmark parity. The marked case sits at the 99.36% percentile.

Business interpretation. The case is not merely above the benchmark — it is above nearly every comparable corridor in the same market year. That is a far stronger statement than a benchmark comparison alone.

Data-quality caveat. Peer comparison is at HS6 aggregate level and cannot see specification differences between corridors.

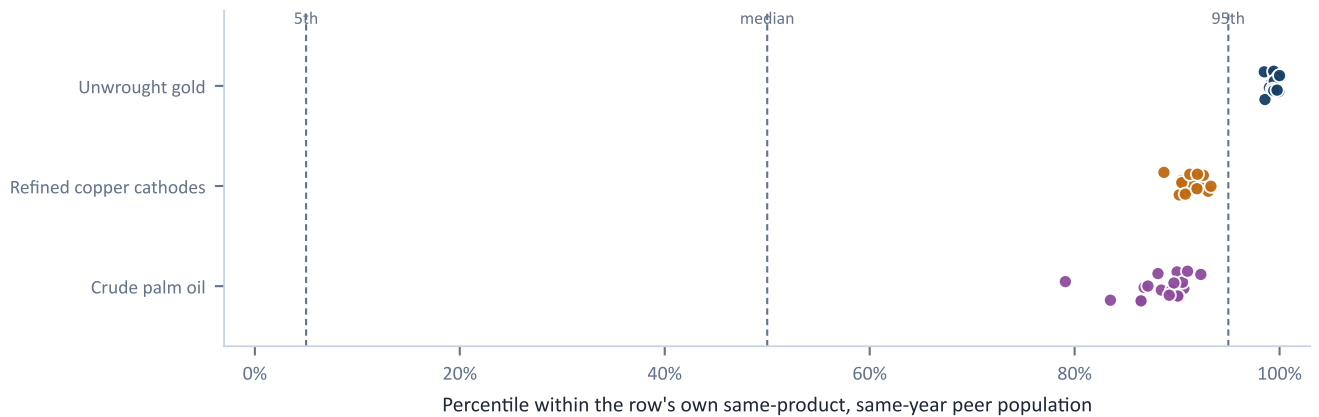
10.1.7 Peer positioning across the whole queue

Question. Is the rank-1 result typical of the queue, or exceptional within it?

Result.

Table 10.6. Peer positioning of the Top 50 within their own same-family, same-year populations.

Measure	Value
Queue rows with a computable peer position	50 of 50
Median peer percentile	91.7%
At or above the 95th percentile of their peers	18
At or below the 5th percentile of their peers	0



Source: data/outputs/top_ranked_corridors.csv against the clean panel. Each queue row is compared with its own peer group, not with other queue rows.

Figure 10.6. Every Top 50 row positioned within its own same-family, same-year peer distribution. Values near 100% are high against peers; values near 0% are low; values near 50% are ordinary on this measure.

Table 10.7. The ten most peer-extreme queue records.

Rank	Year	Corridor	Product	Unit value ÷ benchmark	Peer percentile	Peer corridors
32	2018	USA → JPN	Gold	9.06×	100.00%	1,248
33	2018	LVA → CHE	Gold	8.98×	99.92%	1,248
26	2023	USA → S19	Gold	3.68×	99.85%	1,362
18	2022	KOR → NPL	Gold	8.71×	99.79%	1,405
47	2018	CAN → JPN	Gold	4.77×	99.76%	1,248
17	2022	MYS → NPL	Gold	7.88×	99.64%	1,405
29	2023	DEU → LTU	Gold	2.62×	99.49%	1,362
13	2019	IND → ITA	Gold	3.36×	99.47%	1,329
44	2018	USA → PER	Gold	3.46×	99.44%	1,248
19	2022	JPN → PHL	Gold	6.51×	99.43%	1,405

Business interpretation. The median queue row sits at the **91.7%** percentile of its peers and **18 of 50** are at or above the 95th percentile — so peer extremity is a general property of the queue, not a rank-1 curiosity. Notably, **0 rows** sit in the low tail: in this run the queue is driven by high-side positioning, not by both tails equally.

How it informs the process. Peer position is one of the four comparison views on the dashboard's Selected Case Review page precisely because it is the comparison that survives commodity-level and market-level effects.

10.1.8 What actually distinguishes the queue

Question. Which measurable features separate the Top 50 from the rest of the eligible population?

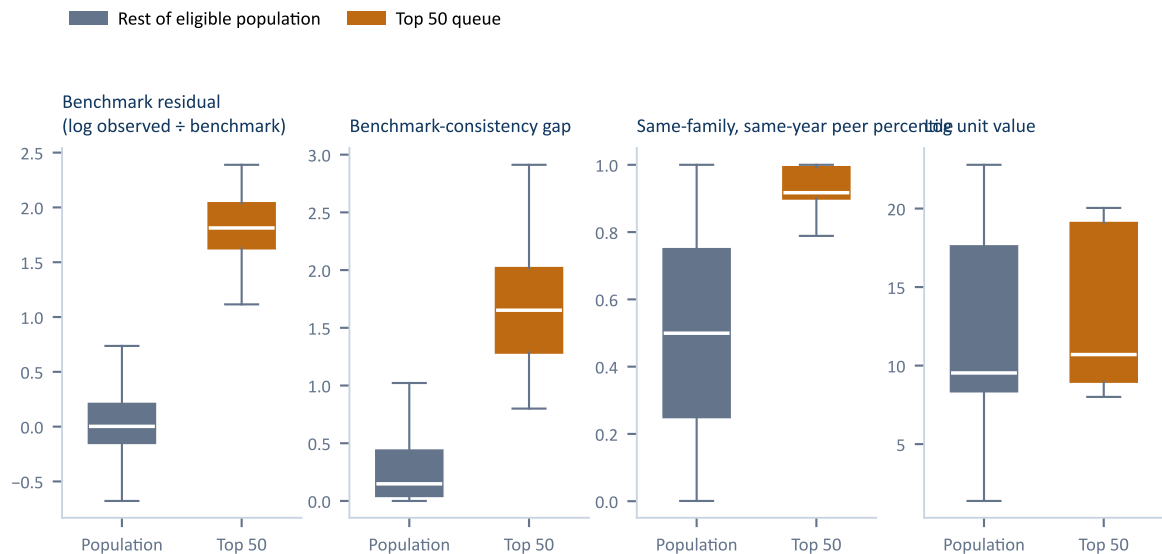
Why it matters. SHAP says which inputs moved the *model*. This asks which observable facts differ — a question an analyst can act on and a reviewer can check.

Method. Compare the distribution of each candidate driver between the queue and the rest of the eligible population; express the queue median as a percentile of the population.

Result.

Table 10.8. Key scoring drivers — Top 50 against the rest of the eligible population.

Driver	Population median	Top 50 median	Top 50 median as a population percentile	Population 95th	Top 50 95th
benchmark_residual	0.002	1.813	94.0%	2.078	2.261
benchmark_consistency_gap	0.148	1.652	93.2%	2.033	2.728
same_family_year_peer_percentile	0.499	0.917	91.8%	0.950	0.998
log_unit_value	9.532	10.703	53.5%	18.010	19.739



Source: reconstructed scores over the 23,656 eligible real observations. Whiskers exclude outliers for readability.

Figure 10.7. Distribution of four key drivers: the Top 50 against the rest of the eligible population. Benchmark residual, benchmark-consistency gap and peer percentile separate cleanly; log unit value does not.

Business interpretation. Three features separate sharply and one does not, and the one that does not is the informative result:

- **Benchmark residual** — population median 0.002, queue median 1.813, which is the **94.0%** percentile of the population. Queue rows sit far above the market.
- **Benchmark-consistency gap** — 0.148 against 1.652 (93.2% percentile). Queue rows moved differently from their market, which is what the residual alone cannot tell you.
- **Peer percentile** — 0.499 (dead centre, as it must be by construction) against 0.917 (91.8% percentile).
- **Log unit value** — 9.532 against 10.703, the **53.5%** percentile. Essentially no separation.

KEY POINT

The log-unit-value result is the most important line in this table. If the queue were merely selecting expensive commodities, log unit value would separate strongly — it is the largest-magnitude feature in the set. It does not. The ranking is driven by **relative** position against market, history and peers, not by absolute price level. That is the difference between a method that ranks gold and a method that ranks unusual behaviour.

How it informs the process. These four comparisons are exactly what the dashboard's *Why It Ranked High* tab shows per case, and what the evidence rows record with thresholds and caveats.

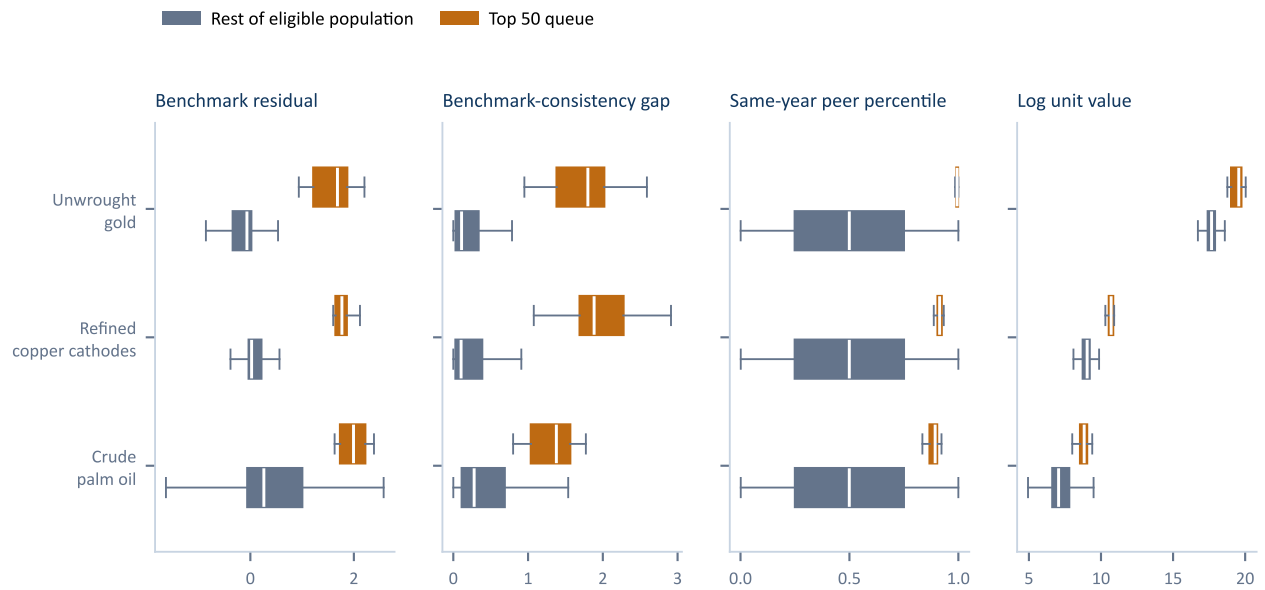
10.1.9 Do the drivers behave the same way in every commodity?

Question. Is the separation in Section 10.1.8 a property of the method, or an artefact of one family dominating the pooled comparison?

Why it matters. Gold is 43.7% of the eligible population. A pooled comparison could be describing gold and little else.

Method. Split the same four drivers by product family and compare the queue against the rest of that family's own population.

Result.



Source: reconstructed scores over the eligible real population. Whiskers exclude outliers for readability.

Figure 10.8. The four key drivers split by product family. In each panel the grey box is the rest of that family's eligible population and the copper box is that family's Top 50 members. Benchmark residual, benchmark-consistency gap and peer percentile separate within every family; log unit value separates within none.

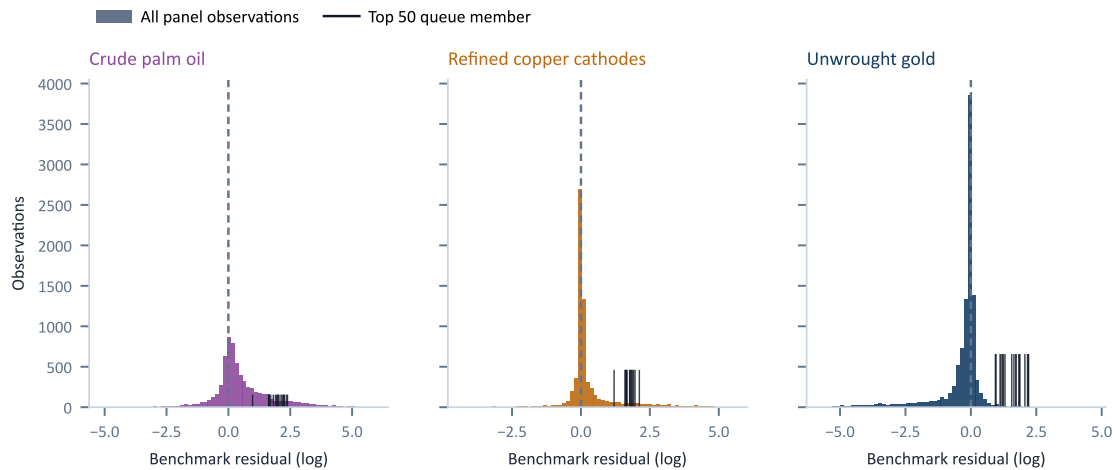
Business interpretation. The pattern repeats within all three families. Benchmark residual, benchmark-consistency gap and peer percentile shift markedly between population and queue in palm oil, copper and gold alike — while log unit value does not shift in any of them. The separation is therefore a property of the method, not of one commodity's price scale.

Data-quality caveat. Family sample sizes differ by a factor of two, so the width of the boxes is not comparable across panels; only the population-to-queue shift within a panel is.

10.1.10 Where does each queue row sit in its family's residual distribution?

Question. The peer percentile in Section 10.1.7 compares a row with its own year. What does the whole family distribution look like, and where do the queue rows fall in it?

Result.



Source: data/processed/corridor_product_year_panel.parquet. Zero is parity with the same-year benchmark; residuals beyond plus or minus 6 log units are omitted from the histogram only.

Figure 10.9. Distribution of the benchmark residual across all panel observations in each family, with every Top 50 member marked by a vertical rule. Zero is parity with the same-year benchmark. Each family's bulk sits close to parity; the queue members sit in the right tail.

Business interpretation. Every family's mass sits near zero — most corridors trade close to the world benchmark most of the time, which is what makes a departure informative. The queue rules cluster in the right tail of each distribution rather than being spread through it.

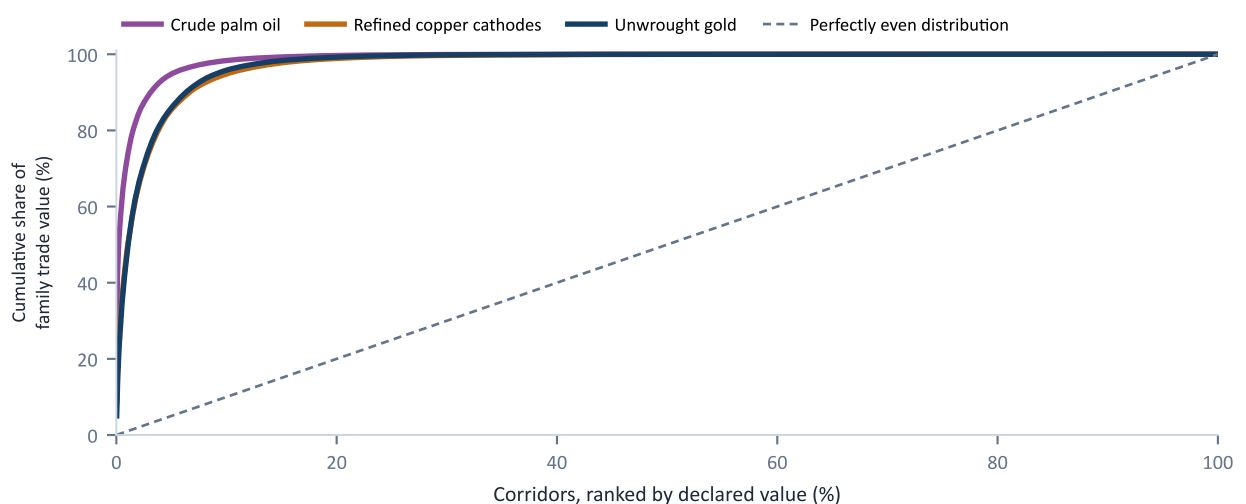
Data-quality caveat. The histogram omits residuals beyond ± 6 log units for readability; those rows remain in every calculation and several are queue members.

10.1.11 How concentrated is trade across corridors?

Question. Is declared trade value spread across many corridors, or carried by a few?

Why it matters. It sets the base rate for what “unusual” can mean. In a highly concentrated market, a handful of corridors carry almost everything and a small corridor's arithmetic is inherently more volatile.

Result.



Source: data/processed/corridor_product_year_panel.parquet, aggregated over 2017-2024.

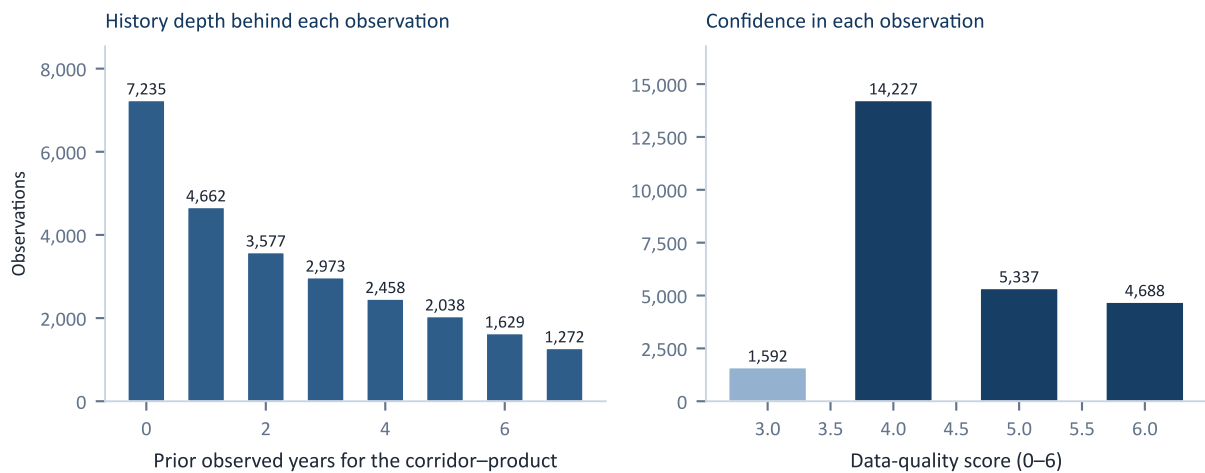
Figure 10.10. Cumulative share of each family's declared trade value against corridors ranked by value. The diagonal is a perfectly even distribution. All three families are heavily concentrated: a small share of corridors carries most of the value.

Business interpretation. All three families are strongly concentrated, gold most of all. That has a direct operational consequence: the long tail of small corridors is where implied unit values are most volatile, because a small denominator amplifies any rounding or revision in the reported quantity. Several queue members sit in that tail — the rank-1 case declares 0.032 metric tons — which is exactly why the evidence rows carry a mandatory caveat rather than a conclusion.

10.1.12 How much support sits behind an observation?

Question. How many prior years does a typical comparison rest on, and how well supported is a typical row?

Result.



Source: data/processed/corridor_features.parquet and the clean panel. Data quality measures how much can be said about a row, never how unusual it is.

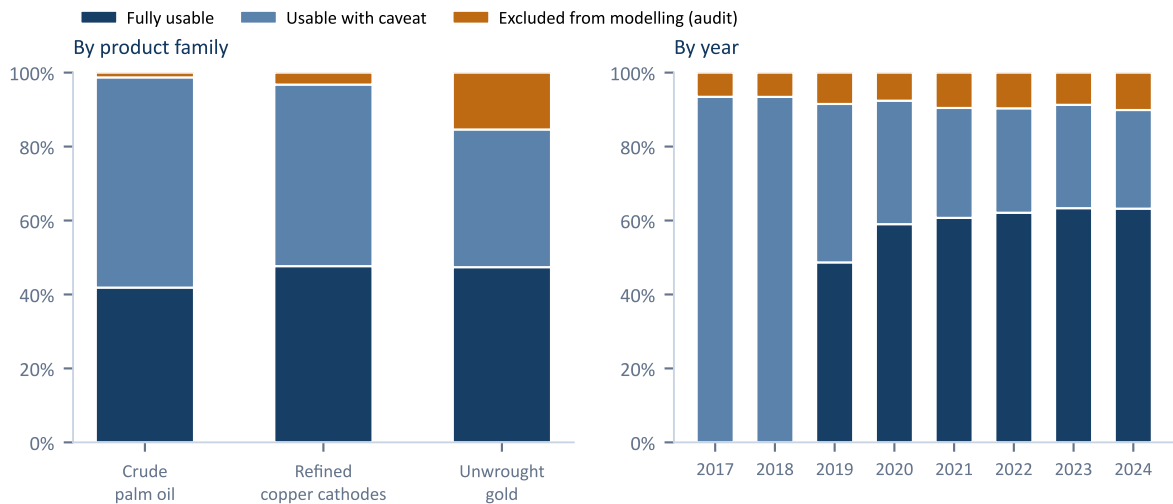
Figure 10.11. Left: how many prior observed years each observation's corridor-product had, which is the depth available to the historical baseline. Right: the additive 0-to-6 data-quality score across the panel. Both describe how much can be said about a row, never how unusual it is.

Business interpretation. History depth falls away steeply: many observations are a corridor-product's first or second appearance, which is why 30.9% of feature rows have no historical baseline at all and why `short_history` is the dominant caveat reason in Chapter 7. The data-quality score concentrates at the upper end (panel mean 4.51 of 6), so most rows are well supported — but the rows that are not are ranked more cautiously by construction, never more aggressively.

10.1.13 Is usability evenly distributed?

Question. Are the quality statuses spread evenly across families and years, or concentrated?

Result.



Source: data/processed/corridor_product_year_panel.parquet. Shares of rows, not of value. No row is excluded entirely.

Figure 10.12. Quality-status composition of the panel. Left: by product family. Right: by year. Shares of rows, not of value. No row is excluded entirely in this run.

Business interpretation. Gold carries by far the largest share of rows excluded from modelling, consistent with Section 10.2. The year view shows the caveat share falling as the window progresses — corridors accumulate history, so `short_history` becomes less common — which is a property of the panel's construction rather than a change in the trade.

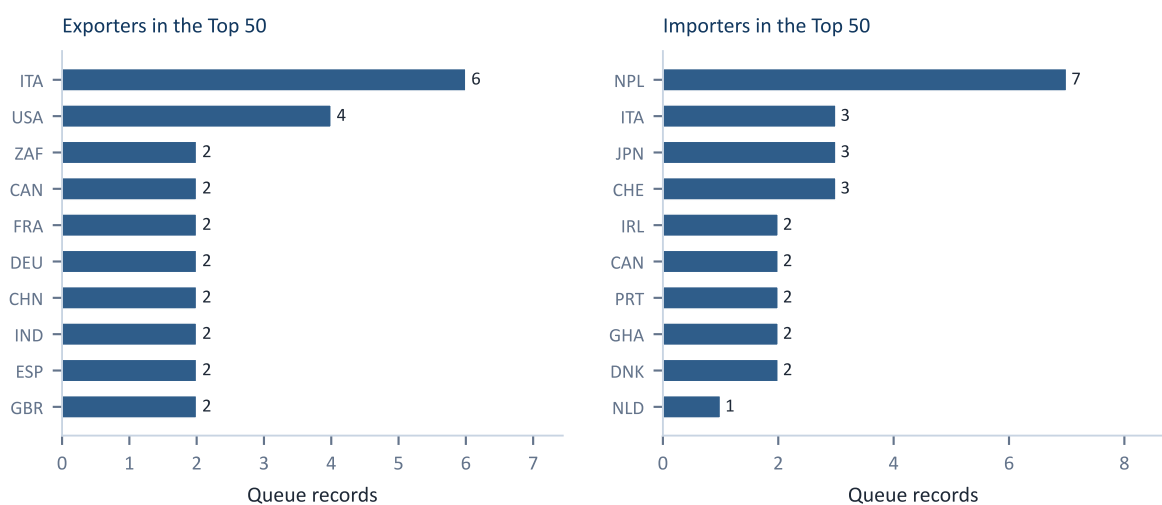
10.1.14 Which countries appear in the current queue?

Question. Which exporters and importers are represented among the Top 50?

CAUTION

This is a description of one run's queue, not a country-risk ranking. A country appears here because a corridor it participates in produced unusual measurable signals in public data — nothing more. Chapter 4's language rules apply in full.

Result.



Source: data/outputs/top_ranked_corridors.csv. Counts describe this run's queue; they are not a country-risk ranking.

Figure 10.13. Exporters and importers appearing most often in the current Top 50 queue. Counts describe this run only.

Business interpretation. The queue spans 34 distinct exporters and 33 distinct importers across 50 records, so it is not concentrated in a single trading relationship. Any recurring country is a prompt to check market and reporting context for that corridor, never an inference about the country.

10.1.15 Reading the evaluation metrics as a stakeholder

The notebook re-presents the model comparison in stakeholder terms, which is worth restating because the framing is the point.

Table 10.9. Precision, recall and the hard-negative guard, all families combined.

Split	Method	Precision@50	Recall@50 (TPR)	Hard-negative FPR@50	Ordinary FPR@50
Validation 2021–2022	Rules baseline	6%	2.8%	1.39%	0.76%
Validation 2021–2022	Logistic regression	4%	1.9%	1.39%	0.78%
Validation 2021–2022	XGBoost	62%	28.7%	0.00%	0.32%
Validation 2021–2022	Isolation Forest	8%	3.7%	0.00%	0.76%
Validation 2021–2022	Hybrid (selected)	46%	21.3%	1.39%	0.43%
Test 2023–2024 (held out)	Rules baseline	14%	6.5%	0.00%	0.73%
Test 2023–2024 (held out)	Logistic regression	6%	2.8%	0.00%	0.80%
Test 2023–2024 (held out)	XGBoost	50%	23.1%	0.00%	0.43%
Test 2023–2024 (held out)	Isolation Forest	6%	2.8%	0.00%	0.80%
Test 2023–2024 (held out)	Hybrid (selected)	48%	22.2%	0.00%	0.44%

Business interpretation. On the test years, 24 of 50 reviewed rows were injected patterns against an expected 0.90 at random — a 26.8× concentration. Recall is 22.2%: a fifty-row queue finds about a fifth of all injected patterns in the split. That is the shape of a capacity-constrained triage queue, and it is the correct trade-off to present to a stakeholder: a shorter queue is more concentrated but finds less; a longer queue finds more but costs more review.

Data-quality caveat. `recall_at_k` is the same quantity as true-positive rate at the chosen cut-off. These are synthetic-scenario results, not detection rates.

10.1.16 Global model contribution

The notebook’s final analytical section reads the global SHAP summary and is explicit that it is optional, governance-facing, and not case evidence. `benchmark_residual` is the strongest average contributor. The full ranking appears in Figure 9.3.

10.2 Gold missing-quantity analysis

10.2.1 The question the anomaly model cannot answer

The core method ranks unit-value patterns that **can be measured**. The gold notebook asks the complementary governance question: *where does official gold trade data contain a declared value but no usable quantity, so an implied unit value cannot be computed at all?*

That is a coverage question, not a risk question, and the notebook’s own conclusion boundary states it: a missing quantity is a data-quality and auditability limitation. It is not evidence of misinvoicing, money laundering, criminal intent, or suspicious activity.

Definition used throughout. A row is a *coverage gap* when declared value is valid but quantity is not:

$$\text{gap}_{c,p,y} = \text{valid}_{c,p,y}^V \cdot (1 - \text{valid}_{c,p,y}^Q) \quad (10.1)$$

where: A row with a valid declared value but no usable quantity. No implied unit value exists, so no unit-value score can be computed. It is a coverage limitation, never a signal. · Implemented in dashboard/services/gold_coverage.py — with_gap_flag(); src/tbml_common.py — build_panel()

10.2.2 Scale of the gap

Table 10.10. Gold quantity-coverage headline.

Measure	Value
Gold rows in the panel	12,205
Rows outside unit-value coverage	1,877 (15.38%)
Declared value in those rows	\$1.183B (0.0415% of gold value)
Gold value that remains assessable	99.9585%
Coverage-gap rows marked model-eligible	0 — must be zero

KEY POINT

The two denominators tell opposite stories, and reporting only one would mislead in either direction. By **row count** the gap is large — nearly one gold record in six. By **declared value** it is 0.0415%, so 99.9585% of gold value still reaches the queue. Neither figure alone is the answer.

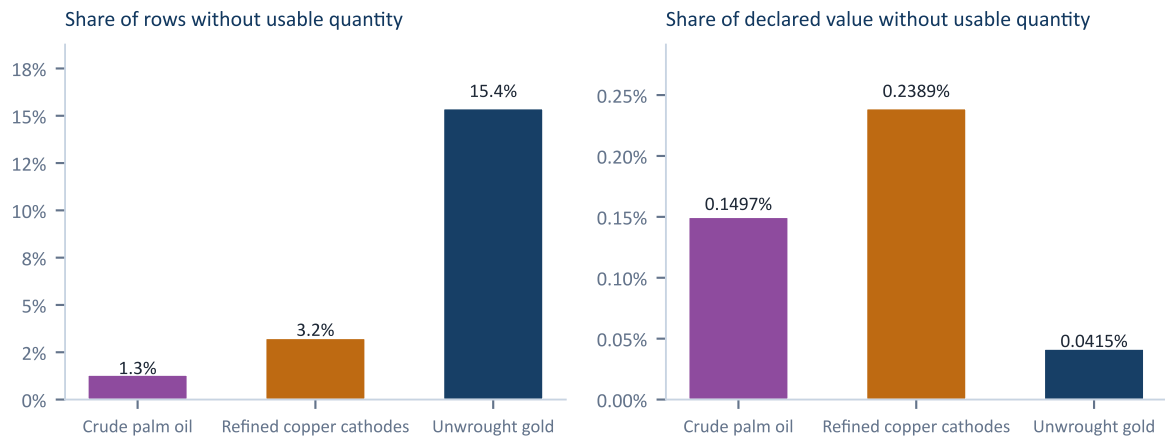
10.2.3 Comparison with palm oil and copper

Question. Is gold unusual on this measure, or is it just the largest family?

Result.

Table 10.11. Coverage gap by product family — row rate against value share.

Product family	Rows	Rows without usable quantity	Row rate	Declared value in those rows	Share of family value
Non-monetary unwrought gold	12,205	1,877	15.38%	\$1.183B	0.0415%
Refined copper cathodes	6,922	224	3.24%	\$1.369B	0.2389%
Crude palm oil	6,717	87	1.30%	\$0.142B	0.1497%



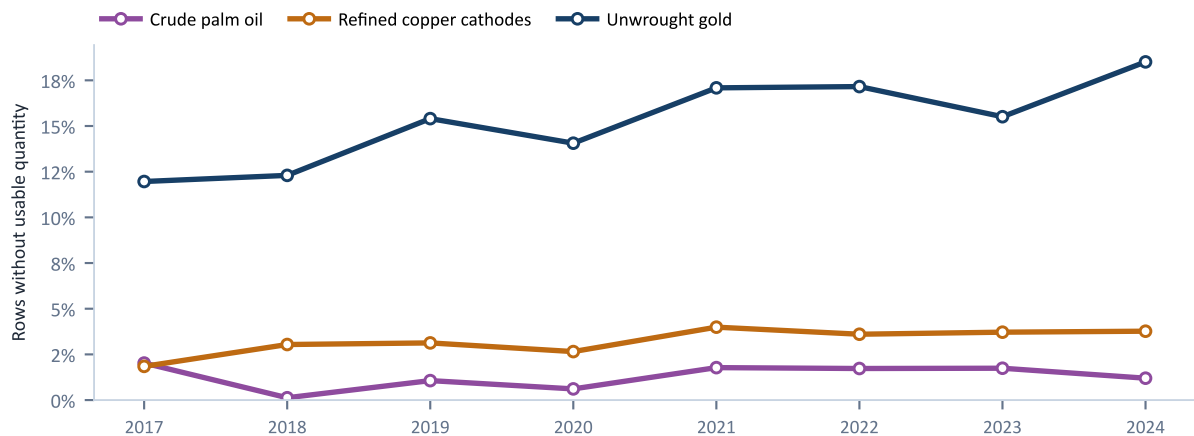
Source: data/processed/corridor_product_year_panel.parquet. The two panels answer different control questions and use different denominators.

Figure 10.14. Quantity-coverage gap by product family on two denominators. Left: share of rows without usable quantity. Right: share of declared value without usable quantity. Gold leads on rows; copper leads on value.

Business interpretation. Gold has by far the largest **row** gap (15.38%) but the *smallest value* gap (0.0415%). Copper has the largest value gap (0.2389% of copper value, \$1.369B) from a modest row gap of 3.24%. Palm oil is smallest on rows (1.30%) and intermediate on value (0.1497%).

Gold is the useful case because its missingness is **broad and persistent**, not because it carries the largest dollar gap.

The same comparison over time confirms that the gap is a standing property of gold reporting rather than a recent development:



Source: data/processed/corridor_product_year_panel.parquet. A rate, not a count — the number of observed corridors changes each year.

Figure 10.15. Share of rows without a usable quantity, by year and product family. Gold sits several times above the other two families in every year of the window. A rate, not a count — the number of observed corridors changes each year.

Gold's row rate exceeds palm oil's and copper's in every one of the 8 years, and the ordering never changes. That stability is what makes this a coverage characteristic to trace with the data owner rather than an event to investigate.

10.2.4 Impact on unit-value calculation

Without a usable quantity there is no implied unit value, and without an implied unit value the following features are all undefined for that row: `log_unit_value`, `benchmark_residual`, `robust_historical_z`, `shifted_corridor_history_median`, `shifted_corridor_history_med`, `unit_value_yoy_change`, `benchmark_adjusted_drift`, `same_family_year_peer_percentile` and `benchmark_consistency_gap`.

That is most of the analytical apparatus. The row cannot be scored — not because a model rejected it, but because the arithmetic has no denominator.

10.2.5 Impact on model eligibility — verified, not assumed

The notebook checks the boundary directly rather than assuming it holds.

Table 10.12. Model-boundary checks (regenerated against current data).

Check	Result	Meaning
Gold rows with missing or unusable quantity	1,877	Retained for audit, outside unit-value modelling
Those rows marked <code>model_eligible</code>	0	Must be zero — it is
Eligible feature rows with <code>missing_quantity_flag = 1</code>	0	The flag is constant zero inside the scored population
Missing quantities in the ranked review queue	0	The queue contains model-eligible observations only

KEY POINT

The third check is the subtle one. `missing_quantity_flag` exists in the feature schema, but eligibility already requires a valid quantity, so within the training and scoring population the flag is **structurally always zero**. It documents a limitation. It cannot teach any model to rank missing-quantity rows, in either direction.

10.2.6 Is trade value still analytically useful?

Yes — for a different purpose. The declared value in a coverage-gap row is a real, verified, hash-traceable figure. It supports materiality assessment (how much declared value sits outside coverage), coverage monitoring over time, prioritisation of source-data follow-up, and audit completeness.

What it cannot support is any unit-value comparison, because there is nothing to divide by.

10.2.7 Why missing quantity is a data-quality condition, not a signal

Three arguments, each independently sufficient:

1. **It is structural, not behavioural.** Missingness concentrates in specific long-running corridors that are missing in every year — a stable reporting pattern, not an event.
2. **The direction of any inference is undefined.** A missing quantity is equally consistent with a reporting convention, a threshold rule, a unit that does not apply, and nothing at all. There is no basis for treating it as elevated or reduced risk.
3. **Attribution is impossible.** BACI is a harmonised country-pair dataset. The exporter and importer fields identify the trade direction, not which party or reporting system produced the gap.

CAUTION

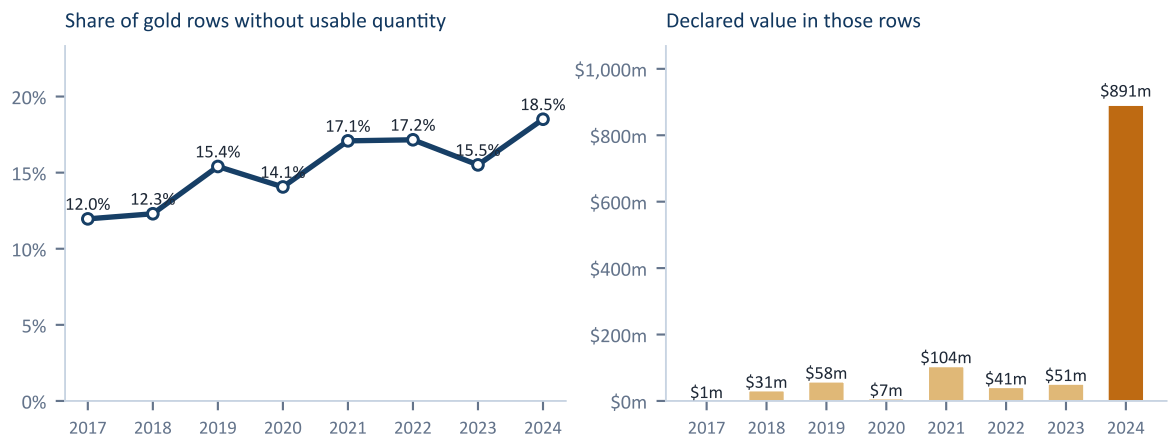
Converting missingness into an anomaly signal would be the single most damaging change that could be made to this project. It would generate confident-looking alerts driven entirely by data-collection artefacts and would be almost impossible to disprove case by case.

10.2.8 Trend over time

Method. Annual **rate**, not count, because the number of observed corridors changes each year — and value separately, because the two behave differently.

Table 10.13. Gold quantity-coverage gap by year.

Year	Gold rows	Rows without usable quantity	Row rate	Declared value in those rows
2017	1,279	153	12.0%	\$0.8m
2018	1,423	175	12.3%	\$31.0m
2019	1,571	242	15.4%	\$57.7m
2020	1,515	213	14.1%	\$7.0m
2021	1,580	270	17.1%	\$103.8m
2022	1,696	291	17.2%	\$41.0m
2023	1,612	250	15.5%	\$50.9m
2024	1,529	283	18.5%	\$890.7m



Source: data/processed/corridor_product_year_panel.parquet. Separate panels, not a dual axis — the rate and the value behave differently.

Figure 10.16. Gold quantity-coverage gap by year. Left: share of gold rows without usable quantity. Right: declared value in those rows. The row rate rises unevenly; the value series is dominated by a single 2024 record.

Business interpretation. The row rate moves from 12.0% in 2017 to 18.5% in 2024, but not monotonically. The value series behaves completely differently: 2024 alone contributes **75.3%** of all gold gap value across the window. That spike demands a concentration check before it is presented as a trend.

10.2.9 Concentration behind the dollar headline

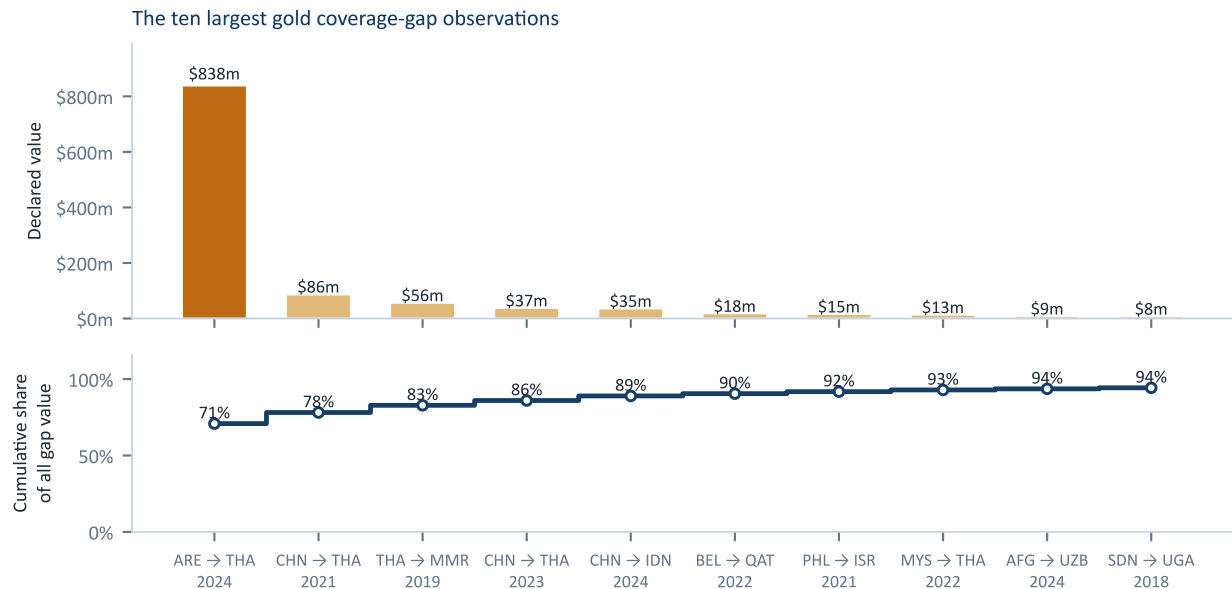
Question. Is the \$1.183B figure broad-based or driven by a handful of records?

Result.

Table 10.14. The ten largest gold coverage-gap observations by declared value, with cumulative share.

#	Year	Route	Declared value	Cumulative share of gap value	Observation ID
1	2024	ARE → THA	\$838,370,797	70.9%	obs_db9b1cc6d05acbab0200
2	2021	CHN → THA	\$85,891,187	78.1%	obs_5324bbef2e3b03c4d203
3	2019	THA → MMR	\$55,785,948	82.9%	obs_530b195992885bfb88eb
4	2023	CHN → THA	\$37,168,921	86.0%	obs_ed263e36c0fe11d99a3c

#	Year	Route	Declared value	Cumulative share of gap value	Observation ID
5	2024	CHN → IDN	\$35,339,527	89.0%	obs_4db3718d56d71d840d56
6	2022	BEL → QAT	\$17,735,723	90.5%	obs_bce7220f5f95aaa01c5d
7	2021	PHL → ISR	\$15,498,054	91.8%	obs_dfa0c460d7f55bb06787
8	2022	MYS → THA	\$13,264,169	92.9%	obs_0ba72dd07c4fca54742c
9	2024	AFG → UZB	\$8,535,959	93.6%	obs_860f687750bb38e0ab4d
10	2018	SDN → UGA	\$7,963,152	94.3%	obs_9dd2462b513e711f1e71



Source: data/processed/corridor_product_year_panel.parquet. Two stacked panels on a shared category axis, not a dual axis.

Figure 10.17. Pareto view of the gold coverage-gap value. Bars are the ten largest individual gap observations; the line is the cumulative share of total gap value. The top ten account for 94.3%.

Business interpretation. One **2024 United Arab Emirates → Thailand** observation contributes **70.9%** of the entire gold gap value (\$838,370,797), and the top ten account for **94.3%**. The headline is a single-record story.

CAUTION

This concentration does **not** make that record suspicious. It makes source validation important. Before \$1.183B is quoted as a coverage exposure, that one record should be checked against the source extract and, where possible, against mirror or shipment data. A number driven by one row should never be presented as a systemic finding.

10.2.10 Persistence versus materiality — two different control questions

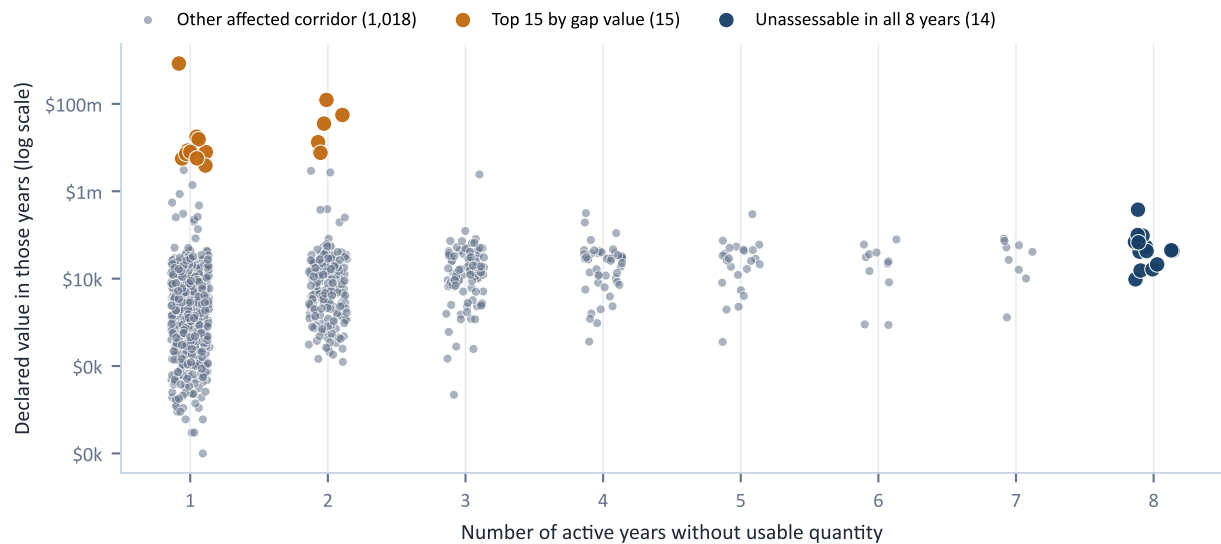
Question. Are the same corridors affected repeatedly, and are those the same corridors that carry the value?

Method. Corridor-level aggregation over the full panel history. A corridor is *persistent* when it is active in all 8 years **and** lacks usable quantity in all 8.

Table 10.15. Persistence summary for gold corridors.

Measure	Value
All gold corridors	3,068
Corridors with at least one coverage-gap year	1,047
Corridors with at least four coverage-gap years	104

Measure	Value
Corridors active in all 8 years	702
Full-history corridors missing quantity in all 8 years	14 (1.99% of full-history corridors)
Overlap between the top 15 by gap value and the persistent set	0
Total declared value across the persistent corridors	\$1,000,119 (0.0845% of gap value)



Source: data/processed/corridor_product_year_panel.parquet. Persistence and materiality identify disjoint sets of corridors.

Figure 10.18. Persistence against materiality for affected gold corridors. Horizontal axis: number of active years without usable quantity. Vertical axis: declared value in those years, log scale. The persistent corridors and the high-value corridors are disjoint sets.

Business interpretation. The overlap is **0**. The corridors that repeatedly cannot be assessed and the corridors that carry the missing value are **entirely different sets**. That single number is the operational finding of the whole analysis: it means one control cannot serve both, and two separate follow-up lists are required.

- **Materiality follow-up** — individual high-value records, checked against source. Episodic, large, one-off.
- **Persistence follow-up** — repeatedly unassessable corridors, traced to reporting conventions. Systematic, small, recurring.

Table 10.16. Persistence follow-up — the ten corridors with the most coverage-gap years.

Route	Corridor ID	Active years	Years without usable quantity	Declared value in those years
Australia → Brazil	AUS->BRA	8	8	\$378,890
Netherlands → Ireland	NLD->IRL	8	8	\$99,651
Netherlands → Hungary	NLD->HUN	8	8	\$96,644
Netherlands → Greece	NLD->GRC	8	8	\$69,237
Slovenia → North Macedonia	SVN->MKD	8	8	\$67,717
Netherlands → Cyprus	NLD->CYP	8	8	\$53,422
Netherlands → Estonia	NLD->EST	8	8	\$44,594
Ireland → Netherlands	IRL->NLD	8	8	\$43,257
Netherlands → Slovakia	NLD->SVK	8	8	\$42,816
Greece → Netherlands	GRC->NLD	8	8	\$41,367

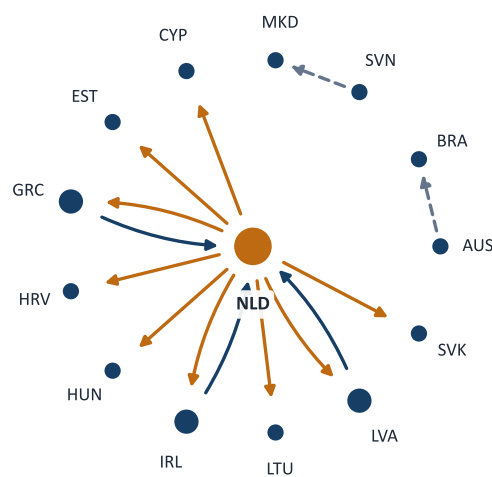
10.2.11 Structure of the persistent gap

Result. The 14 persistent corridors form a strikingly narrow structure.

Table 10.17. The 14 persistent gold coverage-gap corridors.

Route	Corridor	Direction group	Part of a two-way pair	Years without usable quantity	Declared value across 8 years
Australia → Brazil	AUS→BRA	Not Netherlands-linked	No	8	\$378,890
Greece → Netherlands	GRC→NLD	To Netherlands	Yes	8	\$41,367
Ireland → Netherlands	IRL→NLD	To Netherlands	Yes	8	\$43,257
Latvia → Netherlands	LVA→NLD	To Netherlands	Yes	8	\$9,643
Netherlands → Cyprus	NLD→CYP	From Netherlands	No	8	\$53,422
Netherlands → Estonia	NLD→EST	From Netherlands	No	8	\$44,594
Netherlands → Greece	NLD→GRC	From Netherlands	Yes	8	\$69,237
Netherlands → Croatia	NLD→HRV	From Netherlands	No	8	\$15,349
Netherlands → Hungary	NLD→HUN	From Netherlands	No	8	\$96,644
Netherlands → Ireland	NLD→IRL	From Netherlands	Yes	8	\$99,651
Netherlands → Lithuania	NLD→LTU	From Netherlands	No	8	\$16,364
Netherlands → Latvia	NLD→LVA	From Netherlands	Yes	8	\$21,168
Netherlands → Slovakia	NLD→SVK	From Netherlands	No	8	\$42,816
Slovenia → North Macedonia	SVN→MKD	Not Netherlands-linked	No	8	\$67,717

→ From the Netherlands → Not Netherlands-linked ● 2 connected routes
→ To the Netherlands ● 12 connected routes ● 1 connected route



Routes touching each country

Code	Country	Routes
NLD	Netherlands	12
GRC	Greece	2
IRL	Ireland	2
LVA	Latvia	2
AUS	Australia	1
BRA	Brazil	1
CYP	Cyprus	1
EST	Estonia	1
HRV	Croatia	1
HUN	Hungary	1
LTU	Lithuania	1
MKD	North Macedonia	1
SVK	Slovakia	1
SVN	Slovenia	1

Source: data/processed/corridor_product_year_panel.parquet. Layout only — position, colour and size carry no geographic, risk or suspiciousness meaning.

Figure 10.19. The 14 persistent gold coverage-gap corridors as a directed network. Node size is the number of connected persistent routes; arrows run exporter to importer. Nodes carry ISO3 codes, and the panel beside the graph resolves every code and gives its route count — the same reference the dashboard's coverage page provides. Position is layout only and carries no geographic, risk, or suspiciousness meaning.

12 of the 14 routes (85.7%) connect to the Netherlands — 9 outbound and 3 inbound. **Greece, Ireland, Latvia** appear in *both* directions with the Netherlands across all 8 years. The remaining routes (Australia → Brazil; Slovenia → North Macedonia) are separate components.

CAUTION

This is a **directional pattern in a harmonised dataset**, not an attribution. BACI's exporter and importer fields identify the trade direction; they do not identify which party or reporting system produced the missing quantity. The correct phrasing is "Netherlands-linked", never "Netherlands reporter problem". Node position, colour and size are readability choices and carry no geographic, risk or suspiciousness meaning. Country prominence in a network like this can also simply reflect general trade coverage.

Business interpretation. A reciprocal, eight-year-stable, bilateral structure is much more consistent with a stable product or reporting convention than with anything behavioural. It is exactly the kind of finding a data owner can trace: check source coverage, reporting conventions and processing rules for these specific routes.

Materiality. The 14 persistent corridors together represent **\$1,000,119** — 0.0845% of gold gap value. Persistent and material are, once again, different things.

10.2.12 How unscored records are retained

Unscored records are retained in full and remain traceable throughout:

- They are present in the panel (`quality_status = excluded_from_modeling_retained_for_audit`, 2,188 rows).
- They appear in `exclusion_audit.csv` with their reason.
- They keep `obs_id`, `source_row_id`, `source_version` and `source_file_hash`, so any record can be traced to its raw source row.
- They carry their declared value, year, corridor and family, so materiality and coverage can be measured.
- The dashboard presents them on a dedicated page, deliberately separated from the anomaly queue.

10.2.13 Recommended handling in a bank implementation

Structural. Maintain **two queues**, visually and operationally distinct: a unit-value review queue (the existing ranked observations) and a **quantity data-quality queue** (excluded observations, prioritised by declared value, repeated gap years, and source traceability). Never merge missingness into the anomaly score.

Within the data-quality queue. Keep the two lenses separate:

- *Materiality* — which individual records carry the most declared value? Validate against source extracts and mirror data.
- *Persistence* — which corridors repeatedly cannot be assessed? Trace to reporting conventions and processing rules.

Presentation guardrails, taken from the notebook's own recommendations: do not label countries, corridors or missing rows as risky; do not use risk colouring for this module; state that network position is layout only; show numerator and denominator in every rate; preserve the human-review boundary on every drill-down and export.

Production control design. A real deployment would additionally need ownership, refresh frequency, a source-repair workflow, materiality thresholds and closure reasons for the data-quality queue.

10.2.14 Reproducibility of this analysis

The gold notebook ends with 19 assertions — 11 snapshot checks binding the reported values to the source file hash, and 8 analytical invariants. The regenerated values in this chapter were compared against the notebook's recorded expectations and match, including the source file hash `0e493e057c8fd579...`, 25,844 panel rows, 12,205 gold rows, 1,877 coverage-gap rows, 1,047 affected corridors, 702 full-history corridors, 14 persistent corridors and 12 Netherlands-linked.

The invariants that must hold, and do: `obs_id` is unique; every gold coverage-gap row is model-ineligible; every eligible feature row has `missing_quantity_flag = 0`; annual gold rows and gap value reconcile to their totals; no corridor's gap years exceed its active years; and every row in the ranked queue has a usable quantity.

10.2.15 What this extension adds

It converts an exclusion rule into a **measurable control**. It exposes concentration before a dollar total is over-interpreted. It separates two follow-up actions that a single “risk” number would have merged. And it does all of that without weakening the conclusion boundary by a single word.

10.3 Chapter findings at a glance

Every row below is a measurement made in this chapter against the artefacts of the recorded run, with the section that derives it. None of them is a finding about conduct.

Table 10.18. What the profiling established, and what each result supports.

Question asked	What the recorded run shows	What the result supports
Is the queue in the tail of the population, or at the top of a flat one?	Queue median benchmark residual sits at the 94.0% percentile of all 23,656 scored observations	The ranking is genuinely selective, so review capacity is being spent on the extreme
Is any product family over-represented?	18 gold (36%) · 18 palm oil (36%) · 14 copper (28%), against each family’s share of the eligible population	Composition is a property of this run; it is not a statement about commodity risk
Where does the queue concentrate in time?	16 of 50 rows fall in 2022	A year cluster to check against market context before anything is read into it
Do queue rows stand out against their own peers?	18 of 50 rows at or above the 95th percentile of their same-family, same-year peers; median 91.7%	Peer position is measured per case rather than assumed from the rank
Which inputs actually separate the queue?	Benchmark residual, benchmark-consistency gap and peer percentile separate cleanly; log unit value does not	The ranking is driven by comparisons, not by absolute price level
Do those drivers behave the same way in every commodity?	The same three separate within each of the three families	The method is not being carried by one commodity’s arithmetic
How concentrated is trade itself?	A small share of corridors carries most declared value in all three families	Concentration is a market property and sets the base rate for “unusual”
How much support sits behind a typical row?	30.9% of feature rows have no historical baseline; panel mean data-quality score 4.51 of 6	Thinly supported rows are ranked more cautiously by construction, never more aggressively
Is usability evenly spread?	11,894 fully usable, 11,762 usable with caveat, 2,188 retained for audit only, 0 excluded entirely	Data quality is reported beside review priority, never merged into it
How large is the gold quantity gap?	1,877 of 12,205 rows (15.38%) but only 0.0415% of declared gold value	Row coverage and value coverage are two different control problems
Is that gap isolated or recurring?	One 2024 ARE → THA record holds 70.9% of gap value; 14 corridors lack usable quantity in all 8 years, worth \$1,000,119	Materiality and persistence need separate follow-up, and separate owners
Does any of this conclude wrongdoing?	No. Every result above is a ranking position or a coverage measurement	The conclusion boundary is unchanged by everything in this chapter

KEY POINT

Two questions run through the whole chapter and are answered separately every time: *is this row unusual on measurable public signals?* and *how much can be said about this row at all?* Merging them would produce a single number that is easier to present and impossible to act on.

CHAPTER 11

Dashboard and Analyst Workflow

The dashboard is an implemented, deployed product, not a design mock-up. It is a multi-page Streamlit application of 11 page modules, 11 service modules and 18 shared components (13,183 lines), covered by 12 test modules that assert its data contracts and page behaviour. All 160 tests pass.

Open the deployed application <https://tbml-review-triage.streamlit.app/>
 Source code <https://github.com/jessybaolin/evidence-based-tbml-corridor-drift>

11.1 What the dashboard is, and what it is not

Table 11.1. Dashboard scope.

It is	It is not
A read-only presentation layer over generated pipeline artefacts	A model-training or scoring environment
A browser application rendered entirely from Python by Streamlit	A Node.js / React / separate-backend stack
A stakeholder-facing analytical prototype	A production AFC monitoring platform
Governed by a boundary ribbon on every page	A case-management or disposition system
Free of any AI or language-model layer	A generative-AI product

Changing a filter re-slices a cached dataframe. It never re-runs the pipeline. The application writes nothing into the repository.

11.2 Architecture, routing and the boundary ribbon

10 content pages are registered through `st.navigation` in four groups, plus a full-screen welcome route that is the default. Figure 11.1 shows the application as it runs: the hand-built navigation on the left carries the four groups and the creator block; the boundary ribbon is pinned across the foot of every page.

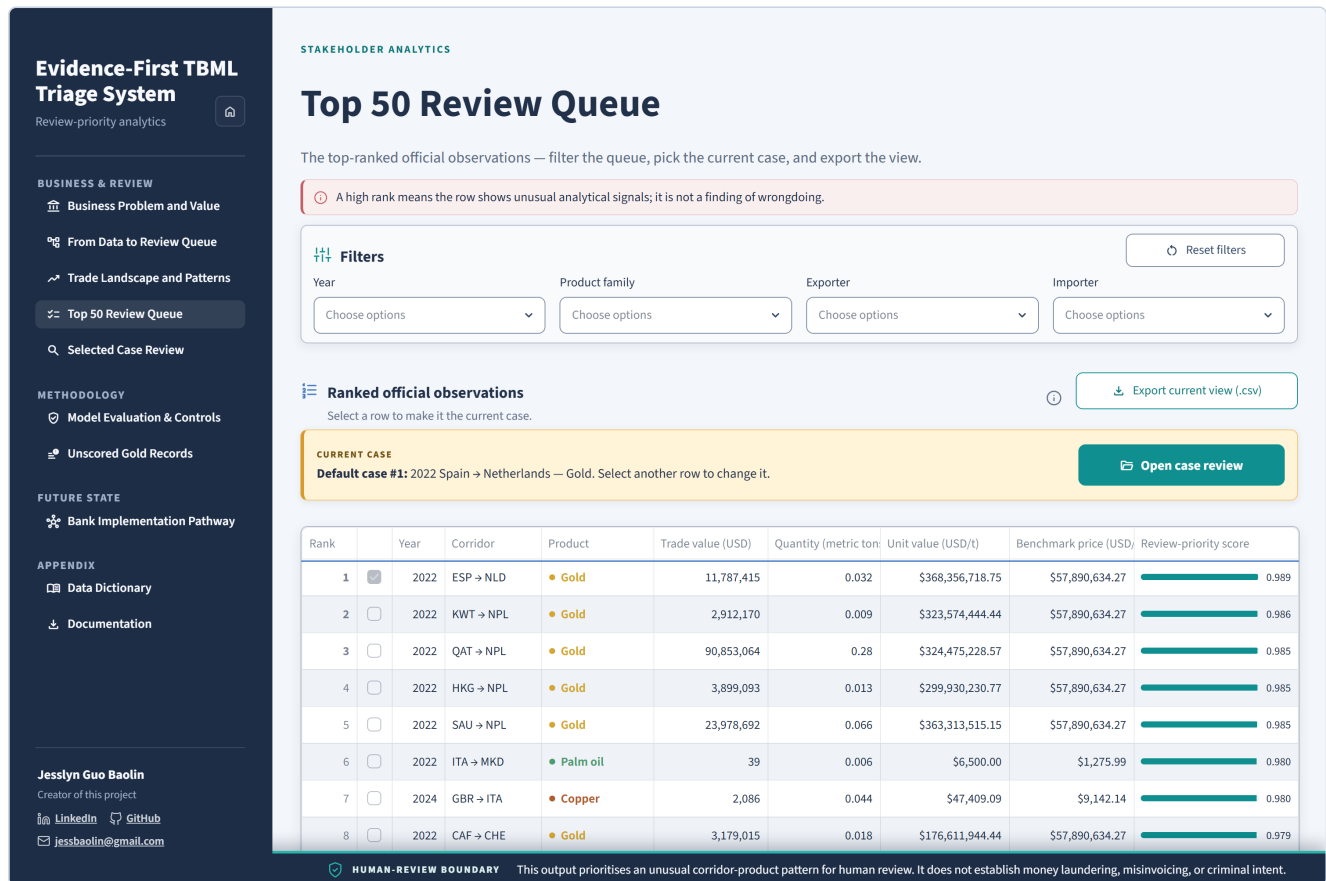


Figure 11.1. The deployed application, Top 50 Review Queue page. Left: the hand-built sidebar navigation — four groups (Business & Review, Methodology, Future State, Appendix) plus the home control and the creator block pinned at the foot. Right: the interpretation caveat, the four queue filters, the current-case banner, the export control, and the ranked table. The human-review boundary ribbon runs across the foot of every page.

Table 11.2. Registered pages and navigation groups.

Group	Page	Module
(root)	Welcome	app_pages/landing.py
Business & Review	Business Problem and Value	app_pages/executive_overview.py
Business & Review	From Data to Review Queue	app_pages/from_data_to_review_queue.py
Business & Review	Trade Landscape and Patterns	app_pages/portfolio_analytics.py
Business & Review	Top 50 Review Queue	app_pages/review_queue.py
Business & Review	Selected Case Review	app_pages/case_investigation.py
Methodology	Model Evaluation & Controls	app_pages/model_and_controls.py
Methodology	Unscored Gold Records	app_pages/gold_quantity_coverage.py
Future State	Bank Implementation Pathway	app_pages/bank_implementation_pathway.py
Appendix	Data Dictionary	app_pages/appendix.py
Appendix	Documentation	app_pages/documentation.py

Streamlit's built-in sidebar menu is hidden (`position="hidden"`) and replaced with a hand-built navigation, so the sidebar is one styled container from the brand block at the top to the creator block pinned at the foot.

The **human-review boundary** is rendered once in the entry point as a fixed footer ribbon that rides on every dashboard page, loaded verbatim from `configs/project.yml`. It cannot drift from the pipeline's approved wording because it is the same string.

CAUTION

`dashboard/README.md` describes "six navigable pages" and lists eight. The code registers 10 content pages plus the welcome route. This document follows the code; the README is out of date on this point.

11.3 Failure behaviour — never substitute data

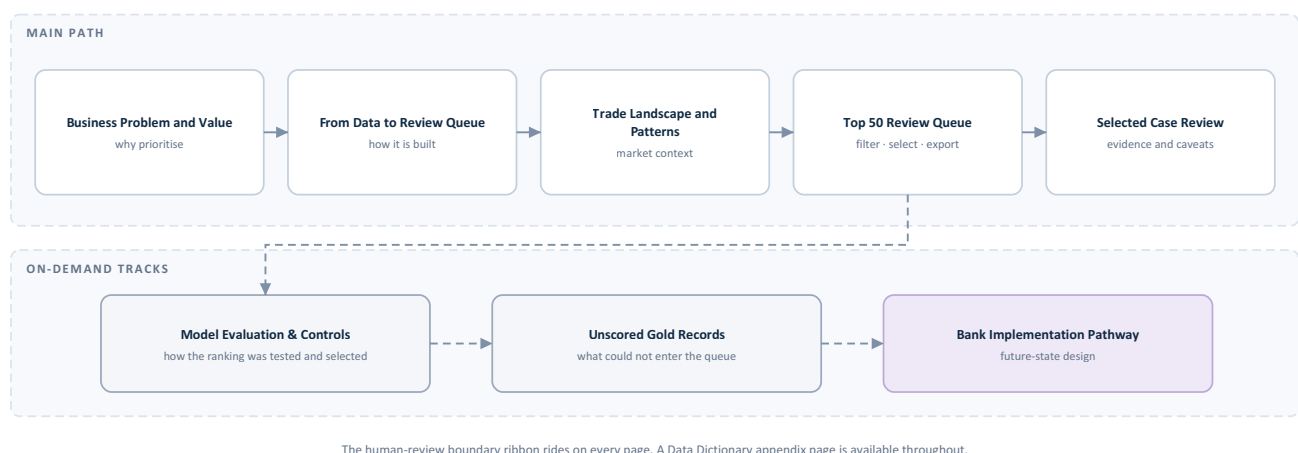
Every artefact the dashboard reads is registered once in `path_resolver.DATA_FILES` with a purpose and a required/optional flag.

- A missing **required** file raises `MissingOutputError`, which the entry point renders as a full-page error naming the exact file and the exact pipeline command to run.
- A missing **optional** file returns `None`, and the affected panel shows a labelled empty state: *"This panel is unavailable because the required project output has not been generated. Run the relevant analytical pipeline stage and reload the dashboard."*

KEY POINT

The dashboard never substitutes demo data, sample values, or placeholder numbers. If a number cannot be derived from a real artefact, the panel says so.

11.4 The analyst journey



The human-review boundary ribbon rides on every page. A Data Dictionary appendix page is available throughout.

Figure 11.2. The end-to-end analyst journey through the deployed application. The main path runs from the business problem to a selected case; model validation and coverage limits are on-demand tracks rather than mandatory steps.

- 1. Welcome.** A full-screen landing page states what the system is and hands over to the dashboard.
- 2. Business Problem and Value.** The prioritisation problem, why these three commodities, what the system delivers, and the boundary. All headline figures are computed live from the artefacts.

3. From Data to Review Queue. A two-scene walk-through: sources and scope, then data preparation. It shows one real sample observation moving through verification, unit standardisation, row construction, usability controls and time-safe signal creation, with a technical companion drawer holding the pipeline diagrams, the raw BACI field names and the SHA-256 provenance record.

4. Trade Landscape and Patterns. Market scale by family and year, benchmark movement, and the profile of the current queue by year and family.

5. Top 50 Review Queue. Filter, select, export. Section 11.5.

6. Selected Case Review. One case in depth. Section 11.6.

7. Model Evaluation & Controls and **8. Unsourced Gold Records.** On-demand tracks: how the ranking was tested and selected, and what could not enter the queue at all. Sections 11.7 and 11.8.

9. Bank Implementation Pathway, 10. Data Dictionary and **11. Documentation.** Future-state design, field reference, and the download point for the two written deliverables.

11.5 Product: Evidence-first Top 50 ranked queue

This is the working surface of the product. Figure 11.3 shows the ranked grid at full column width, as the deployed application renders it.

Rank		Year	Corridor	Product	Trade value (USD)	Quantity (metric tons)	Unit value (USD/t)	Benchmark price (USD/t)	Review-priority score
1	☒	2022	ESP → NLD	● Gold	11,787,415	0.032	\$368,356,718.75	\$57,890,634.27	0.989
2	☐	2022	KWT → NPL	● Gold	2,912,170	0.009	\$323,574,444.44	\$57,890,634.27	0.986
3	☐	2022	QAT → NPL	● Gold	90,853,064	0.28	\$324,475,228.57	\$57,890,634.27	0.985
4	☐	2022	HKG → NPL	● Gold	3,899,093	0.013	\$299,930,230.77	\$57,890,634.27	0.985
5	☐	2022	SAU → NPL	● Gold	23,978,692	0.066	\$363,313,515.15	\$57,890,634.27	0.985
6	☐	2022	ITA → MKD	● Palm oil	39	0.006	\$6,500.00	\$1,275.99	0.980
7	☐	2024	GBR → ITA	● Copper	2,086	0.044	\$47,409.09	\$9,142.14	0.980
8	☐	2022	CAF → CHE	● Gold	3,179,015	0.018	\$176,611,944.44	\$57,890,634.27	0.979
9	☐	2018	USA → IRL	● Copper	264,595	6.879	\$38,464.17	\$6,529.80	0.979
10	☐	2020	ITA → TUR	● Copper	24,289,083	628.199	\$38,664.63	\$6,173.77	0.975
11	☐	2022	SGP → IDN	● Copper	112,738	2.09	\$52,042.11	\$8,822.27	0.975

HUMAN-REVIEW BOUNDARY This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Figure 11.3. The Top 50 Review Queue grid as the deployed application renders it, at full column width. The nine published columns run rank, year, corridor, product, declared value, declared quantity, implied unit value, same-year benchmark, review-priority score. The product cell carries the family colour dot; the score cell carries a proportional bar so relative priority is readable without inviting the number to be read as a probability; the selected row carries the current-case highlight. The boundary ribbon is fixed across the foot of the page and the grid scrolls beneath it.

11.5.1 Purpose

To turn 23,656 scored observations into a fixed-capacity, ordered, exportable review list where every row can be opened into its evidence.

11.5.2 Filters and ranking controls

Four multiselect filters are rendered above the table, all scoped to the queue: **Year**, **Product family** (labelled with its HS6 code), **Exporter** and **Importer** (both showing the country name beside the source code, with grouped BACI partner codes retaining their source label). Empty means unrestricted, so a cleared filter can never blank the queue by accident.

Supporting controls: a **Reset filters** button; one keyboard-accessible clear action per active filter, rendered as a chip; and a permanent result count.

The filter engine `apply_queue_filters()` additionally implements score-range and top-N restriction. The current Review Queue page does not render controls for those two, so they are available in the service layer but not exposed in the interface.

Ranking itself is **not** a user control. Rank is assigned by the pipeline and travels with its row: re-sorting the grid reorders the view only, never the rank and never the export order.

11.5.3 Default view and broader views

The default and only published view is the **Top 50**. There is no Top 500 or broader view, because the pipeline persists scores for the top 50 real observations only (`top_ranked_corridors.csv`). Population-level context on other pages therefore uses the panel's benchmark residuals rather than a full score distribution — a limitation the dashboard's own documentation records.

11.5.4 Columns displayed

Nine columns are published, in the order shown in Figure 11.3. The review-priority score is drawn as a proportional bar so relative priority is readable at a glance without inviting the number to be read as a probability.

Table 11.3. The nine Review Queue grid columns, in display order.

Column	Field	What it shows
Rank	<code>rank</code>	Queue position; 1 is the most unusual. Ranks stay with their rows when the grid is re-sorted
Year	<code>year</code>	Calendar year of the annual observation
Corridor	<code>derived</code>	Exporter → importer as three-letter codes; grouped partners keep their source code
Product	<code>family_label</code>	Colour-dotted family label; HS6 mapping shown in the caption
Trade value (USD)	<code>trade_value_usd</code>	Annual aggregate declared value
Quantity (metric tons)	<code>quantity_metric_ton</code>	Annual aggregate declared quantity
Unit value (USD/t)	<code>unit_value_usd_per_metric_ton</code>	Implied unit value — value ÷ quantity
Benchmark price (USD/t)	<code>benchmark_price_usd_per_metric_ton</code>	Same-year World Bank benchmark
Review-priority score	<code>selected_review_priority_score</code>	The blended ranking score

The only presentation-derived field is the corridor string. Every other value comes straight from the published queue at full precision; rounding happens at display time only.

11.5.5 How the review-priority score is explained in the product

Two identical explanations are rendered from one YAML anchor, so the score-column tooltip and the note under the table can never diverge:

“The score combines several unusual-pattern signals to determine which observations should be reviewed first. A higher score means higher review priority—it is not the probability or a finding of financial crime.”

A second footer note explains the implied unit value in the same terms this document uses: it is called *implied* because it is calculated from aggregate public data, not observed from an invoice or transaction price.

11.5.6 Data-quality and evidence indicators

Data quality is carried on the queue rows (`quality_status` , `data_quality_score` , `valid_extreme_flag`) and exported with them, and is presented per case on Selected Case Review with an explicit tooltip: *“Data quality rates how usable the underlying official record is. It is separate from how unusual the observation is.”*

Evidence count is present in the data but **not** used as a filter or size channel, because it is uniformly 4 in the current run. The dashboard records that as a known limitation rather than presenting a channel that carries no information.

11.5.7 Export

Export current view (.csv) downloads exactly the rows currently shown, always ordered by rank, at full numeric precision, as `review_queue_current_view.csv` . The export adds four traceability fields to the nine displayed columns: `obs_id` (joins to evidence and features), `quality_status` (the data-quality caveat), `source_row_id` and `source_version` (pin the exact pipeline inputs).

KEY POINT

In-app column sorting reorders the view only. The export is always rank-ascending, so a downloaded file can never carry a different ordering from the one the pipeline produced.

11.5.8 Row selection

Selecting a cell makes that row the **current case**: the `obs_id` is stored in session state, the row is highlighted in amber, and a pinned banner shows rank, year, corridor and product with an **Open case review** link. The selection survives filtering and page navigation. An orphaned selection — for example after a pipeline re-run — falls back harmlessly to the default case (rank 1).

11.6 Selected Case Review

One observation in depth, across three tabs.

11.6.1 Case identity and observed values

A compact amber selected-case strip carries the identity and the data-quality rating; a **Case facts** panel carries the observed values.

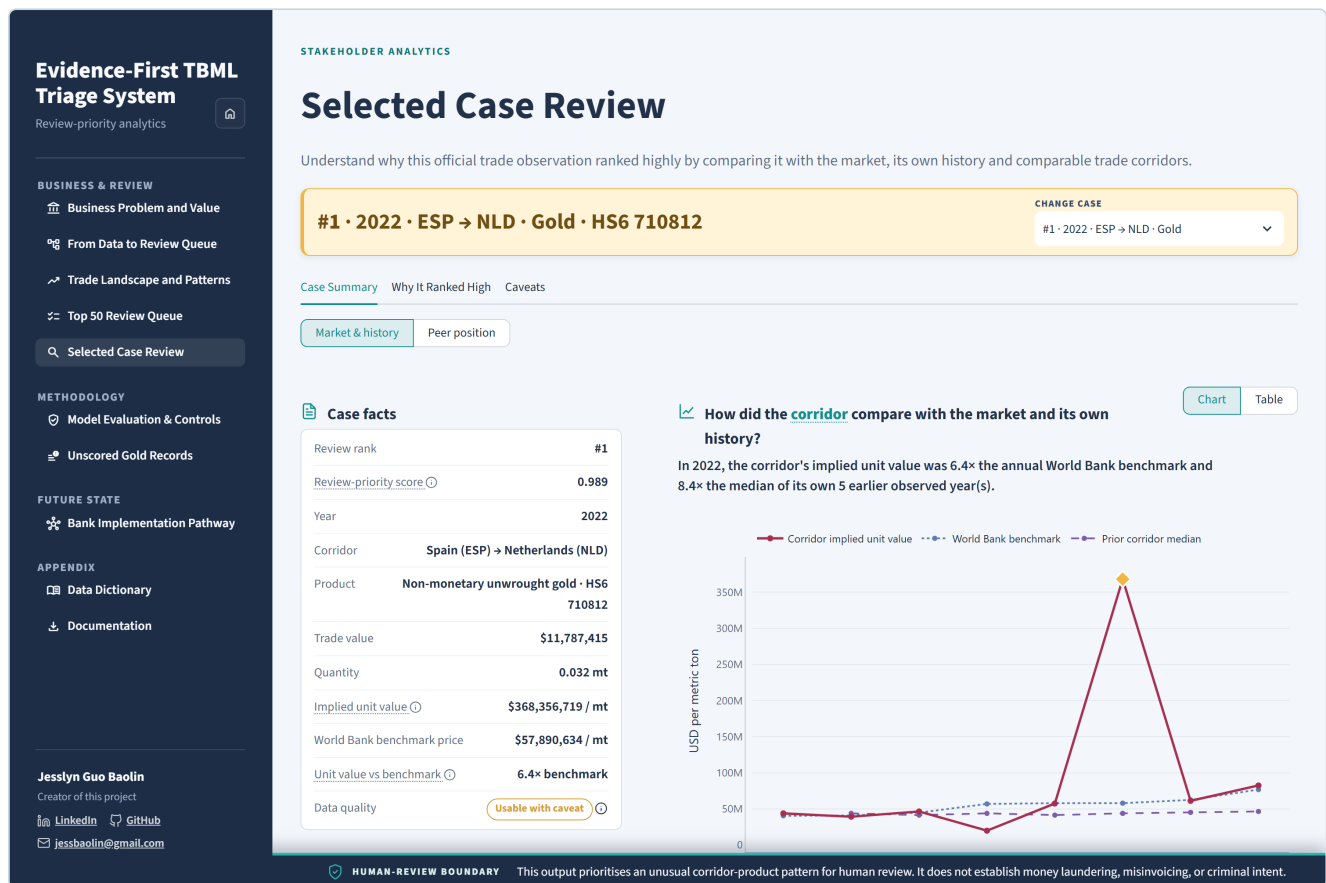


Figure 11.4. The deployed application, Selected Case Review page. The amber strip fixes the case identity and offers the case selector; three tabs separate the summary, the ranking explanation and the caveats; the Case facts panel lists the observed values with their definition tooltips; and the comparison view plots the corridor's implied unit value against the annual World Bank benchmark and the median of its own prior observed years. Unobserved years stay blank rather than being interpolated.

Table 11.4. Case facts panel fields, shown for the rank-1 case.

Field	Value for rank 1
Review rank	1
Review-priority score	0.9891
Year	2022
Corridor	Spain → Netherlands (ESP → NLD)
Product	Non-monetary unwrought gold (HS6 710812)
Trade value	\$11,787,415
Quantity	0.032 metric tons
Implied unit value	\$368,356,718.75 per metric ton
World Bank benchmark price	\$57,890,634.27 per metric ton
Unit value vs benchmark	6.36× benchmark
Data quality	usable_with_caveat (6 of 6)

The valid-extreme note appears where relevant: “An extreme multiple (over 5× or under 0.2× the benchmark) that was checked and kept as a valid observation, not treated as a data error.”

11.6.2 Comparison views

A three-view carousel, each with a **Chart | Table** mode toggle:

- **Market & history** — the corridor’s implied unit value over time against the annual World Bank benchmark and against the median of its own prior observed years. Unobserved years stay blank; they are never filled in or interpolated.
- **Peer position** — a histogram of the unit-value-to-benchmark multiple across all same-family, same-year corridors, with the selected case marked. For the rank-1 case: 6.36× benchmark, 99.36% of 1,405 peers.

Each view carries an explicit population note explaining what a peer is, and a note that both references are context, not invoice-level fair value.

KEY POINT

A **runtime consistency gate** guards the whole page. If any recomputed value disagrees with the published queue, the feature table or the clean panel, the page shows the failure reason **instead of numbers**. Showing nothing is preferred to showing a value that might be wrong.

11.6.3 Why It Ranked High

Three layers, deliberately separated.

A **dynamic ranking summary** grounded *only* in the case’s own evidence rows. `summary_clause_keys()` returns ordered `(evidence_type, direction)` keys and the page maps them to approved copy, so the summary can name a comparison as a reason **only** when that evidence type is actually present. It closes with: *“These are reasons to review this pattern more closely — not a finding of wrongdoing.”*

Four comparison cards showing the real, recomputable value for the case: sharp implied-price movement (`unit_value_yoy_change`), gap to broad market context (`benchmark_residual`), break from prior corridor behaviour (`robust_historical_z`), and benchmark-relative gap change (`benchmark_adjusted_drift`). Each card carries the technical metric name, what it was compared with, how it was derived, and its own caveat. Where a feature is genuinely missing — a corridor with no prior year — the card shows a data-absence message rather than a substituted value.

The complete signal profile, a collapsible table of all twelve display signals with the exact dataset field name, the case result, how it is derived, and its time-safety rule.

Log-space discipline is enforced in one place. `signal_number()` converts log-ratio features with `exp()` to give a multiple, scales the percentile to 0–100, and leaves the z-score, counts and flags untouched — so nothing else in the application can print a raw log value as if it were a price or a ratio.

Global SHAP is shown on the model page only, always labelled as global.

11.6.4 Caveats

Two lists: **case-specific caveats**, taken from the `caveat` field of the case’s own evidence rows, and **project-wide limitations**, taken verbatim from the final analysis report. The tab closes with *“Human review remains necessary before any conclusion.”*

11.6.5 Benign explanations, missing information and review actions

These are carried in the evidence caveats and the typology cards rather than as a separate panel: every evidence row has a mandatory caveat naming what could otherwise explain the value, and the typology cards supply the recommended human-review steps set out in Chapter 9.4.7.

11.7 Unsourced gold records

The dedicated page *Why Some Gold Records Cannot Be Scored* answers the question the anomaly queue leaves behind. Every figure is derived live from the panel through `services/gold_coverage.py`; the notebook remains the verified specification.

11.7.1 Why these records cannot receive a score

Without a usable quantity there is no implied unit value, so every unit-value signal is undefined and no rules or model score can be computed. **1,877 of 12,205 gold records (15.38%)** are in this position.

11.7.2 Why they are retained

They are real, verified, hash-traceable declared values. Discarding them would hide a coverage limitation and break the audit trail. They are retained in the panel with `quality_status = excluded_from_modeling_retained_for_audit`, listed in the exclusion audit, and surfaced on their own page.

11.7.3 What information remains available

Declared value, year, corridor, family, `obs_id`, `source_row_id`, `source_version` and `source_file_hash` — enough for materiality assessment, coverage monitoring, source follow-up and full lineage.

11.7.4 How the dashboard separates data quality from anomaly ranking

Structurally and visually. The page sits in the **Methodology** group, not in the review path. It opens with a coverage-boundary banner stating that a blind spot here means *unavailable scoring*, not hidden risk. It uses no risk colouring. It states that network position is layout only. It shows numerator and denominator in every rate. And it closes with the operational conclusion: *keep incomplete rows outside unit-value scoring; trace the 14 recurring routes; monitor record and value coverage separately*.

The page structure follows three questions:

1. **Is the blind spot material?** Row share against value share, by product family.
2. **Is it isolated or systematic?** One large record (70.9% of gap value) against 14 corridors unscorable in all 8 years.
3. **Where does the repeat pattern sit?** An interactive directed network of the 14 persistent routes, with a country-code reference panel, five directional views, and a full route table.

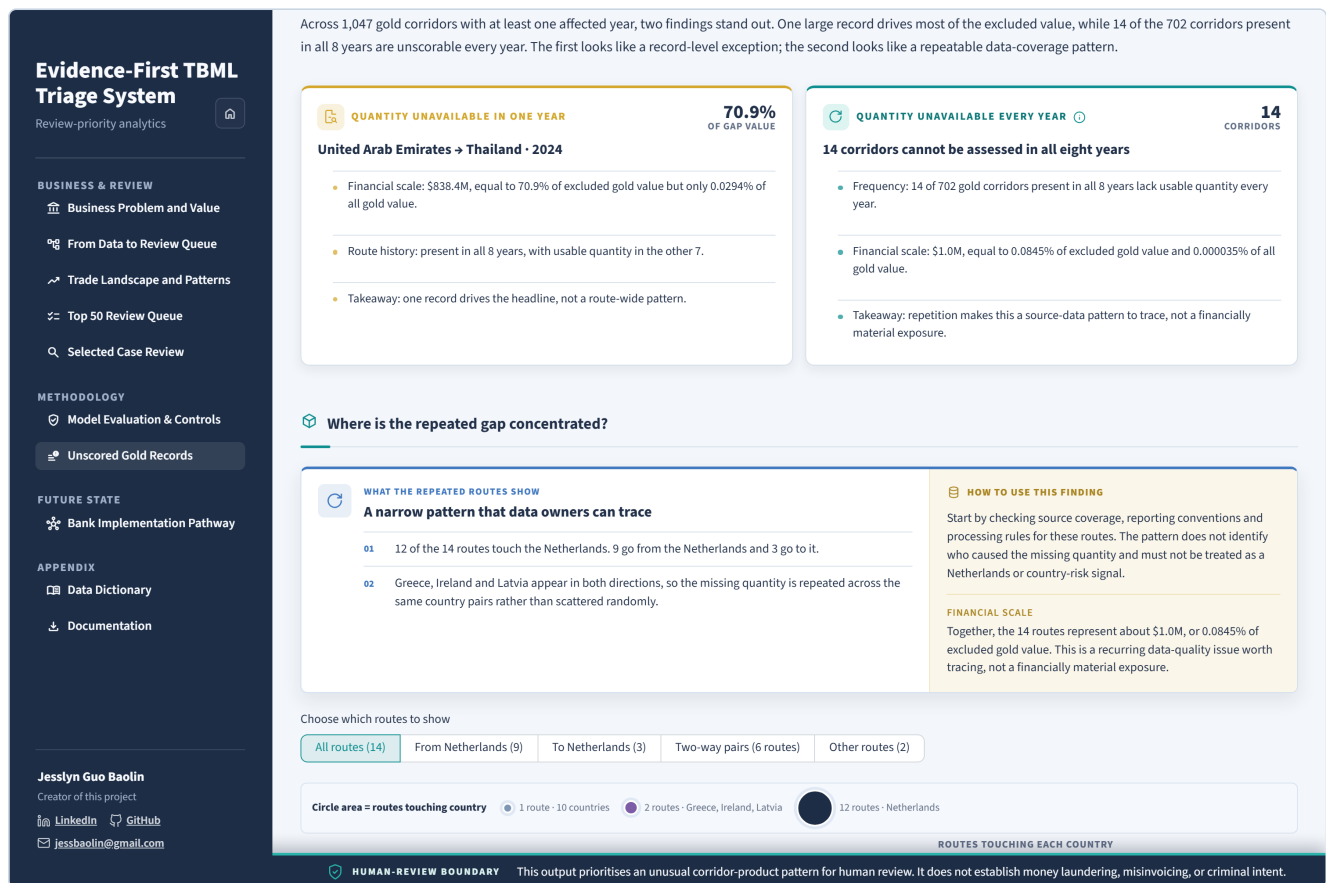


Figure 11.5. The deployed application, Unsourced Gold Records page. The interactive directed network of the 14 persistent coverage-gap routes, with its directional view control, the circle-area size guide, and the country-code reference panel that resolves every ISO3 node label. The page states in its own copy that node position is layout only and carries no geographic or risk meaning.

11.7.5 How a bank could investigate or enrich these records

Match the corridor, product and year to the bank's own trade-finance and payment records; obtain the customs declaration and shipping documents that carry the physical quantity; check whether a reporting convention or threshold explains the omission; and where the record is material, validate it against the source extract and mirror data.

11.8 Model Evaluation & Controls

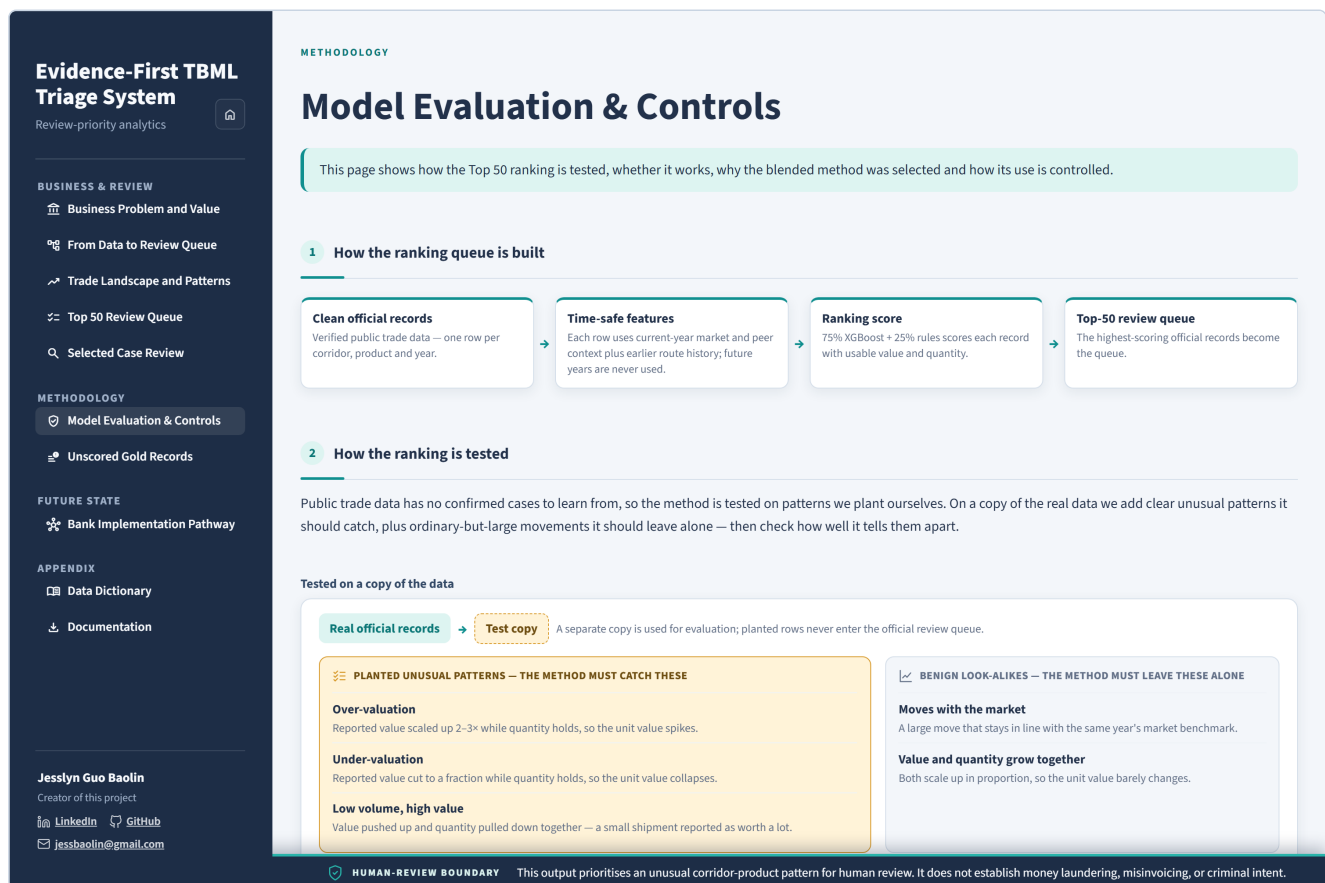


Figure 11.6. The deployed application, Model Evaluation & Controls page — the governance surface. It sets out how the ranking queue is built and how the method is tested on a copy of the data, and it labels every performance figure as a synthetic-scenario result rather than a detection rate.

The governance page, and the most detailed in the application. Its sections run: how the ranking queue is built; how the ranking is tested (the scenario design, with the planted patterns and the benign look-alikes side by side); how well it ranks them (the time-based test design, the headline takeaway, a review-capacity curve, and an expandable calculation panel); why this method was chosen (the retained formula, the fixed weights, the two deductions, and the validation blend table); the held-out test comparison of all five methods; **what keeps it honest** — live integrity checks including the no-synthetic-rows control, and the explicit allowed / not-allowed language pair; and a technical drawer for model reviewers holding the evaluation protocol, reconciled sample counts, the full metric table by split and family, metric definitions, the global SHAP summary, and the selected XGBoost settings.

It closes by handing over to the coverage page: *“One question remains: which official gold records never reached that assessment because usable quantity was unavailable?”*

11.9 Supporting pages

Trade Landscape and Patterns separates market scale from unusual behaviour: trade value and quantity by family and year on independent scales, benchmark movement, and the queue’s own year and family profile. Its caption states the point directly — *scale tells you where the money is, not where the unusual behaviour is.*

From Data to Review Queue walks one real observation through the pipeline, with a time-safe panel that names which years may inform the assessed year and which are hidden.

Bank Implementation Pathway is a clearly labelled future-state design (Chapter 12).

Data Dictionary merges three layers in decreasing authority — the approved feature explanations, the pipeline’s field dictionary, and schema introspection — and labels schema-inferred rows as such rather than inventing definitions. It also renders the official source cards with clickable URLs and their URL origin.

Documentation is the download point for the project’s two written deliverables — the stakeholder project brief and this technical documentation. Each is presented as a card carrying its purpose and its current status; the download controls are rendered disabled, and the status line says so, rather than offering a file the page cannot supply.

11.10 Test coverage

Table 11.5. Test modules asserting dashboard and analytical behaviour (160 tests, status pass).

Test module	Lines	What it asserts
tests/test_case_summary.py	223	Case-summary service: comparison views, multiples, peer position
tests/test_dashboard_component_contract.py	63	Shared component contract
tests/test_dashboard_data.py	76	Loader behaviour and required/optional file handling
tests/test_dashboard_landing.py	136	Landing route and entry behaviour
tests/test_dashboard_metrics.py	148	Queue enrichment, filters, KPI and landscape metrics
tests/test_dashboard_pages.py	798	Every page renders against the real artefacts
tests/test_dashboard_sources.py	104	Source cards, URL policy and provenance
tests/test_dashboard_theme_contract.py	93	Theme tokens and Streamlit base-theme agreement
tests/test_gold_coverage.py	228	Gold coverage figures, persistence and network
tests/test_model_evaluation.py	358	Model-evaluation reads, integrity checks, no-synthetic-rows control
tests/test_trade_landscape.py	159	Trade-scale and benchmark aggregations
tests/test_why_ranked_high.py	265	Log-space conversion, card metrics, evidence-driven summary

These are contract tests: they assert that the queue carries clean-panel values, that no scenario row reaches the queue, that displayed values match the artefacts, that the theme contract holds, that the gold coverage figures reconcile, and that the why-ranked-high logic converts log space correctly.

CHAPTER 12

Bank Implementation Pathway

CAUTION

This chapter is a **future-state design**. Nothing in it is built. The repository contains no connection to bank data, no case-management integration, no access control, no audit logging, and no production decision path. A public-data prototype cannot be moved into production without every control described here.

12.1 The shape of the target state

Public data can show that a corridor–product pattern is unusual. Bank records show who was involved, what was agreed, whether goods moved, and how payment was made. An analyst decides what the combination means. The pathway below joins the first to the second while keeping the third firmly human.

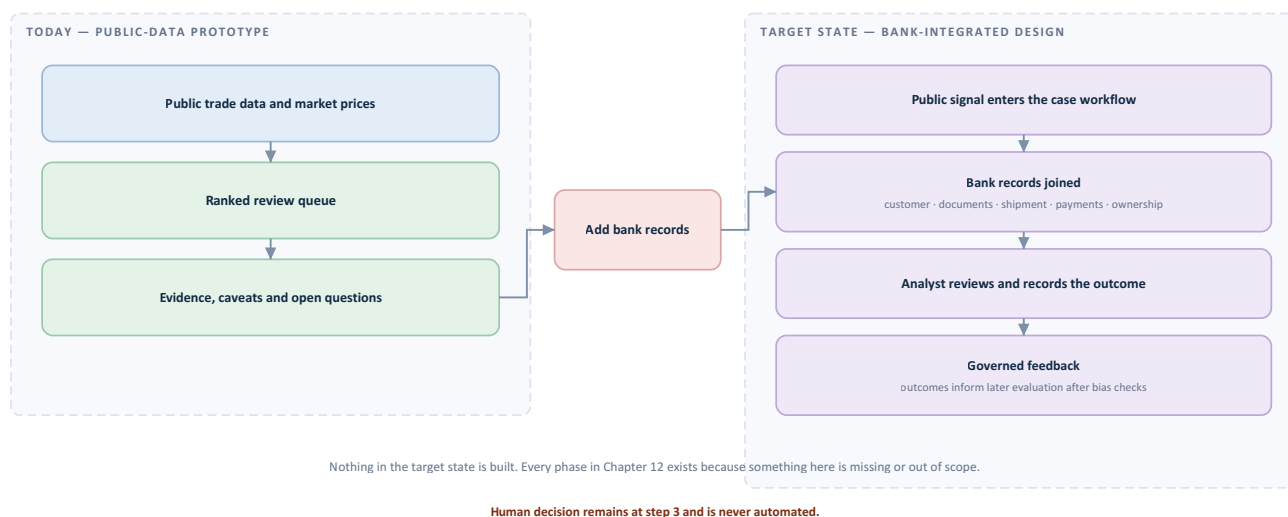


Figure 12.1. Present state and target state. Today the prototype ends at an evidence-backed queue. In the target state the queue enters the bank's existing case workflow, is enriched with bank-held records, is reviewed by an analyst, and the recorded outcome feeds — after quality and bias checks — into later evaluation.

12.2 Phase 1 — Controlled analytical proof of concept

Objective. Establish whether the external signal is informative against the bank's own portfolio, in a contained environment, with no production decision attached.

Scope. Run the existing pipeline over the commodity families and corridors the bank actually finances, extended beyond the three HS6 codes in this repository. Keep it read-only and offline.

Activities. Extend the HS6 scope and re-verify sources. Re-run source verification, panel construction and feature engineering at the wider scope. Reconstruct scenario evaluation at that scope and confirm the ranking behaviour holds. Compare the resulting corridors against the bank's own trade-finance book *manually*, at aggregate level, to see whether the signal lands anywhere near real activity.

Exit criteria. Reproducible pipeline at bank scope; evaluation record equivalent to Chapter 9; a documented view from AFC subject-matter experts on whether the ranked patterns are plausible review candidates.

Controls. No customer data. No production decision. No output leaves the analytical environment.

12.3 Phase 2 — Integration with internal trade-finance and payment data

Objective. Join the external signal to bank-held records, in a governed environment, so a queued pattern can be resolved to actual exposure.

Data domains to be joined.

Table 12.1. Bank-held data domains required, and what each contributes.

Domain	Contributes	Key control questions
Customer and counterparty (KYC/CDD)	Identity, expected activity, relationship history, jurisdictions	Does this corridor and product fit the customer's expected profile?
Trade-finance documents	Contracts, commercial invoices, packing lists, certificates of origin and quality, Incoterms	Do the documents support the price, quantity, goods description and terms?
Payments and correspondent data	Payment instructions and confirmations, routing, currency, value dating, third-party payers	Who paid whom, through which banks, in what currency, when?
Shipping and logistics	Bills of lading, airway bills, customs declarations, inspection records, warehouse receipts	Did the declared goods and quantity actually move?
Beneficial ownership	Ultimate ownership and control, connected entities	Who ultimately controls the trading parties, and are the relationships expected?
Sanctions and adverse information	Screening results for all parties and connected entities	Does any party require separate escalation?
Case history	Prior alerts, reviews, dispositions and filings	Has this relationship or pattern been reviewed before, and what was found?

Matching design. The external signal is at corridor–product–year grain; bank records are at transaction grain. Matching is therefore **probabilistic and advisory**, and every candidate match must show why it is a candidate. A weak match must be visible as a weak match; it must never be silently promoted into a finding.

Controls. Purpose limitation and lawful basis agreed before any join. Data-minimisation on what is pulled. Full lineage from any joined record back to its system of record.

12.4 Phase 3 — Analyst feedback and label development

Objective. Begin building the real labels that public data cannot supply.

Why it comes third. Labels can only be created by humans reviewing real cases with real records. Attempting supervised learning on real outcomes before Phase 2 would have nothing to learn from.

Activities. Capture a structured disposition for every reviewed case: outcome, reason, which evidence mattered, which benign explanation applied, what information resolved it. Track inter-analyst agreement. Build a label taxonomy that distinguishes *reviewed and explained*, *reviewed and escalated*, *insufficient information* and *not reviewed*.

Controls. Labels reflect review outcomes, not confirmed criminality, and the distinction must be preserved in the schema itself. Review capacity and analyst assignment introduce selection bias into any resulting label set, and that bias must be measured before the labels are used.

12.5 Phase 4 — Model governance and validation

Objective. Bring the scoring approach inside the bank's model-risk framework.

Table 12.2. Model-governance requirements.

Requirement	What it means here
Model inventory and tiering	Register the score; tier it by use. A triage-ordering model that never makes a decision on its own tiers lower than a decisioning model, but it is still a model
Conceptual soundness	Document why each feature exists, its time-safety rule, and its known failure modes — Chapter 8 is the template
Data quality	Source verification, hash pinning, the exclusion audit and the coverage analysis of Chapter 10.2
Independent validation	Re-derive the metrics; challenge the scenario design; test the sensitivity of the queue to the blend weight and to the queue size
Performance monitoring	Define what “working” means for a triage queue when ground truth is partial and delayed
Explainability	Evidence rows as the primary explanation; model contribution as a secondary, clearly-labelled diagnostic
Documentation	A model card covering purpose, inputs, method, limitations, monitoring and approvals
Approval and review cycle	Named approver, scheduled re-validation, change control on features, weights and thresholds

Specific gaps to close before approval. No uncertainty quantification exists (Chapter 4, M3). No calibration exists, and none should be added unless the score’s use changes. Family-level performance varies materially (M5) and must be either accepted with a documented rationale or addressed. The synthetic-label basis must be stated in the model card as the primary limitation.

12.6 Phase 5 — Controlled pilot

Objective. Run the queue with real analysts on real cases, at limited scale, with a defined stop condition.

Design. A limited corridor and product scope; a fixed queue capacity matched to genuine review capacity; a defined pilot period; a control comparison, since a queue that merely reorders work an existing process would have found anyway adds no value.

What to measure. Review completion rate against capacity. Time to first meaningful finding. The proportion of cases resolved by a benign explanation, and which explanation. Analyst-reported usefulness of the evidence rows. False-lead rate. Whether any case escalated on the strength of the public signal *and* bank records.

Human-in-the-loop controls. No automated escalation. No automated filing. No automated customer action. Every disposition is a recorded human decision with a stated reason.

Stop conditions. Agreed in advance: unusable precision, unmanageable review burden, or evidence of bias in what the queue surfaces.

12.7 Phase 6 — Production monitoring and continuous improvement

Objective. Operate the queue as a governed production capability.

Table 12.3. Production controls required.

Control	Requirement
Case-management integration	The queue writes into the existing case system; dispositions are captured there; nothing is worked outside it
Access control	Role-based access to the queue, to evidence, and to any joined bank record; least privilege enforced
Data privacy	Purpose limitation, retention schedule, minimisation, and a lawful basis for every joined domain
Audit logging	Every view, filter, export and disposition logged with user, timestamp and reason

Control	Requirement
Evidence retention	Evidence rows retained immutably with the case, so a decision can be reconstructed years later
Model monitoring	Score-distribution stability, queue-composition stability, feature-availability trends, and eligibility-rate trends
Drift monitoring	Input drift (source coverage, reporting conventions, benchmark behaviour), output drift (score and queue composition), and concept drift where labels permit
Coverage monitoring	Record-level and value-level coverage, tracked separately — the direct operational lesson of Chapter 10.2
Feedback loops	Dispositions inform re-evaluation only after quality and bias checks; a feedback loop that learns from a biased review process will amplify the bias
Change control	Features, weights, thresholds and queue size under version control with an approval gate
Governance approvals	Model-risk approval, data-privacy approval, and AFC sign-off before production use, with a scheduled re-validation

12.8 Where an AI assistant could help — and where it must not

The dashboard's future-state page sets out a bounded role for AI, and the boundary is the point. The design is expressed in the product as a three-step flow and an explicitly labelled concept mockup.

Evidence-First TBML Triage System
Review-priority analytics

BUSINESS & REVIEW

- Business Problem and Value
- From Data to Review Queue
- Trade Landscape and Patterns
- Top 50 Review Queue
- Selected Case Review

METHODOLOGY

- Model Evaluation & Controls
- Unscored Gold Records

FUTURE STATE

- Bank Implementation Pathway**

APPENDIX

- Data Dictionary
- Documentation

Jesslyn Guo Baolin
Creator of this project
LinkedIn GitHub
jessbaolin@gmail.com

Who is the customer, what activity is expected, and does this trade route fit the relationship?

Do contracts and invoices support the price, quantity, goods description, terms and Incoterms?

Shipment and customs records
Did the stated goods and quantity move as declared in shipping and customs records?

Payments and correspondent data
Who paid whom? Through which banks, in what currency, how much and when?

Sanctions and adverse media
Do any parties or connected entities need a separate sanctions or adverse-media review?

Where AI could help: connect the records, leave the decision to the analyst

AI'S ROLE IN REVIEW
The ranking shows where review could begin. With access to approved bank records, an AI assistant could find related activity across KYC files, trade documents, shipment records and payment systems, compare the details, and organise source-linked evidence showing what agrees, conflicts or remains missing. It would not decide whether the activity is suspicious or what action to take.

FIND THE RECORDS
Find the bank activity behind the public signal
Use the countries, product, year and counterparties to find possible customer

COMPARE THE DETAILS
Check whether the records tell the same story
Compare names, goods, prices, quantities, currencies, dates and terms across contracts,

PREPARE FOR REVIEW
Lay out the case for the analyst
Build a timeline and relationship view, show which records support or challenge each possible explanation, and suggest the questions that still

HUMAN-REVIEW BOUNDARY This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Figure 12.2. The deployed application, Bank Implementation Pathway page — the bounded role for an AI assistant, stated as three steps: find the bank activity behind the public signal, check whether the records tell the same story, and lay the case out for the analyst. The role statement above the steps is explicit that the assistant would not decide whether activity is suspicious or what action to take.

What it could do. With access to approved bank records: find the activity behind a public signal across KYC files, trade documents, shipment records and payment systems; compare names, goods, prices, quantities, currencies, dates and terms across those records and mark what matches, what conflicts and what is missing; and lay the case out for the analyst as a timeline and a relationship view with the questions that still need answers.

What it must not do. Decide whether activity is suspicious. Decide what action to take. Assert a fact without a source. Answer a question the records do not answer.

Required guardrails, as stated in the product design: use only records the analyst can access; link every fact to its source; say explicitly when the records do not answer a question; keep a log and fall back to fixed templates if validation fails.

12.8.1 The concept mockup, as it appears in the product

The dashboard carries a working mockup of how the assistant would behave inside the Selected Case Review page. It is drawn from the same rank-1 case documented in Chapter 9.4.3, so the public-data figures in it are the real ones.

Selected Case Review · Rank 1 · ESP → NLD

Flagged case

ESP → NLD • Gold · 2022

0.989 review-priority score

Trade value	\$11,787,415
Quantity	0.032 t
Unit value	\$368,356,718.75/t
Benchmark price	\$57,890,634.27/t

Evidence assistant
Connects records · never decides

Powered by AI

Records compared

- ✓ **Match** Customer relationship fits the ESP → NLD gold route (KYC/CDD)
- ✗ **Conflict** Invoice unit value sits far above the market benchmark (Invoice #4471)
- **Gap** No customs weight found for the declared 0.032 t (Customs — none)
- ✓ **Match** Payment routed through the expected correspondent bank (SWIFT MT103)
- ⚠ **Review** Adverse-media item on a connected entity needs a look (Adverse-media)
- **Gap** Ultimate owner of the buyer not yet confirmed (Ownership — partial)

Ask the case

Any sanctions or adverse-media hits on the counterparties?

No sanctions match 1 item to review

No sanctions-list match on either counterparty. One adverse-media item on a connected entity — flagged for a separate human review.

Screened: sanctions & adverse-media service

Ask about this case...

Answers cite their sources · the assistant does not decide the outcome.

Linked bank records • Integrated

Open the source files behind this case.

- KYC/CDD**
Customer profile · 2 files
- Trade-finance documents**
Invoice, contract · 3 files
- Shipment & customs**
Bill of lading · 1 file
- Payments & correspondent**
SWIFT MT103 · 2 files
- Sanctions & adverse media**
Watchlist + media screen
- Beneficial ownership**
UBO structure · 2 files

HUMAN-REVIEW BOUNDARY This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Figure 12.3. The AI evidence-assistant concept mockup, captured from the deployed application's Bank Implementation Pathway page. Left: the flagged case, carrying the real rank-1 public-data figures. Right: the assistant's record comparison, where each line is marked Match, Conflict, Gap or Review and carries the bank record it came from, followed by a question-and-answer exchange whose answer cites its screening source. Bottom: the linked bank records the case would draw on. The badge on the assistant reads "Connects records · never decides".

CAUTION

This is a **concept mockup**, and the product labels it as one. No AI model is called anywhere in the repository, no bank record exists in it, and the invoice, customs, SWIFT, adverse-media and ownership entries shown in the mockup are illustrative placeholders — not data. Only the flagged-case panel on the left carries real values, and those come from the public-data queue.

Three details in the mockup carry the design intent:

- **Every line is attributed.** Each compared record shows its source — `KYC/CDD`, `Invoice #4471`, `Customs — none`, `SWIFT MT103`, `Adverse-media`, `Ownership — partial`. A gap is shown as a gap, not silently omitted.
- **The verdicts are descriptive, not evaluative.** *Match*, *Conflict*, *Gap* and *Review* describe the state of the records. None of them says whether the activity is suspicious.
- **The answer names its screening source and hands the decision back.** The footnote under the exchange reads “*Answers cite their sources · the assistant does not decide the outcome.*”

KEY POINT

This is the same discipline the current repository already applies without any model at all. `src/08_build_briefs.py` renders briefs deterministically from structured evidence and validates every one against evidence coverage, unsupported numeric claims, invented sources, missing caveats, the exact boundary sentence, and a prohibited-phrase list. An AI layer would have to meet that standard, not relax it.

12.9 Summary of the pathway

Table 12.4. Phase summary.

Phase	Objective	Principal control
1	Controlled analytical proof of concept	No customer data, no production decision
2	Integration with internal trade and payment data	Purpose limitation; advisory matching with visible confidence
3	Analyst feedback and label development	Labels are review outcomes, not confirmed criminality
4	Model governance and validation	Independent validation; documented limitations; named approver
5	Controlled pilot	Human-in-the-loop only; pre-agreed stop conditions
6	Production monitoring and improvement	Full monitoring, audit, privacy and change control

CAUTION

The public-data prototype cannot be moved directly into production. Every phase above exists because something in the current implementation is missing, unverified, or explicitly out of scope.

CHAPTER 13

Reproduction Workflow

Every command below is taken from the repository's own documentation and scripts, and was executed on Windows 11 with Python 3.12 during the preparation of this document. Commands are PowerShell; on Linux or macOS replace `.venv\Scripts\activate` with `source .venv/bin/activate`.

13.1 Clone the repository

```
git clone https://github.com/jessybaolin/evidence-based-tbml-corridor-drift.git
cd evidence-based-tbml-corridor-drift
```

13.2 Create the environment and install dependencies

Two dependency sets exist. For the full pipeline, use `requirements-pipeline.txt`. For the dashboard alone, `requirements.txt` is sufficient and much smaller.

```
py -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements-pipeline.txt
```

13.3 Prepare the source files

The pipeline expects 8 declared inputs under `data/raw/`. They are present in the repository; if any is missing, `src/01_verify_sources.py` fails with the exact filename.

Table 13.1. Required inputs under `data/raw/`.

File	Source
<code>baci_hs17_v202601_selected_2017_2024_151110_740311_710812.parquet</code>	CEPII BACI, filtered to the three HS6 codes and eight years
<code>data_source_notes.json</code>	Provenance and checksum record for the extract
<code>CMO-Historical-Data-Annual.xlsx</code>	World Bank Commodity Markets Outlook annual series
<code>country_codes_V202601.csv</code>	CEPII country reference
<code>product_codes_HS17_V202601.csv</code>	CEPII HS17 product reference
<code>Trade-Based-Money-Laundering-Trends-and-Developments_2020.pdf</code>	FATF–Egmont (typology context only)
<code>Trade-Based-Money-Laundering-Risk-Indicators_2021.pdf</code>	FATF–Egmont (typology context only)
<code>baci-readme.txt</code>	Publisher's own readme

13.4 Validate the environment

```
python src/00_check_environment.py
```

Writes `data/outputs/environment_check.json` and **exits non-zero if any required package is missing**, so a broken environment stops the run immediately.

13.5 Run the data pipeline

The scripts are ordered and each depends on its predecessor.

```
python src/01_verify_sources.py          # chain of custody
python src/02_ingest_official_sources.py # raw -> interim staging
python src/03_build_panel.py             # canonical clean panel + exclusion audit
python src/04_build_features.py          # time-safe features + feature dictionary
python src/05_build_scenarios.py         # seeded scenario injection (labels held apart)
python src/06_train_evaluate_models.py   # fit, evaluate, select, score, Top 50
python src/07_build_evidence.py          # recomputable evidence rows
python src/08_build_briefs.py            # deterministic caveated briefs + validation
python src/09_generate_reports.py        # diagrams and written reports
```

Table 13.2. What each stage should print, and the artefacts it should produce.

Stage	Expected console output	Artefacts
01	Inventory rows, BACI row count, counts-match flag, benchmark rows, PDF checks	source_file_inventory.csv, source_manifest.json
02	wrote 25,844 BACI rows and 24 benchmark rows	2 interim Parquet files
03	panel rows=25,844; audit/caveat rows=13,950	panel Parquet + CSV, exclusion audit
04	features rows=25,844; feature table written	corridor_features.parquet, reports/feature_explanation_table.md
05	scenario injections=540; positives=324; hard negatives=216	scenario panel, labels, injections, split manifest
06	Initial metric table, then the selection JSON	model artefacts, top_ranked_corridors.csv, 3 figures
07	Wrote 200 evidence rows	evidence_table.csv (+ repo-root copy), updated queue
08	Brief count and validation status	briefs, validation report, prompt assets
09	Report paths	reports, diagrams, report_manifest.json

13.6 Run the notebook analyses (optional)

The two analytical notebooks in `data-profiling/` read the generated outputs and produce no pipeline artefacts. They are **not required** for the dashboard or the reports.

```
python -m pip install jupyterlab
python -m jupyter lab data-profiling/business_Review_Data_profiling_before_revamp.ipynb
python -m jupyter lab data-profiling/gold_nan_quantity_analysis.ipynb
```

CAUTION

`data-profiling/` is git-ignored. The notebooks are local analytical work, not published deliverables, and none of them should be run before the pipeline: they read `data/processed/` and `data/outputs/`. Do not overwrite the original notebooks — open a copy if you intend to edit.

13.7 Verify the outputs

Three independent checks confirm a run is correct.

Test suite. 160 tests over the dashboard’s data contracts, metrics, page behaviour, theme contract, gold coverage and model evaluation.

```
python -m pytest tests/ -q
```

Key output reconciliation.

Table 13.3. Expected values after a clean run against the pinned inputs.

Check	Expected
BACI source SHA-256	0e493e057c8fd579...
Panel rows	25,844
Model-eligible rows	23,656
Rows excluded from modelling, retained for audit	2,188
Feature rows	25,844
Scenario injections	540
Clean-panel hash at injection	1552233d8f262081...
Selected challenger / weight	xgboost / 0.75
Review queue rows	50
Evidence rows	200
Gold coverage-gap rows	1,877
Persistent gold gap corridors	14

Source manifest. `data/outputs/source_manifest.json` should show every BACI cross-check passing and both FATF PDFs readable.

13.8 Run the dashboard

```
python -m streamlit run dashboard/streamlit_app.py
```

Streamlit prints a local URL, usually `http://localhost:8501`. Keep the terminal open while using the application.

13.9 Troubleshooting

Table 13.4. The most common failure modes and their causes.

Symptom	Cause	Resolution
Required input file is missing: ...	A file is absent from <code>data/raw/</code>	Restore the named file; the 8 inputs are listed in Section 13.3
Run <code>02_ingest_official_sources.py</code> before panel construction	Stage 03 run before stage 02	Run the stages in order
World Bank unit mismatch for <commodity>	The workbook’s unit string no longer matches <code>configs/benchmark_series.yml</code>	Do not bypass this check — it is what prevents an unconverted price entering the pipeline. Confirm the published units and update the config deliberately
Expected 24 annual benchmark rows; parsed N	The workbook layout or year coverage changed	Inspect the <i>Annual Prices (Nominal)</i> sheet; the parser discovers positions by content, so a genuine change must be reflected in configuration

Symptom	Cause	Resolution
Canonical key is not unique; examples=...	A reference join multiplied rows	Check <code>country_codes_V202601.csv</code> for duplicate numeric codes
Forbidden fields entered features: ...	A label column reached the feature frame	This guard is working as designed — remove the leak rather than the guard
Train and validation splits need both positive and negative scenario labels	Injection produced no positives in a split	Confirm <code>configs/project.yml</code> split years and that scenario candidates exist per family
Required project output is missing: ... (in the dashboard)	The pipeline has not been run	The error names the exact file and the command; run it and reload
ModuleNotFoundError: dashboard	Launched from outside the repository root by an unusual method	Use <code>python -m streamlit run dashboard/streamlit_app.py</code> from the repository root
Port already in use	Another Streamlit session is running	Add <code>--server.port 8502</code>
<code>src/09</code> fails at PDF conversion	WeasyPrint's native libraries are absent	Every analytical output and the dashboard are unaffected; install the GTK/Pango dependencies only if the PDF reports are needed
Diagrams render as placeholders	Graphviz <code>dot</code> is not on the PATH	Install Graphviz; the <code>.dot</code> sources are always written regardless

CHAPTER 14

Conclusion

14.1 What the project demonstrates

It demonstrates that public trade statistics can be turned into a **defensible review-priority queue**, and that the defensibility comes from the discipline of the construction rather than from the sophistication of the model.

Five things are established by the implementation and its outputs.

A complete, verifiable path exists from published source file to reviewable case. 8 hash-pinned inputs, cross-checked against the publisher's own provenance record; 25,844 panel observations with 0 rows silently lost; 22 time-safe features; 5 compared scoring methods; a 50-row queue; 200 evidence rows; 5 validated briefs; a deployed application; 160 passing tests. Every step is reproducible from the repository.

Time safety and label separation can be enforced structurally rather than by convention. Baselines are built from strictly prior years by construction. Label columns are rejected by an assertion if they reach the feature frame. The clean panel is content-hashed before the scenario copy is taken. The test years are opened once. Each of these is code, not a promise.

A queue can be evaluated honestly without ground truth. Controlled scenarios paired with hard negatives measure two different things — whether a method finds what it should, and whether it stays quiet on what it should ignore. On the held-out test the selected method concentrated injected patterns **26.8× above random** while admitting **no hard negative at all** into the top 50. Every metric is labelled as a synthetic-scenario result.

Evidence-first is achievable, not aspirational. Evidence is rebuilt from the feature table rather than decomposed from a score, so it survives a change of model. Each of the 200 rows names its metric, its observed value, its comparison group, its threshold, its severity and its caveat. Every generated brief passes an automated check that its citations exist, its numbers are supported, its caveats are present, the exact boundary sentence appears, and no prohibited phrase is used.

Data quality can be measured as a control rather than folded into risk. 2,188 observations cannot be scored because they have no usable quantity. Rather than dropping them or letting missingness leak into the ranking, the project quantifies the gap on two denominators, shows that the value is dominated by a single record (70.9% of the gold gap), shows that 14 corridors are unscorable in every year of the panel, and demonstrates that those two sets **do not overlap at all** — so they require two different controls.

14.2 What the current result is useful for

- **Prioritising analyst attention** across a defined commodity and corridor scope, with a stated concentration factor rather than an assertion of usefulness.
- **Explaining a ranking** to an analyst, a reviewer, or a stakeholder in terms of observable, recomputable facts.
- **Framing a review** — what to check, what could benignly explain it, and what information is missing.
- **Demonstrating a governance pattern** for evaluating a ranking model where ground truth does not exist.
- **Measuring coverage** as a first-class control, separately from risk.
- **Communicating with stakeholders** through a working product rather than a description of one.

14.3 What remains outside its scope

The list is short, and it is structural rather than a matter of effort.

The project cannot identify a party, price an invoice, confirm that goods moved, trace a payment, establish ownership, estimate a probability of wrongdoing, or reach any conclusion about criminality. It cannot do these things because the data does not contain them, and no modelling choice changes that.

Its evaluation rests on synthetic labels and reports no uncertainty. Its explanation is per-case for evidence and global-only for model contribution. Its dashboard is a read-only prototype without case management, access control, audit logging, monitoring, or a feedback loop.

14.4 How evidence-first design improves interpretability and governance

Three properties follow directly from the design, and each is worth more than a marginal gain in a ranking metric.

Explanations are independent of the model. Because evidence is recomputed from features rather than derived from the score, replacing the model does not invalidate the evidence. A model-risk reviewer can check an evidence row without accepting anything about the model.

Language is constrained mechanically. The boundary sentence has one source. Every evidence row carries a mandatory caveat. Typology cards fix both the allowed and the prohibited phrasing. A generated brief fails validation if it overstates. Interpretation discipline is enforced by code rather than by review.

Concepts that are usually conflated are kept apart. Data quality and review priority. Model contribution and factual evidence. Typology context and row-level evidence. Coverage gaps and risk signals. Each separation is visible in the artefacts, the code, and the interface.

The cost is honest and worth stating: the retained hybrid scored marginally below standalone XGBoost on the held-out test (48% against 50% precision at 50). The auditable component was required before the test set was opened, and it was kept. That is one row in fifty, exchanged for a score a reviewer can partly recompute by hand.

14.5 What would be needed for bank implementation

Chapter 12 sets out the six phases. In one paragraph: extend the scope and re-verify at bank scale; join the signal to customer, document, shipment and payment records under a lawful basis and with visible match confidence; capture structured analyst dispositions to begin building real labels; bring the score inside the model-risk framework with independent validation and a named approver; pilot with real analysts under human-in-the-loop controls and pre-agreed stop conditions; and operate with case-management integration, access control, audit logging, privacy controls, evidence retention, model and drift monitoring, coverage monitoring, and change control.

Nothing about the public-data prototype makes any of that unnecessary. What it does supply is a starting signal, a defensible construction, and a documented set of limitations to design against.

14.6 The boundary

Everything in this document — every number, every chart, every ranked record, every evidence row — is subject to a single constraint, stated once in `configs/project.yml` and reproduced identically in every output the project produces:

HUMAN-REVIEW BOUNDARY

This output prioritises an unusual corridor-product pattern for human review. It does not establish money laundering, misinvoicing, or criminal intent.

Loaded verbatim from `configs/project.yml` (`conclusion_boundary`) and reproduced identically in every project output.

APPENDIX A

Comprehensive Data Dictionary

Generated from the actual schemas of `data/processed/corridor_product_year_panel.parquet`, `corridor_features.parquet`, `data/outputs/top_ranked_corridors.csv` and `evidence_table.csv`, merged with the approved definitions in `reports/feature_explanation_table.md` and `reports/data_dictionary.md`. Where no approved definition exists the row is marked *schema-inferred* rather than given an invented definition.

A.1 Identity and provenance fields

Field	Dataset	Type	Unit	Definition
<code>obs_id</code>	panel, features, queue, evidence	string	—	Deterministic analytical key for one corridor–product–year observation
<code>source_row_id</code>	panel, features, queue, evidence	string	—	Provenance ID for the exact raw source row
<code>corridor_id</code>	panel, features	string	—	Directed exporter→importer route label
<code>year</code>	all	int64	calendar year	Calendar year of the annual aggregate
<code>hs6</code>	all	string(6)	—	Six-character HS17 product code
<code>family_id</code>	panel, features, queue	string	—	Product-family label
<code>exporter_iso3 / importer_iso3</code>	panel, features, queue	string(3)	—	ISO3 country codes
<code>source_version</code>	panel, features, queue, evidence	string	—	Pipeline source-version stamp
<code>source_file_hash</code>	panel	string	—	SHA-256 of the BACI extract

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
<code>obs_id</code>	SHA-256 prefix over (year, exporter_code, importer_code, hs6)	No	Identity	Join key	Yes	Stable across runs; not a business identifier
<code>source_row_id</code>	SHA-256 prefix over (year, hs6, exporter, importer, value, quantity)	No	Identity	No	Export	Changes if any source value changes
<code>corridor_id</code>	<code>exporter_iso3 + '-' + importer_iso3</code>	No	Identity	Grouping	Yes	A route, not a relationship or an entity
<code>year</code>	Source field <code>t</code>	No	Identity	Split key	Yes	Annual grain only
<code>hs6</code>	Source field <code>k</code> , zero-filled	No	Identity	Grouping	Yes	A product group, not a product
<code>family_id</code>	Joined from <code>configs/hs_families.yml</code>	No	Identity	Grouping	Yes	—
<code>exporter_iso3 / importer_iso3</code>	Mapped from BACI numeric codes	No	Identity	Grouping	Yes	Some codes are grouped reporters, e.g. <code>\$19</code>
<code>source_version</code>	Constant per run	No	Provenance	No	Export	—
<code>source_file_hash</code>	Computed at panel build	No	Provenance	No	No	Binds every row to its input file

A.2 Source and derived measurement fields

Field	Dataset	Type	Unit	Definition
<code>v</code>	raw, panel	float64	thousands of current USD	Published trade value
<code>q</code>	raw, panel	float64	metric tons	Published quantity
<code>trade_value_usd</code>	panel, features, queue	float64	USD	Nominal declared trade value

Field	Dataset	Type	Unit	Definition
quantity_metric_ton	panel, features, queue	float64	metric tons	Declared quantity
unit_value_usd_per_metric_ton	panel, features, queue	float64	USD/mt	Implied aggregate unit value
log_unit_value	panel, features	float64	log USD/mt	Natural log of the implied unit value
benchmark_price_usd_per_metric_ton	panel, features, queue	float64	USD/mt	World Bank annual benchmark, normalised
benchmark_price_original	panel	float64	USD/mt or USD/troy oz	Benchmark as published
benchmark_year	panel	int64	calendar year	Year the benchmark applies to

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
v	Source field	No	Current row	Input	No	Publisher unit
q	Source field	Yes	Current row	Input	No	Missing for 2,188 panel rows
trade_value_usd	$v \times 1000$	No	Current row	Yes	Yes	Nominal; not inflation-adjusted
quantity_metric_ton	q	Yes	Current row	Yes	Yes	Missing blocks unit-value scoring entirely
unit_value_usd_per_metric_ton	$\text{trade_value_usd} / \text{quantity_metric_ton}$	Yes	Current row	Yes	Yes	An annual average, not an invoice price
log_unit_value	$\log(\text{unit_value})$	Yes	Current row	Yes	No	Undefined where unit value is undefined
benchmark_price_usd_per_metric_ton	Published price; gold $\times 32,150.7466$	No	Same-year only	Yes	Yes	Macro market context, not invoice fair value
benchmark_price_original	Source workbook	No	Same-year only	No	No	Gold is per troy ounce before conversion
benchmark_year	Source workbook	No	Same-year only	No	No	Always equals <code>year</code> after the join

A.3 Time-safe feature fields

Feature	Definition
log_unit_value	Log of the implied unit value
shifted_corridor_history_median	Median of prior-year log unit value for the same route-product
shifted_corridor_history_mad	Median absolute deviation of prior-year log unit value
robust_historical_z	Distance from the route's own prior pattern
same_family_year_peer_percentile	Percentile of the benchmark residual among same-family, same-year peers
trade_value_yoy_change	Log change in declared trade value
quantity_yoy_change	Log change in declared quantity
unit_value_yoy_change	Log change in implied unit value
benchmark_residual	Log gap between the implied unit value and the same-year benchmark
benchmark_adjusted_drift	Change in the gap to the benchmark
corridor_activity_history	Count of prior active years for the route-product
corridor_novelty_flag	The route-product has no prior active year
corridor_reactivation_flag	The route-product returned after a gap
value_quantity_divergence	Value movement relative to quantity movement
missing_quantity_flag	Quantity is absent
missing_benchmark_flag	No benchmark matched
missing_history_flag	No prior-year baseline exists
missing_country_mapping_flag	A country code did not map to ISO3

Feature	Definition
valid_extreme_flag	Usable row far above or below its benchmark
benchmark_consistency_gap	How differently the route moved from its market
data_quality_score	Additive confidence score
benchmark_yoy_change	The benchmark's own annual log change

Continued — derivation, time-safety rule and coverage. Every row is a *float64* column of *corridor_features.parquet* and every one is a model input.

Feature	Derivation	Time-safety rule	Coverage
log_unit_value	$\log(\text{unit_value_usd_per_metric_ton})$	Uses only the current row's value and quantity	91.5%
shifted_corridor_history_median	Median over strictly prior years	Prior years only	69.1%
shifted_corridor_history_mad	MAD over strictly prior years	Prior years only	69.1%
robust_historical_z	$(\text{log_unit_value} - \text{median}) / \max(\text{MAD}, 0.05)$	Median and MAD are strictly prior	65.8%
same_family_year_peer_percentile	Average-method rank ÷ peer count	Same-year peers only	91.5%
trade_value_yoy_change	$\log(V / \text{previous } V)$	Previous observation only	72.0%
quantity_yoy_change	$\log(Q / \text{previous } Q)$	Previous observation only	64.8%
unit_value_yoy_change	$\log(U / \text{previous } U)$	Previous observation only	64.8%
benchmark_residual	$\text{log_unit_value} - \log(\text{benchmark})$	Same-year benchmark only	91.5%
benchmark_adjusted_drift	$\text{residual} - \text{previous residual}$	Previous residual only	64.8%
corridor_activity_history	Within-group cumulative count	Prior years only	100.0%
corridor_novelty_flag	1 when activity history is 0	Prior activity only	100.0%
corridor_reactivation_flag	1 when prior activity exists and the gap exceeds one year	Prior activity pattern only	100.0%
value_quantity_divergence	$\text{trade_value_yoy_change} - \text{quantity_yoy_change}$	Previous observation only	64.8%
missing_quantity_flag	Binary indicator	Current row	100.0%
missing_benchmark_flag	Binary indicator	Current row	100.0%
missing_history_flag	Binary indicator	Prior years	100.0%
missing_country_mapping_flag	Binary indicator	Current row	100.0%
valid_extreme_flag	Ratio test	Same-year benchmark	100.0%
benchmark_consistency_gap	$ \text{unit_value_yoy_change} - \text{benchmark_yoy_change} $	Current-year benchmark change only	64.8%
data_quality_score	Four components, 0–6	Prior years where relevant	100.0%
benchmark_yoy_change	$\log(B_y / B_{y-1})$	Computed at benchmark extraction	89.2%

A.4 Data-quality and eligibility fields

Field	Dataset	Type	Unit	Definition
value_valid_flag	panel	bool	—	Declared value present and > 0
quantity_valid_flag	panel	bool	—	Quantity present and > 0
unit_value_valid_flag	panel	bool	—	Both value and quantity valid
valid_extreme_flag	panel, features, queue	bool	—	Usable row more than 5× or under 0.2× the benchmark
benchmark_join_status	panel	string	—	Whether a benchmark matched
history_years_available	panel	int64	count	Prior observed years for the route-product
quantity_score / benchmark_score / history_score / completeness_score	panel	int64	points	The four data-quality components

Field	Dataset	Type	Unit	Definition
data_quality_score	panel, features, queue	int64	0–6 additive	Confidence in the observation
quality_status	panel, features, queue	string	—	Assigned usability status
exclusion_reason	panel, audit	string	—	Why a row is caveated or excluded
model_eligible	panel, features	bool	—	May this row be scored?
missing_quantity_flag / missing_benchmark_flag / missing_history_flag / missing_country_mapping_flag	features	int64	binary	Explicit missingness indicators
data_quality_flags	features	string	—	Human-readable missingness summary

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
value_valid_flag	Computed	No	Current row	Eligibility	No	—
quantity_valid_flag	Computed	No	Current row	Eligibility	Yes	The single gate on unit-value scoring
unit_value_valid_flag	value_valid_flag & quantity_valid_flag	No	Current row	Eligibility	No	—
valid_extreme_flag	Computed from the ratio	No	Same-year benchmark	Rules component	Yes	Retained with a caveat — never discarded
benchmark_join_status	Computed at the join	No	Same-year only	Flag input	No	100% matched in this run
history_years_available	Within-group cumulative count	No	Prior years only	Quality score	No	—
quantity_score / benchmark_score / history_score / completeness_score	Computed	No	Prior years where relevant	Quality score	No	0–1, 0 or 2, 0–2, 0–1
data_quality_score	Sum of the four components	No	Prior years where relevant	Rules deduction	Yes	Confidence, never anomaly; can only reduce a rules score
quality_status	Decision sequence in build_panel()	No	Current row	Eligibility	Yes	—
exclusion_reason	Decision sequence	No	Current row	No	No	Semicolon-separated when multiple
model_eligible	unit_value_valid_flag & status usable	No	Current row	Gate	Yes	—
missing_quantity_flag / missing_benchmark_flag / missing_history_flag / missing_country_mapping_flag	Computed	No	Current row	Model input	No	Data conditions, never signals of behaviour
data_quality_flags	Concatenated flags	No	Current row	No	Yes	—

A.5 Score and queue fields

Field	Dataset	Type	Unit	Definition
rank	queue	int64	position	Queue position, 1 = highest score
rule_score	queue, model scores	float64	0–1	Transparent weighted-sum score
selected_challenger_score	queue	float64	0–1	Selected challenger output
selected_review_priority_score	queue	float64	0–1	The final ranking score
logistic_score / xgboost_score / isolation_score / hybrid_score	model scores	float64	0–1	Per-method scenario scores
key_evidence_count	queue	int64	count	Evidence rows attached to the case

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
rank	Assigned after sorting	No	—	Output	Yes	Travels with its row when re-sorted
rule_score	Nine bounded components minus two deductions	No	Time-safe inputs	Blend input	Yes	An engineering ranking score, not a probability
selected_challenger_score	XGBoost predict_proba	No	Time-safe inputs	Blend input	Yes	Not a calibrated probability
selected_review_priority_score	$0.25 \times \text{rules} + 0.75 \times \text{challenger}$	No	Time-safe inputs	Ranking	Yes	A ranking signal, not a probability of crime
logistic_score / xgboost_score / isolation_score / hybrid_score	Fitted models	No	Time-safe inputs	Evaluation	Model page	Scenario evaluation only
key_evidence_count	Back-filled by src/07	No	—	Output	Yes	Uniformly 4 in this run — carries no information

A.6 Evidence fields

Field	Dataset	Type	Unit	Definition
evidence_id	evidence	string	—	Stable evidence-row identifier
evidence_type	evidence	string	—	Category of comparison
metric_name	evidence	string	—	The feature that fired
observed_value	evidence	float64	feature-specific	The feature's value for the case
comparison_value	evidence	float64	feature-specific	The value compared against
comparison_group	evidence	string	—	What the value was measured against
threshold	evidence	float64	feature-specific	The notable-magnitude threshold
direction	evidence	string	—	absolute / high / high_or_low
severity	evidence	string	—	high / medium / low
plain_english_summary	evidence	string	—	Fixed-wording description
caveat	evidence	string	—	Mandatory caveat

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
evidence_id	SHA-256 prefix over (obs_id, metric, value)	No	—	No	Yes	Same inputs always produce the same ID
evidence_type	From METRIC_SPECS	No	—	No	Yes	Five types in this run
metric_name	From METRIC_SPECS	No	—	No	Yes	—
observed_value	Read from the feature table	No	Time-safe	No	Yes	Recomputable without the model
comparison_value	Named comparison field	Yes	Time-safe	No	Yes	Absent where the spec names no comparison field
comparison_group	From METRIC_SPECS	No	—	No	Yes	—
threshold	From METRIC_SPECS	No	—	No	Yes	Fixed configuration, not learned
direction	From METRIC_SPECS	No	—	No	No	—
severity	Magnitude ÷ threshold banding	No	—	No	Yes	Not a risk rating
plain_english_summary	Per-metric template	No	—	No	Yes	No free-form generation
caveat	From METRIC_SPECS	No	—	No	Yes	No evidence row exists without one

A.7 Scenario-evaluation fields

CAUTION

The fields below exist **only** in the scenario-evaluation artefacts. They are never present in the clean panel, the feature table, the review queue, the evidence table, or the dashboard. Attributing any of them to the source trade data would be incorrect.

Field	Dataset	Type	Unit	Definition
synthetic_review_priority	scenario_labels.parquet	int64	binary	The evaluation target
scenario_id	scenario_labels, scenario_injections	string	—	Which transformation was applied
hard_negative	scenario_labels	bool	—	Injected but deliberately benign
split	scenario_labels	string	—	train / validation / test
trade_value_usd_before / after, quantity_metric_ton_before / after	scenario_injections.csv	float64	USD, metric tons	Before-and-after audit of every injected row
clean_panel_hash	scenario_injections, split manifest	string	—	Hash of the panel injected into

Continued — how each field behaves.

Field	Derivation	Nullable	Time safety	Model use	Dashboard use	Caveat
synthetic_review_priority	Assigned at injection	No	—	Label	Never	An evaluation device, not a criminal label
scenario_id	Assigned at injection	Yes	—	Audit	Never	Null for non-injected rows
hard_negative	Assigned at injection	No	—	Evaluation	Never	Tests over-alerting
split	Mapped from year	No	Temporal	Evaluation	Never	Disjoint by assertion
trade_value_usd_before / after, quantity_metric_ton_before / after	Recorded at injection	No	—	Audit	Never	Full reproducibility record
clean_panel_hash	SHA-256 of the frozen panel	No	—	Provenance	Never	Proves the panel predates the scenarios

APPENDIX B

Data Sources and Download Links

Table B.1. Official sources — publisher, version and coverage.

Source	Publisher	Version / release	Years used
BACI — International Trade Database at the Product Level	CEPII	Release 202601, HS17, released 22 January 2026	2017–2024
Commodity Markets Outlook — “Pink Sheet” annual prices	World Bank	CMO-Historical-Data-Annual.xlsx, sheet <i>Annual Prices (Nominal)</i>	2017–2024
Trade-Based Money Laundering: Trends and Developments	FATF and Egmont Group	Published December 2020, 66 pages	n/a — reference document
Trade-Based Money Laundering: Risk Indicators	FATF and Egmont Group	Published March 2021, 10 pages	n/a — reference document
BACI country codes	CEPII	V202601	n/a — reference data
HS17 product codes	CEPII	HS17 V202601	n/a — reference data

CEPII is the Centre d'Études Prospectives et d'Informations Internationales.

Table B.2. Official sources — local filename, access date and download link.

Source	Local filename	Access date	Official link
CEPII BACI	data/raw/ baci_hs17_v202601_selected_2017_2024_151110_740311_710812. parquet	2026-06-28 (extract created 2026-06-28)	cepii.fr
World Bank CMO	data/raw/CMO-Historical-Data-Annual.xlsx	2026-06-28	worldbank.org
FATF–Egmont 2020	data/raw/Trade-Based-Money-Laundering-Trends-and- Developments_2020.pdf	2026-06-28	fatf-gafi.org
FATF–Egmont 2021	data/raw/Trade-Based-Money-Laundering-Risk- Indicators_2021.pdf	2026-06-28	fatf-gafi.org
BACI country codes	data/raw/country_codes_V202601.csv	2026-06-28	cepii.fr
HS17 product codes	data/raw/product_codes_HS17_V202601.csv	2026-06-28	cepii.fr

Full URLs — CEPII BACI: https://www.cepii.fr/CEPII/en/bdd_modele/bdd_modele_item.asp?id=37 · World Bank Commodity Markets: <https://www.worldbank.org/en/research/commodity-markets> · FATF–Egmont 2020: <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Trade-based-money-laundering-trends-and-developments.html> · FATF–Egmont 2021: <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Trade-based-money-laundering-indicators.html>

URL origin. The CEPII BACI URL is recorded in project metadata (`data/raw/data_source_notes.json`). The World Bank and FATF–Egmont URLs are **not** in project metadata; they are maintained in `dashboard/services/source_registry.py` and were verified against the official sites on 2026-07-14. Source URLs document dataset provenance only; they are never row-level evidence.

Table B.3. SHA-256 inventory of all declared inputs.

File	Size (bytes)	SHA-256 (first 24)
baci_hs17_v202601_selected_2017_2024_151110_740311_710812.parquet	340,203	0e493e057c8fd57902328551...
data_source_notes.json	4,342	a8a973d0797cbfcd4064b1df...
CMO-Historical-Data-Annual.xlsx	3,177,955	9fbc348f40ecdb02eb1bcf8...
Trade-Based-Money-Laundering-Trends-and-Developments_2020.pdf	3,108,080	0969e49877a1937a7bc49ac0...
Trade-Based-Money-Laundering-Risk-Indicators_2021.pdf	2,079,148	670b80270b33354b17263d44...
country_codes_V202601.csv	5,346	c6892f28bea23d689b29b074...
product_codes_HS17_V202601.csv	598,831	4abb74d330c3a1969ccfc465...
baci-readme.txt	571	156c8fe9d8582f24fdec5c43...

APPENDIX C

GitHub Repository and Deployed Application

Table C.1. Repository and deployment reference.

Item	Value
GitHub repository	https://github.com/jessybaolin/evidence-based-tbml-corridor-drift
Git remote	https://github.com/jessybaolin/evidence-based-tbml-corridor-drift.git
Default branch	main
Branch at time of writing	develop (commit 50c4096)
Deployed application	https://tbml-review-triage.streamlit.app/
Application entry point	dashboard/streamlit_app.py
Deployed runtime dependencies	requirements.txt — 6 pinned packages
Pipeline dependencies	requirements-pipeline.txt — 24 packages
Streamlit configuration	.streamlit/config.toml — base light, primary #0F8F8F, text #22314A, headless
Dashboard design tokens	dashboard/config/dashboard_theme.yml
Development container	.devcontainer/devcontainer.json — mcr.microsoft.com/devcontainers/python:1-3.11-bookworm, forwards port 8501
Continuous integration	None — no .github/workflows/ directory exists
Secrets	None — the application reads no credential and calls no external service
Deployment branch binding	Not recorded in the repository (held in the Streamlit Community Cloud account)

C.1 Repository structure

```
evidence-based-tbml-corridor-drift/
├── configs/                6 YAML files – scope, families, benchmarks, thresholds,
│                           scenarios, typology cards
├── data/
│   ├── raw/                8 declared inputs (hash-pinned)
│   ├── interim/            staged Parquet copies
│   ├── processed/          panel, features, scenarios, audit
│   └── outputs/            model, queue, evidence, briefs, verification
├── src/                    11 modules – the analytical pipeline
├── dashboard/
│   ├── streamlit_app.py    application entry point
│   ├── app_pages/          10 page modules (9 content pages + welcome)
│   ├── components/        17 shared UI modules
│   ├── services/           11 data and analytics services
│   └── config/             dashboard_content.yml · dashboard_theme.yml
├── tests/                  12 test modules
├── reports/                generated reports, figures and field documentation
├── prompts/                brief prompt assets (documentation only; no model is called)
├── .streamlit/config.toml  Streamlit base theme and server settings
├── .devcontainer/          development-container definition
├── requirements.txt         pinned deployed-dashboard runtime
├── requirements-pipeline.txt full pipeline and notebook environment
└── README.md
```

C.2 Deployment notes

- The dashboard is a **read-only reader of generated artefacts**. The outputs under `data/processed/` and `data/outputs/` must be committed for the deployed application to have anything to render; nothing is regenerated in the cloud.
- Streamlit Community Cloud installs `requirements.txt` only — six pinned packages, none of them machine-learning libraries.
- An accepted push to the connected branch triggers an automatic rebuild and redeploy.
- Page modules live in `dashboard/app_pages/`, deliberately **not** `dashboard/pages/`, so Streamlit's legacy multi-page compatibility path is never triggered and the boundary ribbon renders on every route.

APPENDIX D

Formula Reference

A compact catalogue of every formula implemented in the project. Full definitions, derivations, time-safety rules and caveats are in Chapter 8.

D.1 Symbols

Table D.1. Symbol reference.

Symbol	Meaning	Symbol	Meaning
c	Corridor (exporter→importer)	$r_{c,p,y}$	Benchmark residual
p	HS6 product code	$\tilde{m}_{c,p,y}$	Shifted corridor historical median
f	Product family	$d_{c,p,y}$	Shifted corridor historical MAD
y	Calendar year	$z_{c,p,y}$	Robust historical z-score
$V_{c,p,y}$	Trade value, nominal USD	$\mathcal{H}_{c,p,y}$	Strictly prior observed years
$Q_{c,p,y}$	Quantity, metric tons	$\pi(c,p,y)$	Previous observed year
$U_{c,p,y}$	Implied unit value, USD/mt	$\mathcal{P}_{f,y}$	Same-family, same-year peer set
$\ell_{c,p,y}$	$\log U_{c,p,y}$	$\mathbb{1}[\cdot]$	Indicator function
$B_{f,y}$	Benchmark price, USD/mt	w	Hybrid challenger weight

D.2 Panel construction

$$V_{c,p,y} = v_{c,p,y} \times 1000 \quad (\text{D.1})$$

where: $V_{c,p,y}$ is the published trade value in thousands of current US dollars; $v_{c,p,y}$ is the nominal US-dollar value used throughout the project. · Implemented in `src/tbml_common.py` — `build_panel()`

$$U_{c,p,y} = \frac{v_{c,p,y}}{Q_{c,p,y}} \quad \text{defined only where } \text{valid}_{c,p,y}^U = 1 \quad (\text{D.2})$$

where: $U_{c,p,y}$ is the implied aggregate unit value in US dollars per metric ton. It is an annual average across every flow in the observation, not a transaction price. · Implemented in `src/tbml_common.py` — `build_panel()`

$$\ell_{c,p,y} = \log U_{c,p,y} \quad \text{undefined where } \text{valid}_{c,p,y}^U = 0 \quad (\text{D.3})$$

where: $\ell_{c,p,y}$ is the log implied unit value — the quantity on which every history and change feature is built. · Implemented in `src/tbml_common.py` — `build_panel()`, `build_features()`

$$B_y^{\text{gold}} [\text{USD/mt}] = B_y^{\text{gold}} [\text{USD/troy oz}] \times \frac{1,000,000}{31.1034768} \quad (\text{D.4})$$

where: 1 metric ton = 1,000,000 grams and 1 troy ounce = 31.1034768 grams, giving a conversion factor of 32,150.7466. · Implemented in `src/tbml_common.py` — `GOLD_TROY_OUNCE_TO_METRIC_TON_FACTOR`, `extract_worldbank_benchmarks()`

$$r_{c,p,y} = \log U_{c,p,y} - \log B_{f(p),y} = \log \left(\frac{U_{c,p,y}}{B_{f(p),y}} \right) \quad (D.5)$$

where: $B_{f(p),y}$ is the annual World Bank benchmark for the product family of p , normalised to US dollars per metric ton. Exponentiating gives the benchmark multiple. · Implemented in `src/tbml_common.py — build_panel()`

$$\text{extreme}_{c,p,y} = \text{valid}_{c,p,y}^U \cdot \mathbb{1} \left[\frac{U_{c,p,y}}{B_{f(p),y}} > 5 \vee \frac{U_{c,p,y}}{B_{f(p),y}} < 0.2 \right] \quad (D.6)$$

where: Marks a usable observation whose implied unit value is more than five times, or less than one-fifth of, the benchmark. Such rows are retained with a caveat, never discarded. · Implemented in `src/tbml_common.py — build_panel()`

$$DQ_{c,p,y} = \text{valid}_{c,p,y}^Q + 2 \cdot \mathbb{1}[\text{benchmark matched}] + h_{c,p,y} + \mathbb{1}[\text{keys complete}] \quad (D.7)$$

where: The four terms contribute 0–1, 0 or 2, 0–2 and 0–1 respectively. $h_{c,p,y} = 2$ when at least five prior years are available, 1 when at least three, and 0 otherwise. The total ranges from 0 to 6 and measures confidence, never anomaly. · Implemented in `src/tbml_common.py — build_panel()`

$$\text{eligible}_{c,p,y} = \text{valid}_{c,p,y}^U \cdot \mathbb{1}[\text{status} \in \{\text{fully_usable}, \text{usable_with_caveat}\}] \quad (D.8)$$

where: An observation may be scored only where an implied unit value exists and its quality status is usable, with or without caveats. · Implemented in `src/tbml_common.py — build_panel()`

D.3 Time-safe features

$$\tilde{m}_{c,p,y} = \text{median}\{ \ell_{c,p,y'} : y' \in \mathcal{H}_{c,p,y} \} \quad (D.9)$$

where: $\mathcal{H}_{c,p,y} = \{ y' < y : (c, p, y') \text{ observed with a valid unit value} \}$ — strictly prior years only. Undefined when the corridor–product has no prior observation. · Implemented in `src/tbml_common.py — _prior_median_mad()`

$$d_{c,p,y} = \text{median}\{ | \ell_{c,p,y'} - \tilde{m}_{c,p,y} | : y' \in \mathcal{H}_{c,p,y} \} \quad (D.10)$$

where: The median absolute deviation of the same strictly-prior values: a robust measure of how much this corridor–product used to vary, insensitive to a single past outlier. · Implemented in `src/tbml_common.py — _prior_median_mad()`

$$z_{c,p,y} = \frac{\ell_{c,p,y} - m_{c,p,y}}{\max(d_{c,p,y}, 0.05)} \quad \text{undefined where } \tilde{m}_{c,p,y} \text{ is undefined} \quad (D.11)$$

where: The floor of 0.05 replaces a historical MAD of essentially zero, preventing a degenerate denominator for a corridor whose past unit values were near-identical. The floor is applied in log space, so it corresponds to roughly 5% dispersion. · Implemented in `src/tbml_common.py — build_features()`

$$\rho_{c,p,y} = \frac{\text{rank}(\ell_{c,p,y} ; \ell_{f(p),y})}{|\mathcal{P}_{f(p),y}|} \quad (D.12)$$

where: $\mathcal{P}_{f,p,y}$ is the set of all observations in family f and year y with a defined benchmark residual; ranking uses the average method for ties. The result is a share in $[0, 1]$, where 0.5 is the middle of the same-year peer group. · Implemented in `src/tbml_common.py — build_features()`

$$\Delta \log X_{c,p,y} = \log \left(\frac{X_{c,p,y}}{X_{c,p,\pi}} \right) \quad \text{defined only where } X_{c,p,y} > 0 \wedge X_{c,p,\pi} > 0 \quad (D.13)$$

where: $\pi = \pi(c, p, y)$ is the previous **observed** year for this corridor–product — not necessarily $y-1$. A zero or missing endpoint yields an undefined value, never an infinity. · Implemented in `src/tbml_common.py — _log_change()`

$$\Delta V_{c,p,y} = \log \frac{V_{c,p,y}}{V_{c,p,\pi}} \quad \Delta Q_{c,p,y} = \log \frac{Q_{c,p,y}}{Q_{c,p,\pi}} \quad \Delta U_{c,p,y} = \log \frac{U_{c,p,y}}{U_{c,p,\pi}} \quad (D.14)$$

where: $\pi = \pi(c, p, y)$ throughout. All three are natural-log ratios against the previous observed year for the same corridor and product. · Implemented in `src/tbml_common.py — build_features()`

$$\delta_{c,p,y} = \Delta V_{c,p,y} - \Delta Q_{c,p,y} \quad (D.15)$$

where: Positive when declared value grew faster than physical quantity; negative when quantity grew faster than value. · Implemented in `src/tbml_common.py — build_features()`

$$\Delta r_{c,p,y} = r_{c,p,y} - r_{c,p,\pi(c,p,y)} \quad (D.16)$$

where: The change in the gap to the market, not the change in price. Near zero when a corridor's unit value moved because the whole market moved. · Implemented in `src/tbml_common.py — build_features()`

$$\Delta B_{f,y} = \log \left(\frac{B_{f,y}}{B_{f,y-1}} \right) \quad (D.17)$$

where: The benchmark's own year-over-year log change, computed once per family and year during benchmark extraction and joined onto every observation in that family-year. · Implemented in `src/tbml_common.py — extract_worldbank_benchmarks()`

$$g_{c,p,y} = |\Delta U_{c,p,y} - \Delta B_{f(p),y}| \quad (D.18)$$

where: How differently the corridor moved from its wider commodity market. Near zero means the movement is market-explained; large means it is not. · Implemented in `src/tbml_common.py — build_features()`

$$a_{c,p,y} = |\{y' < y : (c, p, y') \text{ observed}\}| \quad (D.19)$$

where: A count of prior appearances of this corridor–product in the panel, computed as the within-group cumulative count over rows sorted by year. · Implemented in `src/tbml_common.py — build_features()`

$$\begin{aligned} \text{novel}_{c,p,y} &= \mathbb{1}[a_{c,p,y} = 0] \\ \text{react}_{c,p,y} &= \mathbb{1}[a_{c,p,y} > 0 \wedge y - \pi(c, p, y) > 1] \end{aligned} \quad (D.20)$$

where: $\pi(c, p, y)$ is the most recent observed year before y for this corridor–product. Novelty marks a first appearance; reactivation marks a return after a gap. · Implemented in `src/tbml_common.py — build_features()`

$$\begin{aligned} \text{miss}^Q &= \mathbb{1}[Q \text{ absent}] & \text{miss}^B &= \mathbb{1}[\text{benchmark unmatched}] \\ \text{miss}^H &= \mathbb{1}[\tilde{m} \text{ undefined}] & \text{miss}^C &= \mathbb{1}[\text{ISO3 unmapped}] \end{aligned} \quad (D.21)$$

where: Four explicit binary indicators of what is absent. They record data conditions and are never interpreted as signals of behaviour. · Implemented in `src/tbml_common.py — build_features()`

D.4 Scoring

$$\text{bnd}(x; \varrho) = \min\left(\frac{|x|}{\varrho}, 1\right) \quad \text{bnd}(\text{undefined}; \varrho) = 0 \quad (\text{D.22})$$

where: ϱ is the component's configured reference value — the magnitude at which the component saturates. An undefined feature contributes nothing rather than penalising the row. · Implemented in `src/06_train_evaluate_models.py` — `bounded_abs()`

$$s^{\text{peer}} = \min(2|\rho_{c,p,y} - 0.5|, 1) \quad (\text{D.23})$$

where: Distance from the middle of the peer distribution, scaled to $[0, 1]$, so that both the high and the low tail contribute. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`

$$\begin{aligned} S^+ = & 0.20 \text{ bnd}(z; 4.0) + 0.18 \text{ bnd}(r; 1.0) + 0.18 \text{ bnd}(\Delta r; 0.70) \\ & + 0.12 \text{ bnd}(\Delta U; 0.80) + 0.12 \text{ bnd}(\delta; 0.80) + 0.10 s^{\text{peer}} \\ & + 0.06 \text{ novel} + 0.08 \text{ react} + 0.14 \text{ extreme} \end{aligned} \quad (\text{D.24})$$

where: Nine bounded components with fixed weights from `configs/thresholds.yml`. The weights sum to 1.18, so S^+ is a weighted sum rather than a weighted average and can exceed 1 before the deductions are applied. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

$$C = 0.15 \cdot (1 - \text{bnd}(g_{c,p,y}; 0.35)) \quad (\text{D.25})$$

where: A market-explanation deduction. A corridor that moved exactly with its benchmark loses the full 0.15; the deduction decays to zero at a movement gap of 0.35. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

$$P = 0.08 \cdot \frac{6 - \min(DQ, 6)}{6} \quad (\text{D.26})$$

where: A data-reliability deduction. A row scoring 6 of 6 loses nothing; each point below 6 subtracts approximately 0.0133, to a maximum of 0.08. An undefined score is treated as 0. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`; `configs/thresholds.yml`

$$s_{c,p,y}^{\text{rule}} = \min(\max(S^+ - C - P, 0), 1) \quad (\text{D.27})$$

where: The transparent baseline score, clipped into $[0, 1]$. It is an engineering ranking score with no external or probabilistic meaning. · Implemented in `src/06_train_evaluate_models.py` — `score_rules()`

$$s_{c,p,y}^{\text{hyb}} = (1 - w) \cdot s_{c,p,y}^{\text{rule}} + w \cdot s_{c,p,y}^{\text{chal}}, \quad w = 0.75 \quad (\text{D.28})$$

where: s^{chal} is the selected challenger's output and w is the blend weight chosen on the validation split only. Stored as `selected_review_priority_score` for real observations. · Implemented in `src/06_train_evaluate_models.py` — `main()`; `data/outputs/model_selection.json`

D.5 Evaluation and evidence

$$\text{Precision@k} = \frac{|\{\text{positives in top } k\}|}{k} \quad \text{Recall@k} = \frac{|\{\text{positives in top } k\}|}{|\{\text{positives in split}\}|} \quad (\text{D.29})$$

where: Rows are ranked by score descending with obs_id ascending as a deterministic tie-breaker; k is capped at the number of rows available. Recall@ k is the true-positive rate at that cut-off. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

$$\text{Lift@k} = \frac{\text{Precision@k}}{\omega_{\text{split}}}, \quad \omega_{\text{split}} = \frac{|\{\text{positives in split}\}|}{n_{\text{split}}} \quad (\text{D.30})$$

where: ω_{split} is the injected-positive prevalence in the split. A lift of 1 would match random review. This is a ranking-concentration measure, not an improvement in real-world detection. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

$$\begin{aligned} \text{HN-FPR@k} &= \frac{|\{\text{hard negatives in top } k\}|}{|\{\text{hard negatives in split}\}|} \\ \text{Ord-FPR@k} &= \frac{|\{\text{ordinary negatives in top } k\}|}{|\{\text{ordinary negatives in split}\}|} \end{aligned} \quad (\text{D.31})$$

where: A hard negative is an injected but deliberately benign row; an ordinary negative is any other non-positive row. Both use the score at rank k as the threshold. Lower is better. · Implemented in `src/06_train_evaluate_models.py` — `metric_at_k()`

$$\begin{aligned} M(m, x) &= 2|x - 0.5| & \text{for } m = \text{same_family_year_peer_percentile} \\ M(m, x) &= |x| & \text{otherwise} \end{aligned} \quad (\text{D.32})$$

where: The non-negative strength of metric m at observed value x . The peer percentile is two-sided, so both a very high and a very low position count as notable. · Implemented in `src/07_build_evidence.py` — `metric_magnitude()`

$$\begin{aligned} \text{sev}(M, \tau) &= \text{high} & \text{when } M/\tau \geq 2 \\ \text{sev}(M, \tau) &= \text{medium} & \text{when } 1 \leq M/\tau < 2 \\ \text{sev}(M, \tau) &= \text{low} & \text{when } M/\tau < 1 \end{aligned} \quad (\text{D.33})$$

where: τ is the metric's configured threshold. The four metrics with the largest M/τ are kept for each queued observation. · Implemented in `src/07_build_evidence.py` — `severity()`, `build_evidence()`

$$\text{gap}_{c, p, y} = \text{valid}_{c, p, y}^V \cdot (1 - \text{valid}_{c, p, y}^Q) \quad (\text{D.34})$$

where: A row with a valid declared value but no usable quantity. No implied unit value exists, so no unit-value score can be computed. It is a coverage limitation, never a signal. · Implemented in `dashboard/services/gold_coverage.py` — `with_gap_flag()`; `src/tbml_common.py` — `build_panel()`

APPENDIX E

Essential Functions

Excerpts from the current implementation, limited to the functions that carry the project's central guarantees. Full source is in the repository.

E.1 Time-safe history construction

`src/tbml_common.py`. This is the single function that makes the historical baseline time-safe: the baseline for each position is recorded **before** the current value is appended to the history, so a row can never see itself or any later year.

```
def _prior_median_mad(values: pd.Series) -> tuple[np.ndarray, np.ndarray]:
    # For each position, compute the median + MAD of ALL PRIOR values only (the current value is
    # appended AFTER its baseline is recorded). This is what makes "shifted history" time-safe:
    # a row can never see its own or any future value when forming its baseline.
    medians = np.full(len(values), np.nan)
    mads = np.full(len(values), np.nan)
    history: list[float] = []
    for idx, value in enumerate(values.to_numpy(dtype=float, na_value=np.nan)):
        if history:
            arr = np.asarray(history, dtype=float)
            med = float(np.median(arr))
            medians[idx] = med
            mads[idx] = float(np.median(np.abs(arr - med))) # median absolute deviation
        if np.isfinite(value):
            history.append(float(value))
    return medians, mads
```

Source: `src/tbml_common.py` — `_prior_median_mad()`

E.2 Time-safe change computation

`src/tbml_common.py`. Year-over-year log change, defined only where both endpoints are strictly positive — so a zero or missing endpoint yields `NaN` rather than an infinity.

```
def _log_change(current: pd.Series, previous: pd.Series) -> pd.Series:
    # Year-over-year log ratio, defined only where both current and previous are positive.
    out = pd.Series(np.nan, index=current.index, dtype=float)
    valid = current.gt(0) & previous.gt(0)
    out.loc[valid] = np.log(current.loc[valid] / previous.loc[valid])
    return out
```

Source: `src/tbml_common.py` — `_log_change()`

E.3 Scenario separation and label isolation

`src/tbml_common.py`, inside `inject_scenarios()`. The clean panel is hashed before injection, the injection refuses to run on a panel that already carries label metadata, and the labels are built into a **separate table** that is written to its own file.

```

clean_hash = stable_dataframe_hash(panel) # freeze hash: proves which panel we injected into
scenario_panel = panel.copy().sort_values("obs_id", kind="mergesort").reset_index(drop=True)
# The incoming clean panel must not already contain label columns.
if {"scenario_id", "synthetic_review_priority"}.intersection(scenario_panel.columns):
    raise ValueError("Scenario metadata must not exist in the clean panel")
rng = np.random.default_rng(seed) # seeded RNG => reproducible injections

# ...transformations applied to scenario_panel, labels recorded separately in label_map...

# ---- Build the separate labels table (one row per obs; non-injected rows default to 0) ----
labels = scenario_panel[["obs_id", "year", "family_id"]].copy()
labels["split"] = labels["year"].map(lambda y: split_for_year(int(y)))
labels["synthetic_review_priority"] = labels["obs_id"].map(
    lambda oid: label_map.get(str(oid), {}).get("synthetic_review_priority", 0)).astype(int)
labels["hard_negative"] = labels["obs_id"].map(
    lambda oid: bool(label_map.get(str(oid), {}).get("hard_negative", False))).astype(bool)

# ---- Guard: train/validation/test must be disjoint sets of observations ----
split_sets = {name: set(labels.loc[labels["split"] == name, "obs_id"])
               for name in ["train", "validation", "test"]}
if (split_sets["train"] & split_sets["validation"] or split_sets["train"] & split_sets["test"]
    or split_sets["validation"] & split_sets["test"]):
    raise AssertionError("Temporal splits overlap")

```

Source: src/tbml_common.py — inject_scenarios()

E.4 The feature-frame leakage guard

src/tbml_common.py, at the end of build_features(). Any label or outcome column reaching the feature frame raises rather than propagating.

```

# ---- Safety guards: no infinities, and NO labels/scenario/crime columns may leak in ----
numeric_cols = feature_columns()
if np.isinf(features[numeric_cols].to_numpy(dtype=float, na_value=np.nan)).any():
    raise ValueError("Feature construction produced infinite values")
forbidden = {"synthetic_review_priority", "scenario_id", "money_laundering",
             "fraud", "criminal", "tbml_confirmed"}
present = forbidden.intersection(features.columns)
if present:
    raise ValueError(f"Forbidden fields entered features: {sorted(present)}")

```

Source: src/tbml_common.py — build_features()

E.5 Model selection on validation only

src/06_train_evaluate_models.py. The challenger is chosen on the configured validation metric with the hard-negative false-positive rate as a secondary guardrail; the blend weight is then chosen on validation precision at k , with deterministic tie-breaks.

```
# ---- Pick the supervised challenger on VALIDATION only ----
validation_all = initial_metrics[(initial_metrics["split"] == "validation")
                                & (initial_metrics["family_id"] == "all")]
supervised = validation_all[validation_all["model"].isin(["logistic", "xgboost"])].sort_values(
    [challenger_metric, "hard_negative_false_positive_rate"],
    ascending=[False, True], kind="mergesort")
selected_challenger = str(supervised.iloc[0]["model"])
selected_col = f"{selected_challenger}_score"

# ---- Pick the hybrid blend weight on VALIDATION only ----
candidates = []
for w in selection_settings["hybrid_weights"]:
    col = f"hybrid_candidate_{w:.2f}"
    scored[col] = (1 - float(w)) * scored["rule_score"] + float(w) * scored[selected_col]
    metrics = metric_at_k(scored[scored["split"] == "validation"], col, top_k)
    candidates.append({"challenger_weight": float(w), **metrics})

# The operational queue has a fixed review capacity, so the blend is chosen on validation
# precision@k. Average precision remains a deterministic tie-breaker, followed by the more
# rules-heavy blend.
hybrid_table = pd.DataFrame(candidates).sort_values(hybrid_sort,
                                                    ascending=hybrid_ascending, kind="mergesort")
selected_weight = float(hybrid_table.iloc[0]["challenger_weight"])
```

Source: src/06_train_evaluate_models.py — main()

E.6 Evidence construction

src/07_build_evidence.py. Evidence is rebuilt from the feature table and ranked by how far each metric exceeds its own threshold — it is never decomposed from a score.

```
# ---- For each observation, pick its strongest metrics and emit evidence rows ----
for _, obs in source.iterrows():
    # Score each available metric by how far it exceeds its threshold.
    candidates = []
    for metric_name, spec in METRIC_SPECS.items():
        if metric_name not in obs or pd.isna(obs[metric_name]):
            continue
        observed = float(obs[metric_name])
        threshold = float(spec["threshold"])
        magnitude = metric_magnitude(metric_name, observed)
        candidates.append((magnitude / max(threshold, 1e-9), metric_name))
    # Keep only the top 4 strongest metrics for this observation (ties broken by name).
    selected = sorted(candidates, key=lambda item: (-item[0], item[1]))[:4]

    # Build one fully-described evidence row per selected metric.
    for _, metric_name in selected:
        spec = METRIC_SPECS[metric_name]
        observed = float(obs[metric_name])
        comparison_field = spec["comparison_field"]
        comparison = None
        if comparison_field and comparison_field in obs and pd.notna(obs[comparison_field]):
            comparison = float(obs[comparison_field])
        mag = metric_magnitude(metric_name, observed)
        sev = severity(mag, float(spec["threshold"]))
        eid = evidence_id(str(obs["obs_id"]), metric_name, observed)
```

Source: src/07_build_evidence.py — build_evidence()

E.7 Validation boundaries — the ranking metric

`src/06_train_evaluate_models.py`. Precision, recall and lift at k , plus the two false-positive rates, with `obs_id` as a deterministic tie-breaker.

```
# obs_id is only a tie-breaker. If two rows have the same score, sort them consistently by ID.
ranked = frame.sort_values([score_col, "obs_id"], ascending=[False, True], kind="mergesort")
k_eff = min(k, len(ranked))
top = ranked.head(k_eff)

positives = int(frame["synthetic_review_priority"].sum())
positives_top = int(top["synthetic_review_priority"].sum())

precision = positives_top / k_eff if k_eff else 0.0
recall = positives_top / positives if positives else 0.0

prevalence = positives / len(frame) if len(frame) else 0.0
lift = precision / prevalence if prevalence > 0 else 0.0 # how much better than random

hard_total = int(frame["hard_negative"].sum())
hard_top = int(top["hard_negative"].sum())
ordinary_total = int(((frame["synthetic_review_priority"] == 0) & (~frame["hard_negative"]))).sum()
ordinary_top = int(((top["synthetic_review_priority"] == 0) & (~top["hard_negative"]))).sum()
try:
    ap = float(average_precision_score(frame["synthetic_review_priority"], frame[score_col]))
except Exception:
    ap = float("nan") # average precision is undefined if a split has only one class
```

Source: `src/06_train_evaluate_models.py` — `metric_at_k()`

APPENDIX F

Artefact and File Inventory

F.1 Analytical pipeline

Table F.1. `src/` — 11 modules, 2,681 lines.

Module	Lines	Purpose
<code>src/00_check_environment.py</code>	68	Environment validation and version record
<code>src/01_verify_sources.py</code>	125	Chain of custody: hashes, cross-checks, PDF readability
<code>src/02_ingest_official_sources.py</code>	51	Raw → interim staging, no derived fields
<code>src/03_build_panel.py</code>	43	Canonical panel and exclusion audit
<code>src/04_build_features.py</code>	82	Time-safe features and the feature dictionary
<code>src/05_build_scenarios.py</code>	62	Seeded scenario injection with separated labels
<code>src/06_train_evaluate_models.py</code>	423	Fit, evaluate, select, score, and cut the Top 50
<code>src/07_build_evidence.py</code>	240	Recomputable evidence rows
<code>src/08_build_briefs.py</code>	256	Deterministic caveated briefs and their validation
<code>src/09_generate_reports.py</code>	464	Diagrams and written report deliverables
<code>src/tbml_common.py</code>	867	Shared library: panel, features, scenarios, benchmarks, boundary

F.2 Configuration

Table F.2. `configs/` — all analytical parameters.

File	Purpose
<code>project.yml</code>	Scope, years, seeds, split design, <code>top_k</code> , brief count, the conclusion boundary
<code>hs_families.yml</code>	The three HS6 families, product names, roles, expected units, approved caveats
<code>benchmark_series.yml</code>	Benchmark series mapping, units, conversion methods, caveats
<code>thresholds.yml</code>	Rule weights and reference values, model-selection settings, evidence limits
<code>scenario_catalog.yml</code>	The five scenario definitions and their targets
<code>typology_context.yml</code>	5 FATF–Egmont typology cards with allowed and prohibited language

F.3 Data

Table F.3. `data/raw/` — 8 declared inputs.

File	Size (bytes)	SHA-256 (first 24)
<code>baci_hs17_v202601_selected_2017_2024_151110_740311_710812.parquet</code>	340,203	0e493e057c8fd57902328551...
<code>data_source_notes.json</code>	4,342	a8a973d0797cbfcd4064b1df...
<code>CMO-Historical-Data-Annual.xlsx</code>	3,177,955	9fbc348f40ecdb02eb1bcf8...
<code>Trade-Based-Money-Laundering-Trends-and-Developments_2020.pdf</code>	3,108,080	0969e49877a1937a7bc49ac0...
<code>Trade-Based-Money-Laundering-Risk-Indicators_2021.pdf</code>	2,079,148	670b80270b33354b17263d44...
<code>country_codes_V202601.csv</code>	5,346	c6892f28bea23d689b29b074...
<code>product_codes_HS17_V202601.csv</code>	598,831	4abb74d330c3a1969ccfc465...
<code>baci-readme.txt</code>	571	156c8fe9d8582f24fdec5c43...

Table F.4. data/processed/ — analytical tables.

File	Size (KB)
data/processed/corridor_features.csv	16,054.7
data/processed/corridor_features.parquet	4,033.2
data/processed/corridor_product_year_panel.csv	20,272.4
data/processed/corridor_product_year_panel.parquet	2,825.8
data/processed/exclusion_audit.csv	3,681.1
data/processed/panel.csv	20,272.4
data/processed/scenario_features.parquet	4,037.3
data/processed/scenario_injections.csv	115.7
data/processed/scenario_labels.parquet	576.4
data/processed/scenario_panel.parquet	2,916.9
data/processed/scenario_split_manifest.json	0.9

Table F.5. data/outputs/ — model, queue, evidence and verification artefacts.

File	Size (KB)
data/outputs/analyst_briefs.json	12.6
data/outputs/brief_validation_report.csv	0.5
data/outputs/diagram_verification.json	3.2
data/outputs/environment_check.json	1.9
data/outputs/evidence_table.csv	114.2
data/outputs/hybrid_validation_candidates.csv	0.5
data/outputs/logistic_coefficients.csv	1.5
data/outputs/model_bundle.joblib	1,755.5
data/outputs/model_comparison.csv	5.4
data/outputs/model_scores.csv	8,251.1
data/outputs/model_selection.json	0.3
data/outputs/report_manifest.json	0.9
data/outputs/scenario_model_evaluation.csv	5.4
data/outputs/shap_summary_values.csv	0.5
data/outputs/source_file_inventory.csv	1.3
data/outputs/source_manifest.json	10.6
data/outputs/top_50_review_queue.png	753.0
data/outputs/top_50_review_queue_hero.png	196.7
data/outputs/top_ranked_corridors.csv	13.9
data/outputs/xgboost_parameters.json	0.7

F.4 Dashboard

Table F.6. `dashboard/app_pages/` — page modules.

Module	Lines	Page
<code>dashboard/app_pages/appendix.py</code>	107	Data Dictionary
<code>dashboard/app_pages/bank_implementation_pathway.py</code>	431	Bank Implementation Pathway
<code>dashboard/app_pages/case_investigation.py</code>	536	Selected Case Review
<code>dashboard/app_pages/documentation.py</code>	50	—
<code>dashboard/app_pages/executive_overview.py</code>	255	Business Problem and Value
<code>dashboard/app_pages/from_data_to_review_queue.py</code>	576	From Data to Review Queue
<code>dashboard/app_pages/gold_quantity_coverage.py</code>	333	Unscored Gold Records
<code>dashboard/app_pages/landing.py</code>	12	Full-screen welcome route (default)
<code>dashboard/app_pages/model_and_controls.py</code>	581	Model Evaluation & Controls
<code>dashboard/app_pages/portfolio_analytics.py</code>	208	Trade Landscape and Patterns
<code>dashboard/app_pages/review_queue.py</code>	213	Top 50 Review Queue

Table F.7. `dashboard/services/` — data and analytics services.

Module	Lines	Responsibility
<code>dashboard/services/case_summary.py</code>	322	Case comparison views, multiples and peer position
<code>dashboard/services/dashboard_metrics.py</code>	552	KPIs, queue enrichment, filters, landscape and evaluation reads
<code>dashboard/services/data_contracts.py</code>	214	Expected columns and types for every artefact
<code>dashboard/services/data_dictionary.py</code>	342	Merged field dictionary for the appendix page
<code>dashboard/services/data_loader.py</code>	311	Cached read-only loading, required/optional handling
<code>dashboard/services/formatting.py</code>	70	Money, corridor and compact-number formatting
<code>dashboard/services/gold_coverage.py</code>	294	Quantity-coverage metrics, persistence and network
<code>dashboard/services/path_resolver.py</code>	222	Repository-root resolution and the file registry
<code>dashboard/services/session_state.py</code>	68	Case selection and filter state
<code>dashboard/services/source_registry.py</code>	161	Official source cards and URL-origin policy
<code>dashboard/services/why_ranked_high.py</code>	218	Log-space conversion, card metrics, evidence-driven summary

`dashboard/components/` holds 18 shared UI modules (charts, tables, filters, cards, banners, KPI tiles, page shell, icons, status badges, styles, empty states, scroll reveal, source cards, evidence panel, boundary banner, controls, page header, landing page).

`dashboard/config/` holds `dashboard_content.yml` (all stakeholder-facing copy) and `dashboard_theme.yml` (design tokens).

F.5 Reports and notebooks

Table F.8. Reports and analytical notebooks.

Artefact	Produced by	Contents
<code>reports/final_analysis_report.md / .html</code>	src/09	The pipeline's own written analytical report
<code>reports/model_card.md</code>	src/09	Model card
<code>reports/data_dictionary.md / .html</code>	src/09	Pipeline field dictionary

Artefact	Produced by	Contents
reports/feature_explanation_table.md	src/04	Approved per-feature derivation, purpose, time-safety rule, interpretation
reports/analyst_briefs.md	src/08	The 5 rendered analyst briefs
reports/brief_validation_report.csv	src/08	Per-brief validation verdicts
reports/figures/*.png, *.dot	src/06 , src/09	Model comparison, hard-negative comparison, SHAP summary, and four architecture/flow diagrams with their DOT sources
data-profiling/ business_Review_Data_profiling_before_re vamp.ipynb	manual	Business review and profiling notebook (34 cells)
data-profiling/ gold_nan_quantity_analysis.ipynb	manual	Gold missing-quantity analysis (24 cells)

F.6 Tests

Table F.9. tests/ — 12 modules, 160 tests, status pass.

Test module	Lines	What it asserts
tests/test_case_summary.py	223	Case-summary service: comparison views, multiples, peer position
tests/test_dashboard_component_contract.py	63	Shared component contract
tests/test_dashboard_data.py	76	Loader behaviour and required/optional file handling
tests/test_dashboard_landing.py	136	Landing route and entry behaviour
tests/test_dashboard_metrics.py	148	Queue enrichment, filters, KPI and landscape metrics
tests/test_dashboard_pages.py	798	Every page renders against the real artefacts
tests/test_dashboard_sources.py	104	Source cards, URL policy and provenance
tests/test_dashboard_theme_contract.py	93	Theme tokens and Streamlit base-theme agreement
tests/test_gold_coverage.py	228	Gold coverage figures, persistence and network
tests/test_model_evaluation.py	358	Model-evaluation reads, integrity checks, no-synthetic-rows control
tests/test_trade_landscape.py	159	Trade-scale and benchmark aggregations
tests/test_why_ranked_high.py	265	Log-space conversion, card metrics, evidence-driven summary

F.7 Root-level files

Table F.10. Repository root.

File	Purpose
README.md	Project overview, pipeline summary, dashboard run instructions, dependency split
requirements.txt	Pinned deployed-dashboard runtime (6 packages)
requirements-pipeline.txt	Full pipeline and notebook environment (24 packages)
evidence_table.csv	Convenience copy of data/outputs/evidence_table.csv written by src/07
.streamlit/ config.toml	Streamlit base theme and server settings
.devcontainer/ devcontainer.json	Development-container definition
.gitignore	Excludes the virtual environment, caches, local-only docs/ , references/ , data-profiling/ , artifacts/ , and generated reports/ except the field documentation